jumped up and said: "But this is wrong! We want everyone in our organization to be conscious that the day-to-day decisions they make directly affect how long it takes to ship product. That's not here at all!"

It wasn't there, of course, because that value wasn't real in the organization—no one was acting out of it. The manager drew a new influence in to represent the new value he wanted to instill. It's part of a manager's job to monitor and manage the values of an organization, so it was natural for him to see the omission and to think he could do something about it.

### PITFALLS

The primary danger with the cultural model is that you'll try to lead the customers where they don't want to go. Are you really supporting the issues they care about? If you're introducing a new value, do you have evidence that anyone cares about that value? The success of the first notes products were limited because they were pushing open, flat access to information on organizations that were, at the time, hierarchical and closed. Only when the notes products started to include controls over the access to information did they become successful. Make sure the changes you introduce will cause someone in the customer population to sit up and take notice—otherwise, you aren't giving customers a reason to adopt your system.

*Don't try to take customers where they don't want to go*

## THE CONSOLIDATED PHYSICAL MODEL

The physical model's primary message is about how physical space constrains what you can do. The world of walls and buildings is hard to change. Within the walls, however, there is room for adapting the physical environment to the needs of the work. People lay things out to meet their needs, define spaces to support the work they do, and otherwise make the physical environment work for them. The physical model shows both the constraints imposed by the environment and the structure people create within those constraints to get their work done (Figure 12.10).

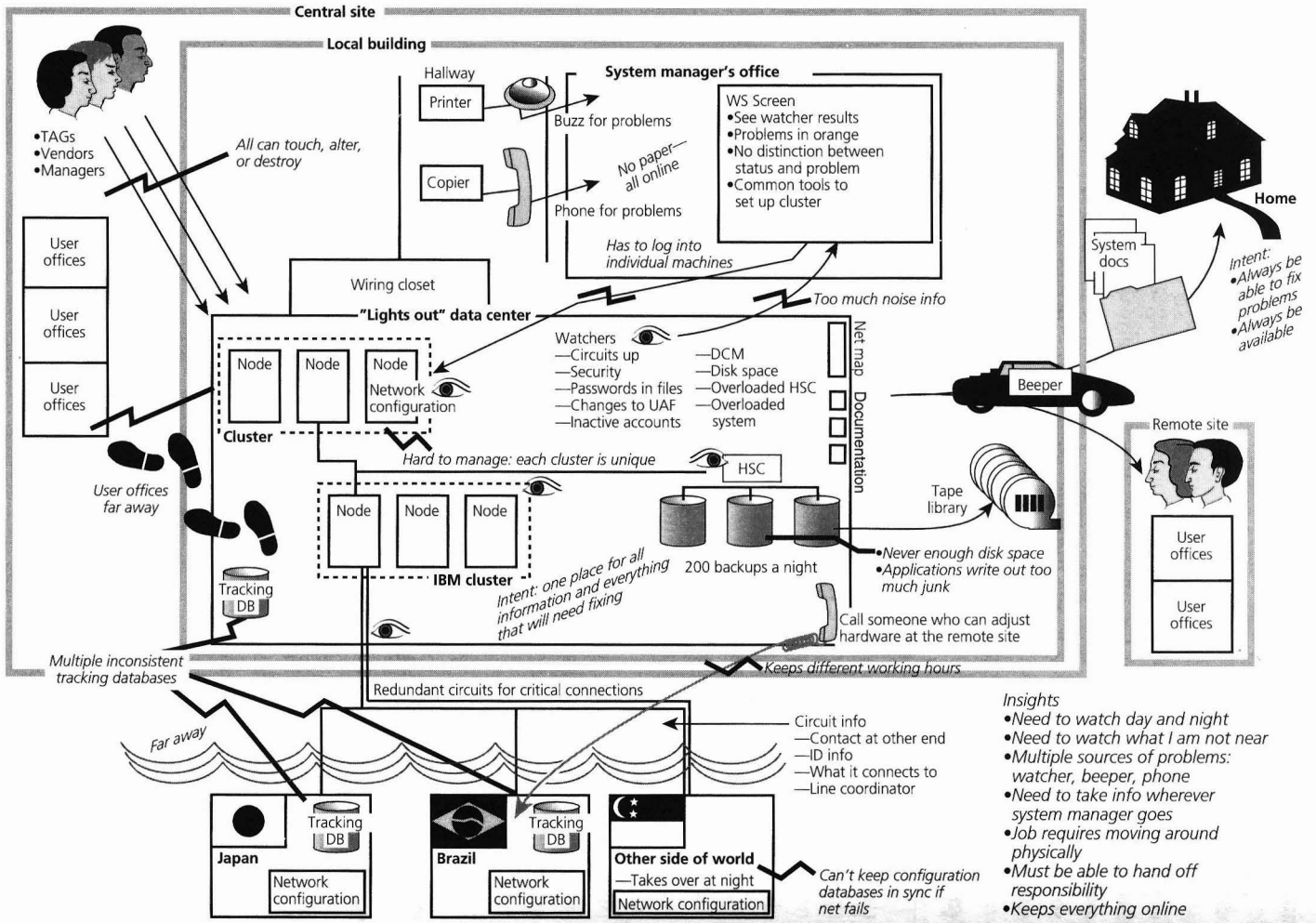

Central site
Local building
•TAGs
•Vendors
•Managers
All can touch, alter, or destroy
User offices
User offices
User offices
User offices far away
Hallway
Printer
Buzz for problems
Copier
No paper—all online
Phone for problems
System manager's office
WS Screen
•See watcher results
•Problems in orange
•No distinction between status and problem
•Common tools to set up cluster
Has to log into individual machines
Wiring closet
"Lights out" data center
Too much noise info
Node
Node
Node
Network configuration
Cluster
Watchers
—Circuits up
—Security
—Passwords in files
—Changes to UAF
—Inactive accounts
—DCM
—Disk space
—Overloaded HSC
—Overloaded system
Net map
Documentation
Hard to manage: each cluster is unique
HSC
Node
Node
Node
IBM cluster
Tape library
200 backups a night
•Never enough disk space
•Applications write out too much junk
Tracking DB
Intent: one place for all information and everything that will need fixing
Call someone who can adjust hardware at the remote site
Keeps different working hours
Multiple inconsistent tracking databases
Redundant circuits for critical connections
Far away
Circuit info
—Contact at other end
—ID info
—What it connects to
—Line coordinator
Tracking DB
Japan
Network configuration
Tracking DB
Brazil
Network configuration
Other side of world
—Takes over at night
Network configuration
Can't keep configuration databases in sync if net fails
Home
System docs
Intent:
•Always be able to fix problems
•Always be available
Beeper
Remote site
User offices
User offices
Insights
•Need to watch day and night
•Need to watch what I am not near
•Multiple sources of problems: watcher, beeper, phone
•Need to take info wherever system manager goes
•Job requires moving around physically
•Must be able to hand off responsibility
•Keeps everything online

**FIGURE 12.10** A consolidated physical model showing a system manager's environment. This model shows the nature of the large-scale physical environment and populates it with important work elements.

The first aspect of the reality check is just to look at the spaces: system management isn't just about the machine room. This is a global network, managed by people on several continents. They all have to coordinate, and they have to hand off problems and control of the network. Even within one site, the spaces aren't close. System managers have to walk between the users' offices, their own office, and the machine room.

Looking at the physical model like an archeologist, the aspect that jumps out immediately is all the different ways system managers create to stay hooked into the systems. Online watchers that send an alert when conditions change, telephones and buzzers by devices that may fail, telephones in the computer room to call other sites monitoring the network—clearly it's critical that system managers feel like they know what state all parts of the network are in at all times.

The message of this model might be summed up in two design implications: First, communicating and coordinating with all the other system managers that keep things going is critical. If a product can help system managers organize and manage their response to system problems and user trouble reports, it will be well received. Second, helping system managers keep tabs on their system is critical—and that means facilitating the communication of problems from users as well as from automated watchers.

## THE REALITY CHECK

The first message of the model is to make it inescapable what the customers' physical world is like. If the work is spread out over several buildings, then expect communication to be a problem. Look for existing communication mechanisms: Are people networked? Is *everyone* networked? What other communication mechanisms exist? Maybe people communicate primarily by yelling to each other. At one site the user talked to the wall and the wall answered back—the partitions were so thin that workers could collaborate even though they couldn't see each other. Is the space noisy or quiet? How many people will be disrupted if the system starts beeping?

The physical model may reveal intents that augment those on other models and may reveal issues that are reflected in other models. The physical model may reveal that your customers walk around a lot; check the cultural model to see if this is a positive value you should encourage ("We know everyone and are always on the spot to help") or an annoyance you should alleviate ("Every phone call means another interruption and another hike"). The physical model may reveal that supplies from different vendors are kept entirely separately; this will explain why the sequence model for working with vendors shows that people work with only one vendor at a time. In this way, the physical model contributes to your understanding of what's important to the work, as well as helping you get real about the constraints your system must live under.

*Know what you can and cannot change*

Design your system to deal with the constraints the physical environment imposes. Allow for the way people move around in doing the work—we've seen people call from a field site back to their main office in another town, asking their coworkers to log them out so they can log in remotely and get their work done. Let mechanisms that work be. If people communicate effectively by yelling to each other, they probably don't need email very much. And don't forget the other side: take advantage of the hardware that is in place. If most of your users already have two monitors, why not use them?

**HINTS**

- Don't depend on what's not there
- Account for movement and multiple locations
- Overcome communication problems
- Take advantage of what is there

## WORK STRUCTURE MADE REAL

Where people can change the physical world to match their work, they build into the physical structures concrete representations of the work structure. Designers can learn the structure of work by analyzing the physical structures people create, just as archeologists learn about cultures by analyzing garbage dumps. When the consolidated model shows a "current work" pile, this is a concrete representation of how people organize their days. A room dedicated to disposing of hazardous materials indicates that how disposal happens is an important concern. Each place—whether a pile, a corner, or a whole room—is a clustering that supports one particular work intent. That intent is real to the users and could be real in the system you deliver.

*Fit with the way people organize their work*

Then the relationship of artifacts and clusters to the user shows what matters in the detailed doing of the work. What's in front of the user, within arm's reach? These are the artifacts that the user chose to have "in their face"—they are the critical things to have handy. What's behind the user, pushed out of the way? These need to be accessible, but they don't need to be immediately to hand. If the user is technical and much of their work is online, look at the screen and how it's laid out—it will capture much of the organization of work.

Look at the structure built into the physical world for clues into how to build the system. When the structure exists because users made it that way (the physical environment didn't force it to be that way), it's a structure that matters to users in organizing their work. Build that structure into the system and you'll support the work better. When people create a "current work" pile, it says that the primary organization is "what I am working on now"—not by project. If you design a system that organizes work only by project folder, it will fight

the way people think about their work. Look at placement to determine the detailed structure of the system. An artifact that is pushed out of the user's way probably shouldn't be the most prominent thing in the system, but things kept at hand can be easily accessible in the system.

**HINTS**

- Build conceptual structures into the system
- Match the intent of the place, not the detailed appearance
- Make the things in the user's face easily accessible
- Put things placed behind the user out of the user's way

## MOVEMENT AND ACCESS

The pattern of movement of people and artifacts through the physical environment provides another layer of insight. The flow of an artifact through a person's office shows the important stages of working on it and indicates what stages an automated system should support. The movement of people through space shows important relationships in the physical environment—so frequent movement between the different system consoles in a lab indicates that the work is on all the systems together, not on each system individually.

Finally, the relationship between spaces reveals distinctions and intents. When home offices are repeatedly located up or down a flight of stairs from the rest of the house, it indicates that separating home office activities from household activities matters. When conference rooms are located around the entrance to the site, and none are found near offices, it indicates that meetings are thought to be how you work with clients, not how you work with each other. The arrangement of space indicates its usage, and its usage suggests attitudes, values, or intents that matter for your design. If it's a problem that workplaces are far away, look for ways to bring that work closer through automation.

*Separate spaces reflect work distinctions: don't violate them*

Movement in the physical world indicates how to structure a system. The steps people take to work on an artifact reveal stages of work. Look at the sequence model to see the work structure, and build a

system that matches that structure or redesigns it. If two kinds of work are kept separate in the real world, maintain that separation in the virtual world—people won't want to mix them. Use the arrangement of the real world to find out what matters. But make sure you match the intent, not only the actual arrangement. Putting the office up or down a flight of stairs indicates this is separate work that wants to be physically separate, whereas the separation between phone and Rolodex is only a reflection of physical limitations.

**HINTS**

- Match or improve the flow of artifacts
- Maintain conceptual separation between parts of the work
- Support the intents implicit in the arrangement of space

## PARTIAL AUTOMATION

It's hardly ever possible to put everything online. People increasingly use email, but paper mail still exists. So even if email is easier to file and track, it hasn't gotten rid of paper filing—what it's done is to introduce a new layer in addition to paper filing.

In the system you build, consider whether you've automated everything about a job or whether customers still need paper. When you build an automated ordering system, will people still print and file a paper version so they can track what they've ordered? When you automate scientific methods, will scientists still have to print them to meet FDA requirements? When you make lab orders electronic, do requesters still have to print the order and attach it to the sample? Make sure you've either covered the whole job or that you dovetail with the paper documentation that is still necessary.

*Work with paper—it's not going away*

**HINTS**

- Address all intents of the paper system
- Provide complete coverage in the online system
- Help keep online and paper in sync if paper is still needed

### Process fixes

If you own the physical environment, you have the option of changing it. You can move people and equipment around to make places that support a single intent in the work. You can hook all the people who need to communicate into the same network. You can reorganize offices to better support movement through them. You can give everyone PDAs and install wireless networks. These changes in the physical environment can be part of the overall response, supporting and supported by the systems you put in place. Planning the rollout so that changes to the physical environment are synchronized with process changes is part of designing the corporate response.

*IT: consider moving walls and restructuring space*

### Pitfalls

The easiest way to mess up the physical environment is to not take it seriously. If people don't have printers by their desks, don't build a system that requires frequent trips to the printer. If your users walk around all the time and like it, don't try to tie them to a desk by giving them a product that only runs on a desktop. Check the cultural model to see if they like walking around. Check the flow to see what communication is enabled. The physical model is your guide to what's real—let it drive what you can do in your system. But don't get too literal either. Try to achieve the users' intent, rather than matching the current environment's limitations.

*Don't ignore the reality of the environment*

## Consolidated sequence models

Sequence models make the detailed structure of a work task explicit. They show how the task is broken into activities, the intents that people are trying to accomplish in doing the task, the different strategies people use, and the individual steps that make up the task. Sequences are your best guide to structuring a system to match and extend the way people approach a task (Figure 12.11).

| FIXING A PROBLEM | | |
|---|---|---|
| **Set up to tackle problem** | • Set up place and context to tackle problem | Go to the place where problem can be solved |
| | • Orient self to problem situation | Get more information on problem |
| | | Look at system to see problem in context |
| | • Anticipate need for help | Think about who is expert in this domain |
| **Search for cause** | • Identify cause of problem | Search for problem cause by hand |
| | • Eliminate repetitive tasks | Create and run specialized procedure to search for cause of problem |
| | | Identify cause of problem |
| **Fix problem** | | If a disk problem: |
| | • Minimize disruption of users' work | See who will be affected by work on disk |
| | | Warn users of work to be done |
| | | Wait for users to get off disk |
| | | Dismount disk |
| | • Move substitute HW into place so users can keep working | Find scratch disk or new disk |
| | | Mount new disk for use in fix |



**FIGURE 12.11** A consolidated sequence model showing how system managers solve problems. This is a partial model, but shows some of the major activities and intents in the problem-solving task. The sequence suggests both what issues a design might address as well as how it should be structured to support the user well.

Looking down the set of activities reveals an initial set of concerns a design might address. The very first activity is "Set up to tackle problem." How could a design support getting set up? Currently, system managers have to go to the place where the problem is. Looking at the intents reveals that "setting up" means both going to the right place and getting information and context about the problem. A system that could both provide remote manipulation and reveal what's going on at the failing system might address the whole activity.

In a later activity, "Escalate problem," the system manager coordinates with backup expert help. There are two strategies: one to work on the problem together and the other to hand over responsibility for the problem. Each strategy needs to be allowed for in a system. The system could focus on supporting collaboration between the system manager and the backup help with groupware-style tools so the two can see what each other is doing. Or, the system could support handing off problems, simplifying the process by passing context and history to the new owner automatically. Or the system could do both. The flow and cultural models will offer additional insight as to which solution would be most valued by the customer population.

| FIXING A PROBLEM | | |
|---|---|---|
| **Fix problem** *continued* | • Move substitute HW into place so users can keep working *continued* | Create directories if necessary |
| | | Copy files to their right places |
| | | Mount new disk publicly if it is permanent |
| | | If other problem: process crashes, fix inappropriate message on VTX, create print queue, install SW |
| | • Apply and check fix | Attempt fix |
| | • Get answers to questions | Use documentation to help do task |
| | • Ensure hypothesis about problem is correct and problem is fixed | Determine if fix worked; if worked, go to "Document solution" |
| | | Didn't work, try to figure out why |
| **Escalate problem** | • Get help, either keeping responsibility or passing it on | Can't figure out problem or not my job to fix problem; call on experts |
| | | If trying to fix on phone: |
| | • Apply advice from expert to solve problem | Decide on fix on phone (go to "Attempt fix") |
| | | If expert needs to see actual system: |
| | • Make it possible for expert to solve problem | Give information for expert to look at problem |
| | | Check site documentation of setup to determine how to identify failing HW |
| | | Give experts information necessary to locate HW |
| | | Wait strategy 1: partner in fix |
| | • Save time and boredom, and maintain responsibility for the solution | Look for problem in parallel to experts (go to "Search for cause") |
| | | Wait strategy 2: give responsibility to expert |
| | • Save time by passing responsibility and doing something else | Do something else while they handle it |
| **Document actions** | | Document solution |
| | • Track work done and changes made | Document solution and actions taken |



**FIGURE 12.11** *continued*

| FIXING A PROBLEM | | |
|---|---|---|
| **Document actions** *continued* | • Make sure affected people hear directly | Notify important people directly |
| | • Make self a person to clients | |
| | • Make sure problem doesn't happen again | |
| | • Keep from creating future problems | Clean up—get rid of temporary files |
| | • Make sure full process works | Notify owners of other parts of the process to do their part |
| | | Done with documenting |
| | | Done |

**FIGURE 12.11** *continued*

## WHAT THE USER IS UP TO

Every consolidated sequence has a primary intent—the reason why the task was worth doing in the first place. In the end, no individual sequence step matters. You can change, eliminate, or automate steps at will as long as you continue to support the user's intent. There are multiple levels of intent: a system manager's intent in responding to a call is to resolve whatever problem the user is having. But behind that, he intends to demonstrate that his organization has the systems under control. And behind that, he wants to show that he delivers real value to the corporation and should continue to be funded. Each level is broader and addresses more wide-reaching issues than the one before.

Every consolidated sequence has numerous subintents that are accomplished along the way. A subintent allows the user to achieve the primary intent—if you redesign the sequence, you may make some subintents irrelevant. That's okay because they are only a way to achieve a more fundamental intent. One team discovered that part of keeping records of lab procedures was to reduce graphs produced by lab equipment by 50% on the copier so scientists could paste them into lab notebooks. Through the development of an electronic lab notebook, the team eliminated the intent of pasting a paper graph into a paper notebook. They simplified the work to the point that the intent was no longer relevant.

*A system has to allow for all the users' intents*

Secondary intents are achieved in the process of performing a sequence. Unlike subintents, they are important in their own right—if you get rid of the whole sequence, you'll still have to find users a way to achieve these secondary intents. So system managers may depend on users asking them for an IP address to find out what new systems are being added and what's on those systems. No matter what happens to IP address assignment, system managers will always want to know about new systems.

The first design question to ask of a sequence is whether the primary intent needs to be met at all. If the intent of the whole sequence is to assign an IP address and you can automate the whole process (or introduce a network that doesn't require unique address assignment), then you've rendered the sequence unnecessary—you've simplified the job. But before you can eliminate the whole sequence, check all the individual intents that the user accomplishes along the way. Don't worry about the subintents—if you eliminate the need for the task, they become unnecessary. But if you eliminate the sequence, you must find another way for the customers to accomplish secondary intents or your system will fail. If you eliminate IP address assignment as a task, system managers will need another way to find out about new systems.

If you choose to keep the sequence, every intent is an opportunity for redesign. Each intent indicates something that matters to the work. If you can provide a way to achieve it more directly, you can simplify the work. When changing the steps for accomplishing an intent, treat it just as you treat the overall intent of the sequence: look at the part of the sequence you are designing away, and make sure your customers can still accomplish all the intents that matter to them.

*The system's job is to achieve intents more directly*

Get behind the specific actions to understand what the user cares about. One design team was looking at the low-level interaction of users with a word processor. Analyzing the sequences of interaction revealed constant repositioning of the cursor. Users would click one character off their intended target, or lose track of where the cursor was, or be unsure where it would end up if they clicked on different places in their document. One user kept hitting the right and left arrows in quick succession. "He's just twiddling his fingers," said one engineer, and that's a natural reaction if you're not used to looking at pattern.

But other engineers on the team were used to looking at pattern, and what they saw was a recurring theme—positioning the cursor was a low-level but constant irritation and an impediment to getting work done. And this itself was part of an overarching theme of glitches and problems in the low-level interaction with the system. The team adopted a design direction of cleaning out all these glitches to make the interface disappear as a problem, including better ways to provide feedback on where the cursor was.

**HINTS**

- Render the primary intent irrelevant
- Support the primary intent a new way
- Account for all secondary intents
- Redesign to support achieving subintents

## HOW USERS APPROACH A TASK

Where customers use different strategies to accomplish work tasks, the consolidated sequence models show what those strategies are. Each strategy indicates a different approach to the work, driven by different circumstances or values. The different strategies may be adopted by different roles, driven by different work styles, and may reflect different intents.

Your system needs to recognize the strategies and support them, or introduce a new way of working that supplants one or more of the strategies in use. If you choose the latter option, account for the underlying characteristics driving customers to choose the strategy you are eliminating. You might decide that system managers who continue to work on a problem after turning it over to their backup experts are wasting their time. But they may do this to save face—to prove that even though they had to ask for help, they are still experts and have just as good a chance of finding the problem as the people they called in. They may do this because they really want to have someone to talk to about the problem while they work on it. Or they may just be bothered by the problem and want to find out what the answer is. In these cases you won't be able to keep people

*Support the strategies you know people use*

from hunting for the solution on their own. You'd do better to recognize and allow for it.

**HINTS**

- Identify and support strategies
- Don't eliminate strategies unless you can account for the circumstances where people choose them

## UNNECESSARY STEPS

After you've decided that your system must support the sequence, it is your guide to the structure of the task. Look at the steps of the sequence to reveal the issues for your design. Are there wasted steps? Are there steps you could eliminate? What role could automation have in simplifying the work?

The major activities in the consolidated sequence show the coherent units of work the system must support. Use them to guide the different things your system must do and how they must be arranged to support the work. Look at the transition between activities. Is there a transition between roles as well? How will your system manage the handoff? Does the new activity imply moving to a new physical place? What needs to be taken to this place, and how does the customer make the transition? What, in general, disrupts the transition between activities, and how will you manage it?

*Eliminate and simplify steps*

Look at the steps themselves. Can you simplify them? Where the customer currently takes several steps, can you automate them down to one? Where a step is currently difficult, can you make it easy? Where is the pain, and where is the tedium? Look for ways in which technology can streamline the work, but make sure you don't have to add steps elsewhere (in setup, or loading information to use later) to eliminate them here.

**HINTS**

- Eliminate steps
- Automate steps
- Eliminate breakdowns
- Facilitate the transition between roles
- Don't create work no one wants to do
- Achieve intents directly

## WHAT GETS THEM STARTED

The triggers show how to alert the user that something needs to be done. Pay attention to the style of the trigger. Is it noisy or quiet? Is it appropriate to the work being triggered? Does it work, or is it a nuisance? Choose whether to duplicate the trigger in your system, if it works, or to replace it with a trigger that works better. Look at the difference between users—the designer who can't stand to have the mail icon blink in the menu bar versus the writer who has a dialog box come up to announce each new mail message. Look at how too many triggers defeats the purpose, as when system managers learn to ignore alarms because there are so many and so many are irrelevant. Choose a way to trigger users that works given who they are.

*Create alerts in ways that fit people's needs*

**HINTS**

- Provide triggers for work tasks
- Match style of trigger to appropriate kind of interruption and the user

## PROCESS FIXES

When the work is internal, the sequence model captures the work procedure to redesign. Designing a new way of working means, among other things, redesigning the sequence model so it represents the new procedures. The organization can put these procedures in place directly. It's a typical failing of business process reengineering projects to overlook the secondary intents that are accomplished by

the current process—failing to recognize them, the reengineered process doesn't cover everything that needs to be done, causing people extra work. By analyzing the existing process in this way, you increase the likelihood that you'll recognize and support all the intents that a work process must support. (We'll discuss prototyping new systems and process fixes in Part 6.)

*Introduce new procedures to improve the work*

### PITFALLS

Certain problems are typical when automating and eliminating steps. We've mentioned failing to account for secondary intents—make sure all intents are accounted for when you redesign. But when you automate a set of steps, be aware that users won't trust that you did it right—at least not right away. They're going to want to see what you did until they are confident you won't mess up. Developers used to insist on seeing the machine code produced by their compilers—but as compilers have become standard and dependable, the need to see the machine code went away.

*Reveal the workings of automation to gain user trust*

Also watch out for the amount of extra work your automation introduces. Have you simplified many steps at the cost of vast amounts of setup and customization? Will the user have to set up the system like it is a separate task? If so, will any real users do this? And look at the amount of work it takes to interface with and maintain the system. Have you introduced a new role, that of the system babysitter, or feeder? If so, will your users be willing to adopt those roles?

## CONSOLIDATED ARTIFACT MODELS

Artifact models show the common structure and intents of the different artifacts used in the work. They are important for showing both the detailed conceptual structure underlying a task and how that plays out when it's made real in the artifact.

### WHY IT MATTERS

Just as sequences exist for a reason, artifacts exist for a reason: they enable customers to accomplish something they care about. There will

be one or more intents for the whole artifact, and then each of the parts may suggest additional intents.

Look for ways to achieve the intents more directly in the system you design. When you see a report passed back with notes and questions scribbled in the margins, you know it supports discussion, not just reporting. Consider supporting the communication directly through email and bulletin boards. But make sure you support all the intents—you have to support them all before you can get rid of the artifact. If you put an existing artifact online, pay special attention to the informal uses of the existing artifact—if you make it impossible to dog-ear corners, scribble notes in margins, or tear off bits to pin to the wall, you won't support the work.

*Beware: an online artifact can render informal usages impossible*

**HINTS**

- Support the intent more directly
- Support intents indicated by informal usage
- Account for all intents

## WHAT IT SAYS

An artifact presents information. Look at the data on an artifact for insight about the work. Does a purchase request form provide a field for justifying a purchase, but not for the cost of the item to be bought? That suggests that cost consciousness is not part of the environment. Who uses the information? Does the artifact pass information between people? Does it present the same information to every person, regardless of their role or what they care about, cluttering up the interface with irrelevant information? Is the artifact acting as the communication mechanism between two roles, to pass the context of a work task?

Consolidated artifacts collapse the history of use across all the actual events captured by individual artifacts. As a result, the artifact shows the scope of all the different usages, collecting data, intents, and concepts into one place. In this way the artifact shows the range of variation your design must account for and collects all the intents that matter to the work. It makes sure your design covers all the bases (Figure 12.12).

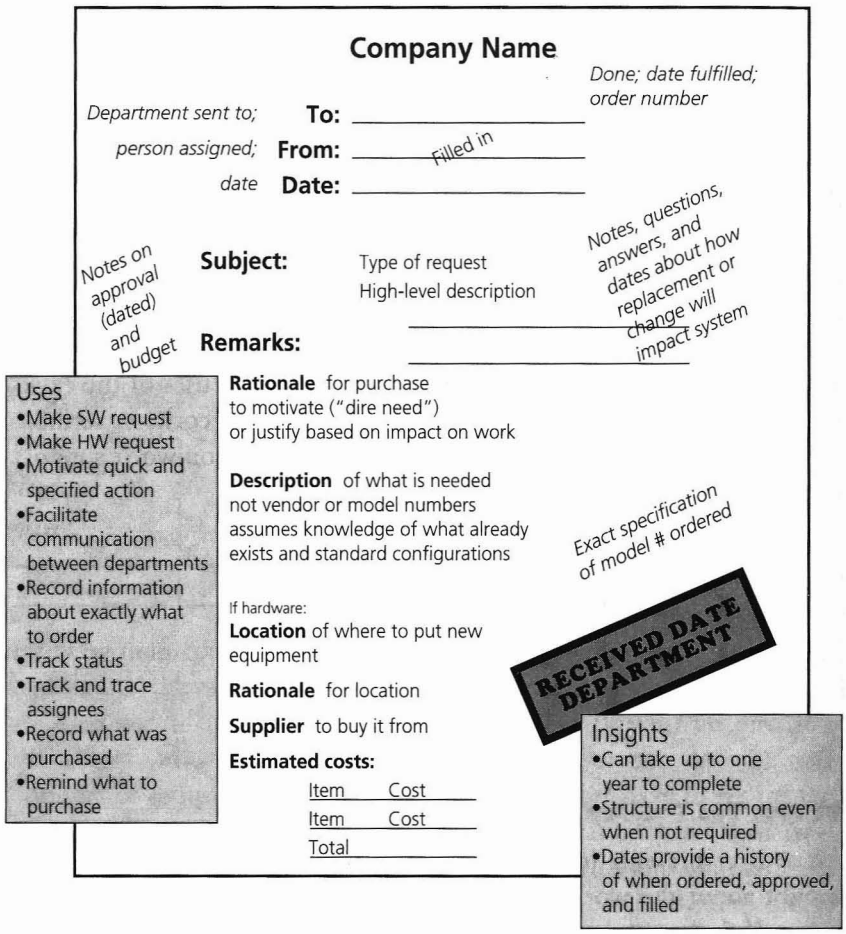



**FIGURE 12.12** A consolidated artifact. This model shows the parts of a purchase request as they might matter to the developers of an ordering system. It shows the parts, but also their intent and how they are used. An automated ordering system would have to support the intents implied by each part.

The model reveals that there are two primary intents to the purchase request artifact: first to justify the need for the item so the requester might actually get it, and second to communicate exactly what to get and where to put it. Automating purchase requests depends on understanding and responding to both intents. However, the description of what to buy is often informal. People assume knowledge of standard configurations and what has been bought before.

The model shows the parts that a purchase request should have, including where to put the item, where to buy it, and sometimes the exact model number to buy. A purchase request that matched a company's formal organization—where the purchasing department decides where to buy things—would not fit the need people feel to say exactly what they want and who should supply it.

Look for opportunities to put artifacts online. If the artifact helps two roles communicate (like a form), can you automate them entirely? Forms tend to capture all the data that anyone might ever need, which means that everyone sees all the data. When the form is used by different people for different parts of a job, the result can be overwhelmingly complex. When you automate the artifact, can you collect all the data in one place, but provide to the different roles only the data they need, so that no one is distracted by irrelevant information? Can you provide information automatically (such as cost center on a purchase request) that is needed by some people but is not important to the requester? And what's the communication the artifact supports? If that communication is discussion, not just data or context, can you support it with communication tools?

*Artifacts should provide only the data people need*

**HINTS**

- Provide data automatically
- Share context between roles directly
- Support communication implied by the artifact

## HOW IT CHUNKS

A consolidated artifact holds distinctions that are indicated by the structure of the artifact. Unlike the distinctions represented by a sequence or physical model, these are extremely particular to the work the artifact supports. An artifact model won't tell you what you should build; but once you have decided that this artifact is important to your proposed solution, it will give you the detailed structure you need to guide the design.

Each grouping of information on the artifact represents a chunk for your system to consider. A form might include routing information that an online version might automate away—but you must make sure that the automated form supports the same kind of routing that the paper form supports (unless you're redesigning that, too). Conversely, our calendar model showed the distinction between meetings and reminders. It showed that notes are associated with specific days. Distinctions like these must be carried over into the new design if it is to work well.

*The structure of the artifact reveals distinctions for the system*

**HINTS**

- Use the structure of the artifact to guide the structure of the system
- Maintain the distinctions that matter to users

## WHAT IT LOOKS LIKE

Artifacts don't consist only of structure and content. They also have a representation, an appearance, which is designed to support the work the artifact is used in. Look at how presentation is used to further the intent of the artifact—or how it gets in the way. When a part is made to stand out, it's intended to catch the eye—how will the analogous artifact in your new system catch the eye? Look at the different ways of making a part stand out. Do they represent different intents, or are they different ways of achieving the same intent? Is standing out the only intent, or are there secondary intents to consider, just as a newspaper headline both stands out and reflects the overall look and tone of the newspaper?

*Presentation matters: it leads the eye and makes parts salient*

Take presentation seriously. It's often treated as secondary, but people work hard to make the things they use look right for the work.

**HINTS**

- Determine the intent of the presentation details
- Mimic the intent of presentation details, not the details themselves

## PITFALLS

Artifacts, because they are real, suggest that every part is needed and every part is relevant. Look beyond the artifact itself to see what's useful. Are all parts of a form used? Is any part of the form used, or is the real communication written in longhand over the top? Even if the data is used, does everyone need it, or would it be better to give different roles their unique views? And make sure that an online version of the artifact doesn't break it up too much. If every part of the artifact maps to a different dialog box, it will be hard to see all the information together.

*Don't just duplicate an artifact online*

## USING METAPHORS

We discussed metaphors briefly in Chapter 4 as a way to think about work structure while setting focus for a project. As you study the work, you may find that these or other metaphors continue to be useful and enlightening. If so, consider redesigning the work explicitly by following the structure of the metaphorical work domain (Kensing and Madsen 1991).

For example, anytime the work you are supporting involves making things—software development is a prime example—housing construction is an interesting metaphor. As a team, draw a flow model of the roles in building a house. Look at the architect's relationship to the homeowner on one side and the primary contractor on the other. How does he or she mediate between the two? How does the architect communicate with the homeowner, and what representations show the homeowners what they will get? Where does the architect's responsibility leave off and the contractor's take over? Look at the emotional tone of the relationships. Architects and contractors frequently argue over what to build and how to build it—contractors have to work out the details of the architect's specifications in lumber and concrete.

*Explore parallel work domains to discover problems and opportunities in yours*

Then use these questions to drive how you restructure the actual work you are supporting. Where is the architect role in software development? Do "software architects" play the same role as building architects? Do they create the same sense of partnership with the customer? How would you redefine the software architect role to incorporate more of the user focus of a housing architect? What tools do architects use in working out designs and in managing their relationships with client and builder? Could the intent of these tools be carried over into a system for software architects? Architects use a wide variety of props to help communicate with clients—floor plans, elevations or front views, perspective views, and complete three-dimensional models of the proposed house on its site. Do software architects have the same range of representations available to them? Could you create similar representations to improve the communication between software designers and users? Should

*Metaphors break you out of the weeds of your own focus*

software architects record their agreement with customers in a contract similar to that used by architects?

This inquiry just scratches the surface of the home building metaphor—you'd also want to look at the relationship between architect and contractor, and between contractor and subcontractors, to begin with—but it gives some sense of how to use the metaphor to drive your thinking. You may find that you need to understand the metaphorical work practice better to use it well. (How do architects and contractors really work together on a day-to-day basis?) If it's worth it, do some interviews in the metaphorical work domain; otherwise, find a more familiar metaphor.

Look for work domains that parallel the domain you're designing for. One team supporting home finances decided the roles were like the pilot and navigator on a plane: one person did the day-to-day work of keeping the finances on course, while the other got involved when deciding how, over the longer term, to get where they wanted to go. Another team decided that the order process in their company was like asking someone else to shop for you in a household. All the problems they saw in the interactions with the purchasing department mirrored the problems of getting someone else to understand what you want. Recognizing such a metaphor gives you a handle on how to support the work in new ways.

*Steal and transform ideas found in a metaphor*

## USING MODELS FOR DESIGN

These discussions of work models and parallel work practice should give you a guide in thinking about the implications of existing work practice for design. The models lay out different aspects of work in front of you so you don't have to hold it all in your head; doing the inquiry into one model after another helps you synthesize across the models, see overarching issues and pattern, and begin to put common solutions together. Discussing the models and possible metaphors in the team leads to shared understanding and perspectives. Through these discussions, teams start thinking about the design response, not just by responding to specific work problems with specific features, but by responding to the whole

*Prepare the team brain: inquire into models and metaphors*

work situation with a coherent system. By the time they get to actually designing, the team is so steeped in the data that they can't help but respond to it.

Through these activities and discussions, the team together works down the chain of reasoning from the facts in the work models, through interpretations, implications for the design, and finally specific design ideas. We'll present an orderly process for doing this in the next chapter.

# 13 *Design from Data*

Going from customer data to a design requires a creative leap, a leap from what matters to what to do about it. Customer data never dictates exactly what to design. Any set of facts can be taken multiple ways, used to inform different kinds of decisions. A product designer looking at a salesman's role might see how to provide information and tracking tools appropriate to life on the road. The division manager might see the frustrations and constraints of the job and how to alleviate them through training and communication sessions with individual salespeople. Upper management might see the constraints imposed by the organizational structure and how the sales role and relationship of sales to the rest of the organization might be redefined to make them more successful. Each different point of view reveals a different set of issues and different solutions.

The range of solutions a design team considers depends on who is on the team and the perspectives they take—the skills and knowledge they have available to them, the charter they think they have from management, and their shared assumptions about what they are up to as a design team (Gomaa 1983). Teams can't invent solutions that they don't have the knowledge to create, don't feel they have permission to carry out, or don't see as being their job. Shrink-wrap software developers won't think of restructuring the organization as part of their design—but even a team chartered to reengineer a business process won't think of restructuring the organization if they don't have the skill to see and design process and if they don't have the backing of management. A cross-functional team makes the widest possible set of skills and perspectives available, and increases the range of solutions they can consider. But that solution creates its own problem: team members tend

*Teams can't invent solutions that they don't have the knowledge to create*

to pull in different directions, with individual members emphasizing the issues and ideas they see. It's up to the design process to unite the team behind a single corporate response. (See Kelley and Hartfield [1996] for further discussion of using multiple points of view to drive invention.)

Getting the team to be creative is tricky. We want the team to think widely, "out of the box." Yet it's in an engineer's nature to immediately do a feasibility estimate of any idea they hear of or invent. That's why they respond so frequently, "We can't do that." Until the entire design for doing it is worked out, the idea does not seem doable. Then the same engineer who said it was impossible Friday will come in Monday morning and announce that it's done. It's not possible to be creative when every idea gets immediately put to the test—and a truly creative idea may well require a substantial time to investigate whether it can be done or not. We often find that the idea we thought was a pipe dream when it was first mentioned turns out to be easy when the implementation is designed. So there's no advantage to filtering ideas early.

*Evaluation stifles creativity*

On the other hand, part of being free to think widely is to feel secure that you won't be committed to implementing the things you think up. We encourage people to think broad, wide, and radical first, without worrying about how to implement their ideas or fit them with existing products. Once you've had the radical idea, you can reduce it to its core intent, decide what's important about it in supporting customers, and scale it back to what's practical in limited time. Following invention, the process provides many evaluation steps within the team and with customers to ensure that the design works for the customer and can be implemented by the people in your organization. Knowing these steps are coming frees the team to step outside the bounds of what they know to be safe.

*Knowing that evaluation will happen later sets creativity free*

In Contextual Design, a team walks through a series of activities intended to get them over the hump of having a broad understanding of their customers' work practice but no agreed solution, to a clear sense of what problems to address and an innovative design to address them. Contextual Design provides a set of steps, linked together into a process, that move the team to a concrete representation of their shared direction, or vision:

**Walking the data:** to see the different aspects of work and synthesize them mentally

**Visioning:** to invent multiple possible responses to the data

**Evaluation and integration:** to develop a single corporate response

**Concurrent action:** to move all parts of the organization forward in parallel

## WALKING THE DATA

The first activities are designed to explore the data and its implications for the design. At this point we aren't looking for specific design solutions; we just want to enable team members to think about the data in detail and explore all the different ways they might respond to it. Just as we set focus before going on an interview so people know what to look at, we use these activities to set the team's focus for design so they know what to build. When the customer data is understood and internalized, team members will find it natural to design solutions that respond to the primary issues it raises.

The first activity for immersing yourself in the data is to read the affinity from end to end—what we call "walking the wall." Walking the affinity right before visioning ensures that the customer issues are fresh in the designers' minds—that the solutions they invent will be grounded in the customers' work practice. Then when they review each other's ideas and see how other people are reacting to the data, they start to build a shared sense of how to respond. We discussed the detailed process of walking an affinity in Chapter 10—everyone who will be involved in the visioning session walks the affinity this way before visioning.

*Anyone who visions must be steeped in the customer data*

After walking the affinity, the team uses the consolidated work models to do the same kind of thinking as the affinity on the different perspectives on work. Each model represents a different point of view, a different dimension of work practice. When people walk one after another, they naturally synthesize all the different dimensions into a single three-dimensional picture of the customer. The previous chapter discussed in detail the kind of issues the team might consider for each type of model; designers do this individually or in small groups, discussing the model and how they should respond as a team. Each

model will generate a set of goals: values to encourage; negative feelings to eliminate; roles and activities to support, combine, or eliminate; and so forth.

Once individuals or small groups have discussed each model, they share their discussions with the rest of the design team, and the team marks parts of the model that they want to support or eliminate. At this initial stage when the team is still deciding on a design direction, they are more interested in the "what matters" type of issue than the structural issues. So they look at flow and cultural models in detail; they look at constraints and primary intents on the physical; and they look at intents, activities, and strategies of high-level structure in the sequence and artifact models. As they read and discuss each model, issues from the other models and from the affinity are naturally incorporated into their discussion. What started as point solutions to individual problems weave together into a synthetic response to the whole work problem.

Walking the affinity diagram and work models focuses the team on specific aspects of work they want to respond to. The team can have an explicit and public conversation, recording the issues right on the affinity and models. You can include others in the discussions by allowing them to participate in reading and responding to the models. And they ground your vision for redesign in real work issues.

*Walking the data creates a team focus for the vision*

After walking the affinity and each model, crystallize your thinking by making a list of the most important issues from that model. This gives you a single, crisp statement of the issues that you can return to as a reminder of your focus for the vision. When the lists are made, the team is primed to start the vision.

## PRIMING THE BRAIN

Before starting with the visioning, the team brainstorms two lists, with no evaluation or filtering, that will be fodder for the vision itself:

**Technology:** Any design response uses technology to solve work problems. To bring the technology they have available to mind, teams list all the technology they might draw on. This list incorporates the mundane (networks, World Wide Web), specialized technology unique to the company (artificial agents, CAD diagramming),

*Lists bring possibilities to mind*

and implementation approaches that the team might otherwise not think of (process design, business partnerships). Anyone on the team who doesn't know about any of these possibilities can learn about them at this point.

**Starting points:** Discussing the work inevitably involves discussing how the team might respond to it. These initial discussions are starting points for the vision. In this list, we capture some of the most important starting points that people don't want to forget: design ideas that have captured the imagination of team members, a slogan that the team wants to commit to, or a metaphor for what the work could be like. "Put the system manager's whole job on a PDA." "Ordering should be one-stop shopping." "The lab should be like Federal Express tracking packages—you always know the state of every experiment." "Do shopping like bumper cars." Each of these ideas is a seed, a starting point for the team to elaborate into a whole approach to a design problem.

## CREATING A VISION

We call our visioning process a grounded brainstorm—"brainstorm" because ideas are not evaluated as they are generated and "grounded" because ideas are driven by the customer's work practice. A visioning session gives a team the chance to choose a starting point and spin it out into a story about the new work practice transformed by technology. The story describes the brave new world the team envisions—without committing them to actually building it. (Greenbaum and Kyng [1991] describes a variety of approaches to inventing new work practices.)

*Choose a seed to start the vision*

In the visioning session, one person (the "pen") stands at a flip chart, drawing the ideas as participants throw them out. The pen has two roles: encourage people to talk, but also fit their ideas into the vision as it is developing. Unlike a normal brainstorm, where each idea is independent, a vision session starts with one of the ideas from the list of starting points and incorporates each idea into a coherent story about the redesigned work. The vision is a drawing showing what the new work practice would

*The "pen" weaves the team's ideas into a story of new work practice*

be like if the vision were in place (Figure 13.1). It shows people in the roles they play, the systems they use, how they communicate with each other and the systems, and how the systems are structured when that's necessary to thinking about the vision. Vision pictures are very informal—they are drawn quickly, without a lot of structure. They tend to have lots of arrows showing communication, lots of faces showing people, and lots of boxes indicating screens, systems, or other technology components. They aren't restricted to the system being designed but may include the delivery mechanism, third-party relationships, and additional services that work together to make the vision possible.

Any vision has a thread, which starts with the initial starting point and then is played out as participants expand on it. A facilitator helps participants pursue a thread by tying together ideas into a story of work practice and suggesting additional issues from the work models or affinity, additional roles from the flow model, or values the team agreed to from the cultural model:

*The facilitator triggers inclusion of issues the team identified*

**Pen:** *So we're starting with the idea that shopping is like bumper cars.* (Draws a bumper car.) *What happens?*

**Designer (D1):** *Well, the whole idea is to get the kids involved. So the kids have to be able to drive.* (Pen draws a kid at the wheel.)

**Another designer (D2):** *Yeah, put the adults in the passenger seat. Then organize the store so you can drive through it in order.* (Pen starts sketching the store.)

**D3:** *And make the aisles narrow enough so you can pick things off both sides as you drive by.* (Pen draws aisles on either side of the car.)

**D1:** *You'll have to make all the aisles one-way.*

**Facilitator:** *But what about backtracking? We saw people have to go back for things they forgot.*

**D2:** *We'll give them a way to backtrack at the end of the aisles.*

The facilitator and the pen should listen for ideas that are on the thread, postponing ideas that are too far off the main line to be starting points for another vision. When an idea conflicts with the thread the team is working on, the pen adds it to the list of starting points.

This keeps the thread coherent while assuring the team member that his idea has been heard and will be dealt with. Eventually the thread will play out—people won't have more ideas for extending it without taking a new starting point. At this point the vision is put aside and the next one started. Don't duplicate ideas from vision to vision—good ideas will be recovered in the next step, so you don't need to go through them again.

Practicality is not a major consideration for a vision. If the team lets go of worrying about whether they can build their ideas immediately, they will be more creative and produce a vision that will account for more of the work practice, more coherently. Our team above is unlikely to ship a product that installs bumper cars in grocery stores, but working out the vision gives them a chance to explore issues of child control and participation that are very likely to be part of their final design. Balance creativity against practicality. After visioning bumper cars and some other fanciful ideas, the team might want to evaluate and consolidate them into a more conservative vision capturing the key benefits but using technology they think they can implement.

Any vision will specify more than the team wants to attempt in a single version, but that's all right—it means the team will have a plan, a strategy, that they can use to drive delivery over several versions. (We'll talk much more about strategic development over several versions when we introduce the User Environment Design in Part 5.) Even if you know you are focused on a short-term deliverable—say, your next update due to ship in six months—you're better off thinking and visioning widely first. Then you can either synthesize and pick and choose the best parts for the deadline, or you can vision widely and then vision explicitly for a six-month deliverable. You'll find you automatically pull in ideas from the wider visions to put together a coherent plan that you can do in the time you have (once you do the inevitable trimming).

*A vision encompasses more than the team can ship in one version*

A good visioning session is a lot of fun. Everyone is tossing in ideas for what to do based on what matters in the work. Everyone is talking at once and building on each other's ideas. The major gating factor for a visioning session is the ability of the pen to draw what he or she hears without a lot of filtering or explanation. If people feel like it's too hard to get their ideas on the paper, the

*In a good visioning session, everyone feels their ideas have been heard*

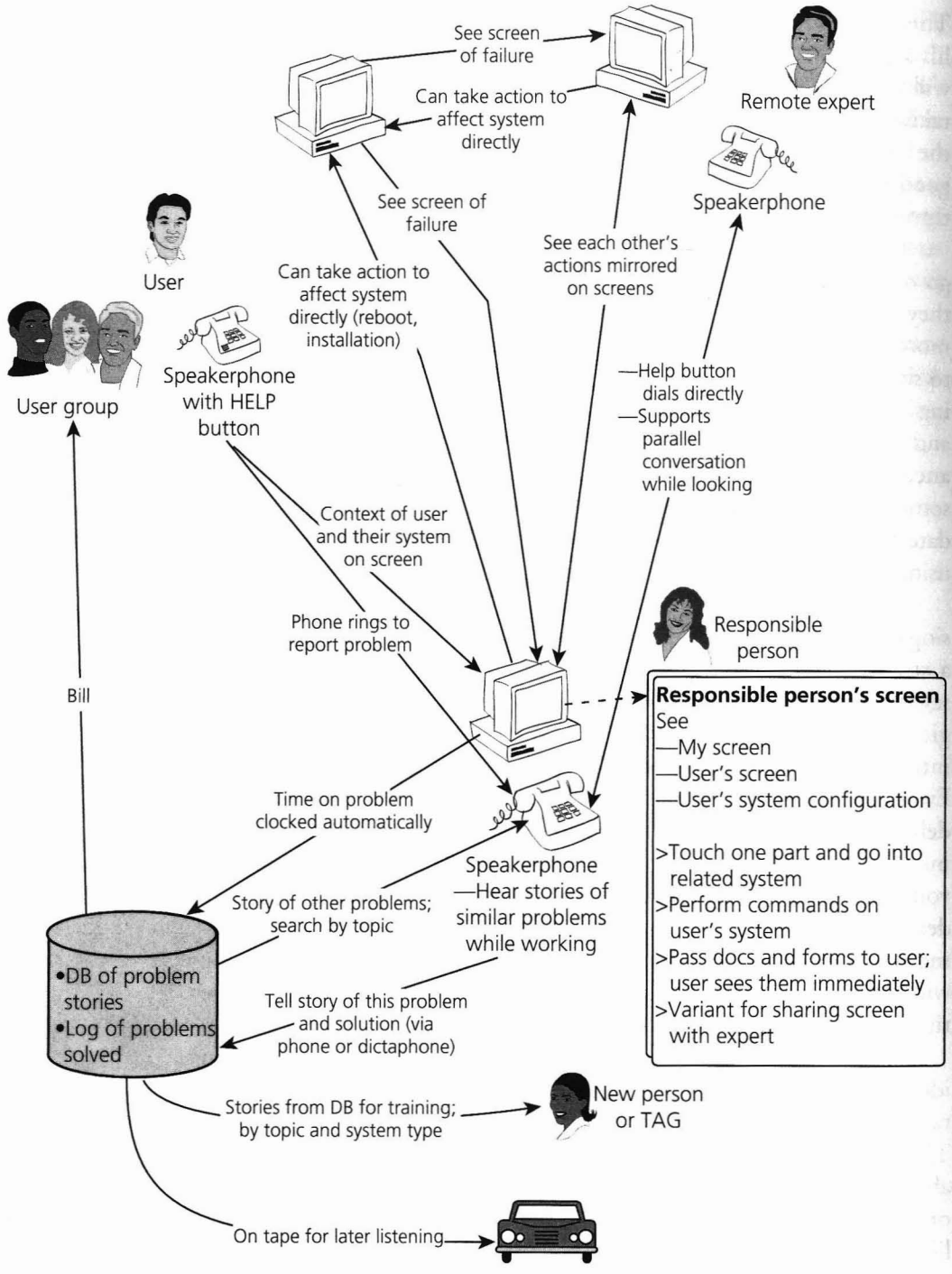

See screen of failure
Can take action to affect system directly
Remote expert
Speakerphone
See screen of failure
See each other's actions mirrored on screens
User
Can take action to affect system directly (reboot, installation)
User group
Speakerphone with HELP button
—Help button dials directly
—Supports parallel conversation while looking
Context of user and their system on screen
Phone rings to report problem
Responsible person
Bill
Responsible person's screen
See
—My screen
—User's screen
—User's system configuration
>Touch one part and go into related system
>Perform commands on user's system
>Pass docs and forms to user; user sees them immediately
>Variant for sharing screen with expert
Time on problem clocked automatically
Speakerphone
—Hear stories of similar problems while working
Story of other problems; search by topic
•DB of problem stories
•Log of problems solved
Tell story of this problem and solution (via phone or dictaphone)
Stories from DB for training; by topic and system type
New person or TAG
On tape for later listening

**FIGURE 13.1** A vision for system management.[1] In this vision, the team has elected to focus on improving the communication between user and system administrator and on improving the diagnostic process. The vision started with the idea that the system administrators wanted to bring the problem to them, to make everything necessary to solve the problem available locally. In following the story, the team integrated a number of other issues—how to make it easy for users to get help, how the system administrator can get backup help, and how to use stories in diagnosing problems.

In the vision, when a user wants help with a system, they just push a big red "HELP" button on their phone. That automatically connects them with the right person for their system and organization and brings up their system information on the administrator's screen. The team wanted to make asking for help through the "approved" mechanism so simple that no one would be tempted to use personal or informal contacts to go around it. They found that having enough context about the user and system was a major impediment to administrators in providing support—researching the system was always the first step toward doing any real work—so they had the system provide as much context as possible. When the time comes to implement, the "HELP" button will probably not survive as visioned—it's not reasonable to make changing every phone a prerequisite to using this system administration software. But the idea will prod the team to think about simpler ways to achieve the same intent—perhaps stickers to put on the phone with the right number, or autodial in the software for computers that support it. The vision is a stake in the ground saying, "This is the goal." The team can scale back and decide on reasonable ways to achieve the goal later.

To support diagnosis, the team noticed that system administrators depend on story and anecdote a lot when troubleshooting. Stories capture knowledge about what might work in different situations, but capturing the stories and making them available is hard. Typically this is done through tale swapping in the informal system administrators' community. Even when people capture a log of what they did, it doesn't have the same flavor as a story—it doesn't include the different alternatives tried and the frustrations of trying to work things out. This vision attempts to capture stories and make them available when needed, while diagnosing another problem, through a database of stories. When done with a call, an administrator can tell the story of what happened into the phone—they are used to dealing with the phone. Later, they can search the database for stories relevant to the problem they are working on—perhaps from the same system or showing the same symptoms. Then they can listen to the stories while working on the problem. The challenge of the "story database" is not so much technical—it just depends on recording and playing back speech—as it is being able to capture enough information about each story so that the subsequent search picks out relevant stories.

This vision shows how even radical solutions to problems can be based directly on understanding the structure and nature of the work.

---

[1] This vision was generated by us for inclusion in this book. It's a disadvantage of working in a highly competitive industry that most of the data used to develop actual products and internal systems is considered too proprietary to release.

session will be frustrating for everyone. For the same reason, limit sessions to about 10 people—more makes it hard to get air time.

Expect to elaborate on each idea for about half an hour, then move on to the next. Keep going until you have at least three or four alternative visions, each on its own flip chart paper.

## CREATING A COMMON DIRECTION

Doing multiple visions lets the team consider alternatives and work out some of their implications. Each vision is built by the whole team and incorporates everyone's different perspective. But at the end of a visioning session, you have multiple visions, each suggesting a different design direction or addressing a different part of the work. How do you choose among them?

In Contextual Design, you don't have to. Instead, you synthesize a new solution incorporating the best of the individual visions. Committees have the reputation of producing mediocre designs because people compromise; instead of doing either of two reasonable designs, they settle for something halfway in between, or they incorporate a few features to make everyone happy. Synthesizing a common vision is a way to avoid this. Rather than compromising on features, producing a design with a little something for everyone, the goal should be a design that is coherent and clean and that supports the work issues everyone identified.

*Creating a common vision is not a compromise*

The key to such a design is to treat each vision, not as a monolithic block that must be accepted or rejected as a whole, but as a collection of options that can be reconfigured and redesigned into a single solution. If the team had to choose one option over another, they would argue—each person would have their own preference as to how to trade off different issues. But it's a false choice. Every vision will have impractical or undesirable elements; most visions will have some elements you don't want to lose. Create a better solution by identifying elements that work, recombining them to preserve the best parts, and extending them to address more of the work and overcome any defects. The individual

*Identify the positive and negative parts of each vision*

visions become databases of design ideas that you can draw on and recombine to come up with a better solution.[2]

We do this through a structured evaluation of each vision. Look at each one in turn and first list the positive points of that vision—the reason why it's good, fits the customer work situation, solves real problems, is easy to build, or fits the skills of the organization. Even people who dislike the vision overall can find points about it that work; people who are particularly against it are on the spot to identify some points they like. List each positive point on a sheet and attach it to the vision. Then list the negative points—all the reasons why it would be hard to build or would break the customer's work practice. People who love the vision can find a few points to dislike—it will help them to let go of an idea they might be overly attached to. List these negatives and attach them to the vision as well. List positives and negatives for each vision in turn (see Figure 13.2 for an example of the system management vision). While you're listing negative points, people will tend to start solving them—to suggest ways that the potential problem can be overcome. These ideas become important in the next step of the process, but don't let them derail you now. Write them on Post-its and stick them to the vision to save them for later.

*As soon as you list negatives, people start fixing them*

Then look across the visions and at the positive points. Use them to identify the core parts of each vision you don't want to lose. Then look at how to combine these points into a single coherent vision. The team will be primed to do this as a result of the discussion of positive and negative points. They'll already have ideas for how to recombine the vision. Usually, most of the elements of the visions don't conflict directly—because each vision took a different approach, it will be possible to bring the best parts together without conflict. Where parts you like do conflict—two different

*Invention is driven by recombining existing good ideas*

[2] This process is based on the ideas underlying Pugh matrices (Pugh 1991). But where Pugh depends on individual creativity, we use the dynamics of the group to produce a single vision that incorporates everyone's perspective. This helps keep people from getting overinvested in one solution. People who feel they can't be creative in a group situation still have the option of working out a design and feeding it into the evaluation process.

| Positives | Negatives |
|---|---|
| +Tracks time automatically<br>+Provides access to similar stories<br>+It's easy to document actions taken<br>+Fast access to help<br>+System manager is given what's needed to solve the problem<br>+Database of stories addresses the training issue | —It's hard to search through verbal text<br>—What if the user's not in their office?<br>—What if not all phones are hooked in?<br>—Need a realistic way of mimicking the HELP button<br>—What if the responsible person isn't there?<br>—Will people really tell stories of what they've done into the phone? |

**FIGURE 13.2** Positives and negatives for the system management vision.

ways of addressing the same problem, for example, when it doesn't make sense to do both—you'll have to choose. But now it's a very focused choice on specific aspects of each vision. If they both support the work well, choose the simpler or the easier to implement. If you aren't sure which is better for the work, use the ideas to identify what data will help make the choice and set up customer interviews to collect it. (We'll discuss working out the ideas with customers in detail in Part 6.) The final step of visioning is to draw the new consolidated vision reasonably neatly.

*The group process builds consensus and reduces overinvestment*

This whole process is designed to bring a disparate, cross-functional team of people to consensus. If some team member is hooked on an idea, be sure to include that idea in the list of starting points. In one team, one member was hooked on the idea of a large monitor displaying test states in a scientific lab—it had gotten to be a joke in the team that this was his solution for everything. Making the large monitor the core of a vision and then doing positive and negative points (he had to come up with three negatives) made it clear what real advantages the large monitor offered. But comparison with other visions revealed that those same advantages could be achieved more simply. In the end, he didn't have a hard time letting go of the idea.

## MAKING THE VISION REAL

The code is only one component of a product. A commercial product also includes the documentation and services that help people use it, the marketing plan that publicizes it, and the testing plan that ensures its reliability. Internal systems downplay marketing and services, but they still have to help users take advantage of the product, tell them about it, and get buy-in. Internal systems also have to roll out the infrastructure, new procedures, and new organizational structures that will take advantage of the new system. With the vision in place, all functions can start working on their parts in parallel. They can first look to see if they can do what the vision requires at all reasonably; this may require technical investigation or may require going for management buy-in. Once the team knows what's involved in doing a piece of the vision, they can choose to attempt it, leave it out, or scale it back so they get the underlying benefit of the piece in a simpler way. Then, once they've decided what part of the vision to work toward for this project, people can work out in detail what's required for their part. All through, the vision acts as a map that keeps the groups coordinated even while they work independently.

*The vision directs concurrent activity*

### PROCESS AND ORGANIZATION DESIGN

Particularly when the system is for internal use, the vision may imply changes to business processes or business organization. The vision offers a new way of working, and the business structure may have to change to adopt that way of working. Salespeople may have a different reporting relationship to the home office. The purchasing department may no longer be an intermediary in making a purchase. Walls between offices might be knocked down to provide team rooms. Planning for these changes can proceed in parallel to the software and infrastructure development activities that will support them (though, of course, the implementation of any changes must be synchronized).

The vision can help a commercial vendor redefine how they do business as well as what products they deliver. Commercial vendors can mine their visions for implications on new delivery mechanisms, how customer service is viewed and how to improve it, and how to improve the sales channel to address issues that get in the customer's way. If one

customer's issue is how long delivery takes, the delivery service might be changed. If salespeople are used as information resources by their customers, formalizing the information provision as a service might be part of the vision. If technology or products developed by a third party are important to the vision, the organization can start to create relationships with these other companies.

*The vision directs organizational restructuring*

## MARKETING PLANS

Marketing builds the market message around the vision and consolidated models (and later the User Environment Design, described in the next part). The consolidated models show what customers care about and what message will interest them. Marketing can build scenarios from sequences and base the story of the new world on the vision. The vision captures the key innovations that constitute the substance of the market message. And the User Environment Design, especially when organized into components, gives marketing a way to communicate the design as providing coherent support for particular aspects of the work. This message communicates directly to people's experience; talking about features and benefits presents the system more abstractly.

The vision shows what customer characteristics make the difference in whether they will be interested in the product or not. The models capture qualitative data about customers—now marketing needs quantitative data to decide whether the product is viable. The vision drives surveys to size the market and make the business case: test how many customers have those characteristics and how much they are likely to spend. As engineering finishes more of the designing and prototyping, these can be incorporated into the marketing events and used as the basis for focus groups, test drives with customers, and so forth. Marketing can also use the vision for prioritization—they can split it into components and decide which components to ship together for a coherent product, and in what order.

*The vision drives marketing techniques to develop the business case*

## SYSTEM DESIGN

The vision defines what is expected of any software and hardware components of the system. Engineers can get an advance look at what demands on technology the system will make. They can start

investigating technological possibilities immediately, including possible platforms, whether specific technology is sufficiently reliable and whether it can meet the requirements of the vision, and UI possibilities. Then, when the decision is made to proceed, the rest of system design is based on the vision and consolidated models, as we shall see.

*The vision drives technical investigation and hardware requirements*

## STORYBOARDS

A vision drawing captures the new vision as a single picture, showing all parts of the vision together. It says *what* the new work practice will be without showing *how* it will happen over time. But to design well, we want to work out the system design in the context of doing work. We want to see how it fits into the overall work task to ensure that the transition into and out of the system works and that the task stays coherent. We do this by working out the vision in storyboards. (This approach to deriving system requirements from usage is becoming popular in the industry; see, for example, Carlshamre and Karlsson [1996]; Jacobson et al. [1992].)

Storyboards show how specific tasks will be accomplished in the new world. The technique was originally borrowed from movie making and has been used by others to work out system designs.[3] A storyboard captures the new procedure for doing a task pictorially, like a storyboard for a film. Each frame in the storyboard captures a single scene—an interaction between two people, a person and the system, a person and an artifact, or a system step. The storyboard frame might show the people interacting and the content of their interaction. It might sketch a system screen with annotations showing how it's used at this point. It might sketch the artifact and how it's used. Or it might just list the actions the system takes on the user's behalf.

Storyboards are based on the vision, follow the structure of a consolidated sequence model, and pull implications from other models as necessary. The vision defines what the new work is like; the consolidated sequences define the structure that underlies doing a task and

[3] Contextual Design is often useful as a framework giving structure to the front-end life cycle, which other techniques can plug into. In this spirit, we've adopted storyboards as a useful technique for working out a design (Rheinfrank and Evenson 1996).

the intents people achieve in doing it. To build a storyboard, choose a task to redesign that is represented by a consolidated sequence. Then review the models and affinity, gathering issues relevant to this task. Collecting the issues resets your focus, allowing your mind to design from all the issues at once.

*Storyboards tie the vision to the structure of the consolidated sequence*

Then sketch out how you want to redesign the task. This is a more detailed vision, focused on the work of this task and constrained by the larger vision. In this step, you work out the exact approach you'll take to dealing with the different issues. Sketch out two or three options, do positives and negatives, and consolidate one approach. Do this quickly—it's more focused and can be fast.

With the detailed vision drawn out, walk through the consolidated sequence step by step. Look at the intents, different strategies, and steps. Account for the intents—if the first step of diagnosis is to find out more about the system and its history, any new system should account for this need. You could support it, by displaying system context automatically when a call comes in; you could eliminate diagnosis by implementing an expert system that can always figure out what's wrong; or you could decide that they have ways to get context already and you don't need to give them more, and leave this step manual. Look for ways to overcome problems and achieve intents more directly, within the context of your detailed vision.

*Draw all steps: manual, system, and UI*

Capture the work practice as you've redesigned it in the storyboard, including interactions with the system, interactions with other people, and manual steps. The goal of the storyboard is to represent the whole work task coherently, so don't limit the storyboard to only those steps that interact with the system. Sketch storyboard frames to represent each step of the new work practice.

Because each frame of a storyboard is a sketch, it limits the amount of detail the designer can squeeze into that frame. This is intentional, just as the sketchiness of the vision is intentional. By its nature, a storyboard inhibits the designer's natural tendency to dive down into the low-level detail of each part of the system before the whole system has been roughed out. A screen sketch in a storyboard isn't a specification for the UI of that

*A UI sketch shows how people will use a system during a work step*

screen—it's a thinking tool enabling the designer to work out what has to happen in the user's interaction with the system at that point in the task. The UI sketches in the storyboard communicate ideas to the UI designer, who will create a consistent and comprehensible UI for the whole system later.

We usually build storyboards in pairs. After finishing the whole task, the pair brings the storyboard back to the whole team for review. They walk through the storyboard, and everyone posts issues on the storyboard: mismatch with the customer's work practice, mismatch with the vision, alternative design ideas, or implementation worries. The pair then reworks the storyboard to account for the issues.

A set of storyboards for the key tasks to be supported in a new system defines how the system will work in supporting those tasks. By telling the whole story of the task, including manual steps, automated steps, and interactions with the system, the storyboard keeps the work task coherent. Storyboards work out elements of the vision by unraveling them, laying them out step by step. They provide the next level of detail for the design. In working out the storyboard, all aspects of work come together: roles interact, people move around and pass artifacts, and culture influences everything. The storyboard synthesizes all these issues into a coherent redesign of a work task within the context of the overall vision (Figure 13.3).

*Storyboards capture all the steps needed to do the redesigned work*

## REDESIGNING WORK

This is what it means to redesign the work: First, understand the structure of work as it exists and the issues implicit in the work. That will tell you what to address. Become knowledgeable about possibilities for redesign, either by learning about different possible technologies or bringing experts into the room and steeping them in the data. Then vision a new world, using the knowledge of the team and building on your understanding of the issues. Once that's done, you can work out the implications of that vision in storyboards that show individual instances of doing the work.

*The vision and storyboards guide the corporate response*

The vision holds your corporate response. It shows how all the different actions you might take as an organization work together to

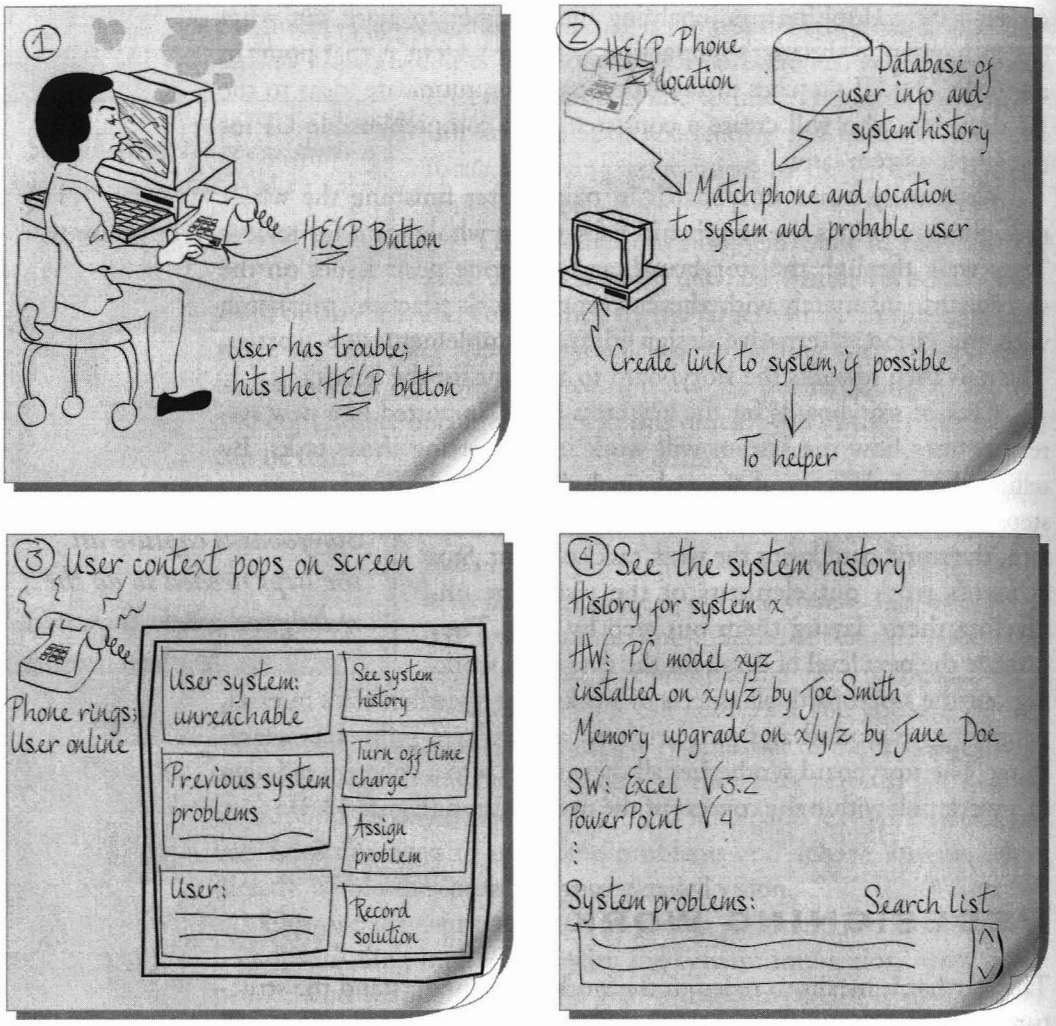



**FIGURE 13.3** A storyboard for the system management vision. This storyboard follows the structure of the consolidated sequence for dealing with a call for help. At each step, the storyboard reinvents how that step would be accomplished given that the user can take advantage of the new system.

address the user's work problem. With a vision in place, the different functions of your organization can work together toward a common goal. Each function then will follow its own process to work out the implications for its part. In the rest of this book, we'll discuss the process for engineering to develop the system itself.

## KELLY'S STORY

Everybody talks about spending a day in the life of the customer, but it's very hard to do and get meaningful data. I've tried a lot of different processes and have found that the people who participate get insight, but there's no way to capture it, communicate what you learned, or use it again on the next project for the same market. Contextual Design is unique in that it gives a framework and models to capture the data in ways that are more meaningful than any other process I've used. It supports sharing and communicating the insight and lets others participate in a way they cannot otherwise.

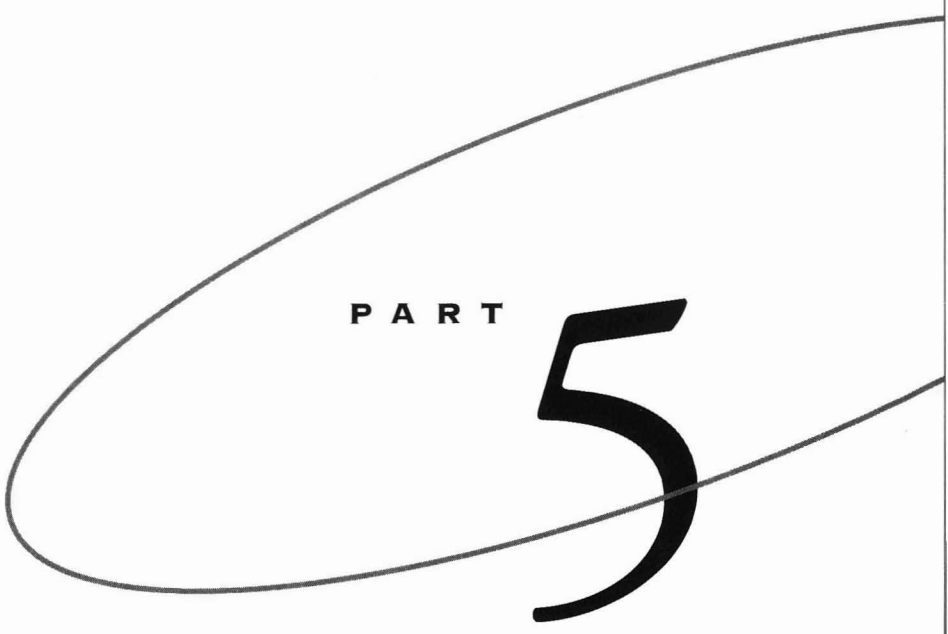
I'm a product manager in a group building large computer systems. We wanted to address a new market segment with our product line, but we had not spent much time understanding what was important to these customers. We used the whole Contextual Design process, with a cross-functional team drawn from marketing, R&D, and manufacturing, and external facilitators to guide us.

We interviewed about 15 customers; consolidated the models and affinity; visioned new product, service, and delivery ideas for the market; built a User Environment Design for some parts; and used prototypes to test the ideas. We found the cultural model particularly useful for defining our marketing objectives, including defining the value proposition for the market. The other models were more useful to R&D in building the actual products.

The vision we produced drove marketing requirements for several different organizations. Producing a computer system for this market requires the combined effort of several software organizations, two hardware organizations, manufacturing, and service organizations. For example, we discovered how critical it is in this market to be able to add capacity easily—this recognition drove hardware requirements for ease of installation, software requirements for dynamic reconfiguration, and a manufacturing requirement for 48-hour turnaround of a new order. That will require process changes in the manufacturing organization, but they've put together a task force to figure out how to do it.

For us in marketing, the strongest part of the process was through visioning. This project wasn't driven by the engineering organizations, so we didn't have enough commitment to the results. No one organization felt their business depended on the success of the project. We could get individuals to buy in, but it was harder to affect the plans of the whole organization. So we found it easier to communicate the results of the vision using our traditional methods. We are getting the changes we designed into products to be delivered over the next 12 to 18 months.

I have to say that none of us had a clue how much time and energy this project would take, particularly since we had no data to start with. The results were well worth it, though. If we hadn't done this work up front, we would have had to do it later, when plans are harder to change. We're planning to use Contextual Design on our next project, but augmented with other marketing techniques and contextual interviews with people who make the buy decision. ❑




# System Design

# 14 *System Design*

We've understood the user's model of work, we've captured it in work models, we've envisioned new ways for people to work—but so what? How does this help us with software design? Way back in Chapter 1, we discussed the idea of a *system work model,* the approach to doing work that's built into every system. The vision of Chapter 13 defines a new way of working, with many delivery mechanisms. IT shops can define new roles and procedures in concert with the business partner; commercial product developers can define services and training. But in this book we're focused on software and hardware systems, which embody the desired system work model. How do you make the transition from the vision and storyboards to a system design that delivers on their promise? In Contextual Design, we introduce a new modeling technique to reveal the system work model and show how all the parts of the system relate to each other in the user's experience.

## KEEPING THE USER'S WORK COHERENT

The challenge is to keep the system work model *coherent,* so that it supports the users and fits with their expectations while extending and transforming their work practice as prescribed by the vision. Coherence isn't just about consistency of the user interface—a coherent system keeps the user's work orderly and natural. When a presentation tool won't let its users change slide notes and slide contents at the same time, making them jump

*Design challenge: to keep work coherent by keeping the system coherent*

back and forth between views, it breaks up the work. When a word processor provides three successive cascading dialog boxes to choose a bullet, it turns a minor function into a whole task, complicating the work. When an email system lets users search the address book by providing a simple text entry field that filters the address book names but uses a separate query window to search the "sent mail" folder (Figure 14.1), it provides inconsistent structures for doing similar work. When the system work model is coherent, it keeps the user's work coherent; when it fragments, it's the user's work that is disrupted.

Keeping the system model coherent is hard enough when it's one user doing a single task. It's even worse in real systems, which support multiple people playing multiple roles, across departments or the whole business, while using several systems. One user we talked to was verifying information given to her by another department. The information on the form was accessed by several different applications. By the time she was done, she had used 11 screens in four applications to check a single form. Another user wanted to see what drugs a person was taking while recording a clinical event. His information was online, but he had to leave the application he was using and get into an entirely separate one to get at drug records. In both cases, the users had multiple systems, each designed to solve a single problem. These systems didn't address the user's whole job and didn't attempt to make the work fit together across the different departments or tools. When work practice is too large and complex to see, or it's too hard to address all at once, it's easier to write simple systems that address single problems. But then the systems chop up the work and leave it up to the user to put it back together by taking extra steps or doing additional work on the side.

*Users have to be the glue between incoherent systems*

*In the face of work complexity, designers create simple solutions*

From the developer's side, the picture is no easier. Software development organizations start projects to address specific problems, and only later do they realize that the systems don't hang together and don't build up into a coherent solution. This leads to conflicts down the road:

> You've put a personal organizer in your product? But we're chartered to build the company's solution for personal organizers! And why does the operating system have a to-do list?
>
> We started 10 years ago with a basic system. But we've added on so much that now we have over 50 applications and

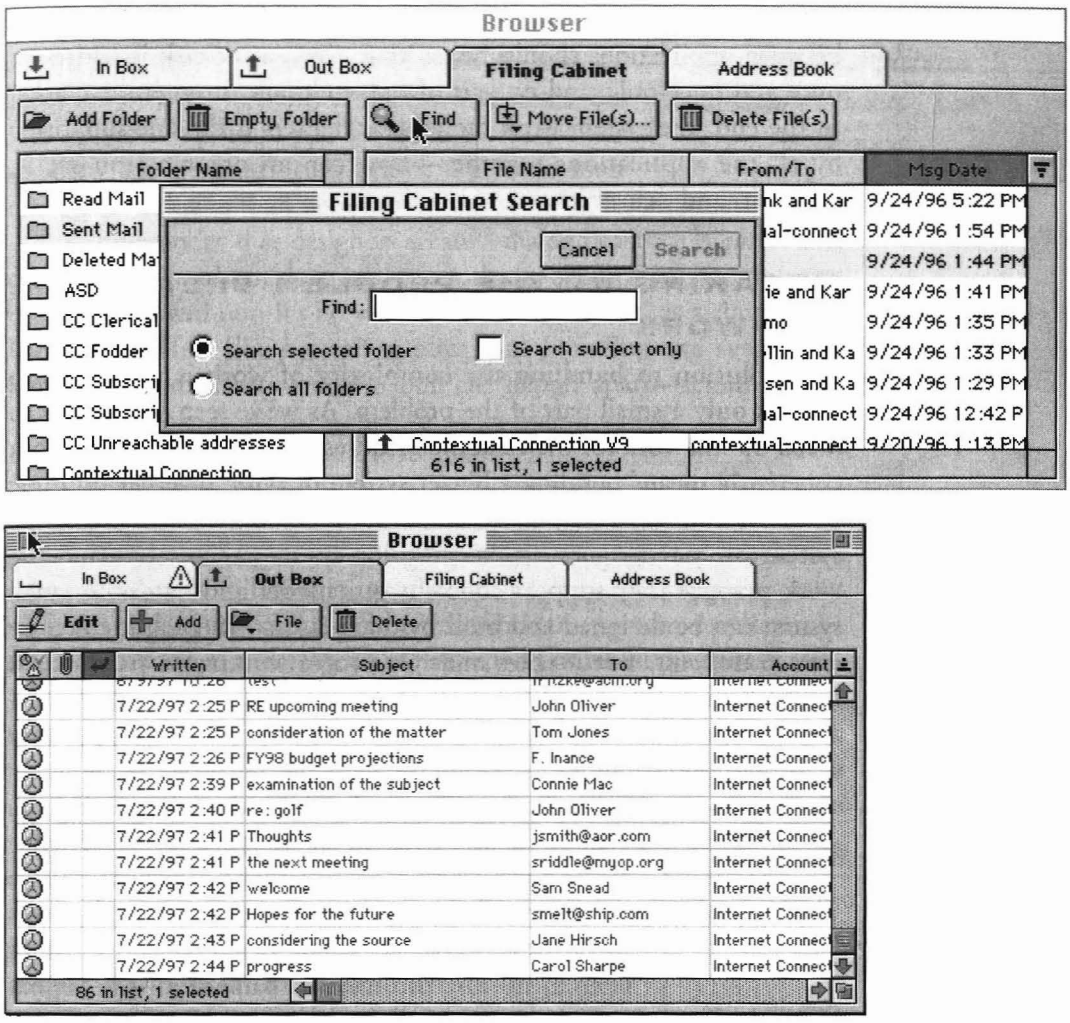



**FIGURE 14.1** Claris Emailer: Two ways of finding: one with a query dialog box, one with a filter.

no clear idea how they fit together. I'm not even sure we know where all the duplications are.

We do charting. We don't do data reduction—that's the database's job. I know you can't currently do the data reduction you need to do to use our charts, but that's the database's problem.

How can an organization figure out where the boundaries between applications should be, so that every work task is addressed once and once only, and no part of the work falls through the cracks? In the end, the system work model that matters is the one supported by all the applications together—how can an organization see it, design it, and deliver it?

## Breaking Up the Problem Breaks Up the Work

One solution to handling the complexity of work is to choose to address only a small part of the problem. As we've seen, that tends to break up the work for the customer. Addressing the whole of the work coherently means building a bigger system or tying together multiple small systems seamlessly (an even harder problem). As the size of the system or systems goes up, keeping the systems themselves and the work practice they support coherent gets harder and harder. A small system can be designed and built by one person—keeping it coherent isn't so difficult. But it takes multiple people working in parallel just to get all the details of a larger system worked out.

A common solution is to anoint an "architect" or architect team with the responsibility of tracking the whole system and catching any discrepancies. The work itself is done by carving the system up into pieces, assigning pieces to individuals, and letting them work out the details. But as we discussed when introducing work models in Chapter 5, as soon as any system grows beyond the very simple, it's just too hard to balance all the factors without some external representation to manipulate. Furthermore, in a large system, too many different people and groups are building too many parts—it's too hard to keep track of all the relationships. As soon as several people get involved in the design, they need an external representation to focus their discussion and capture their agreements. It's no longer enough for the whole design to stay in one person's head.

*No one person can keep all parts of a complex system in her head*

Passing out pieces for people to develop independently throws the whole design out of balance unless everyone really knows the whole design and how their part fits. Give one person a single part to design and build, and what should be a minor feature can turn into a whole

miniapplication (Figure 14.2). Designers find it hard not to treat their assigned part of the system as the most important—not only is their ego involved, but it *is* the most important part for them. It's no wonder so many small features turn into a larger and more complicated design than necessary.

*Developing a piece in isolation leads to overdevelopment*

It's no wonder that designers create a dialog box that is *almost* like the one their neighbor designed, but with the one or two extra features they can't do without. It's no wonder that what started as a simple dialog starts to feel like a small application. Dividing the system up among team members tends to pull the design apart—it's up to the design process to provide mechanisms that keep it whole.

## A SYSTEM HAS ITS OWN COHERENCE

While storyboards capture a coherent story of a single task, each storyboard can only follow that one thread. A full system supports many different tasks and roles. Storyboards work out system implications sequentially, by considering what happens in order to perform a task. But the system needs to hang together with its own organization and structure. That organization and structure has to be designed as a whole if the system is to be coherent.

It's as if the stories of use are paths across a university quad, each one wearing out the grass a little along that path. Then the groundskeepers look at the paths all together and decide that here, where two paths run almost together, they can be merged and paved; and there, where four cross, there might be a little courtyard with benches. The people making the paths are following their everyday life activities without thinking particularly about where they walk but following the best path for them; the groundskeepers are withdrawing from day-to-day events to see the implications on the whole "system."[1] And once the groundskeepers put new physical structures in place, people discover new possibilities and build on them—perhaps the courtyard becomes a favorite spot for

*Good design for individual work tasks is not enough*

[1] This is the same alternation between *withdrawal* to see the structure of what's going on and *return* to the ongoing work that we first encountered in discussing partnership, in Chapter 3.

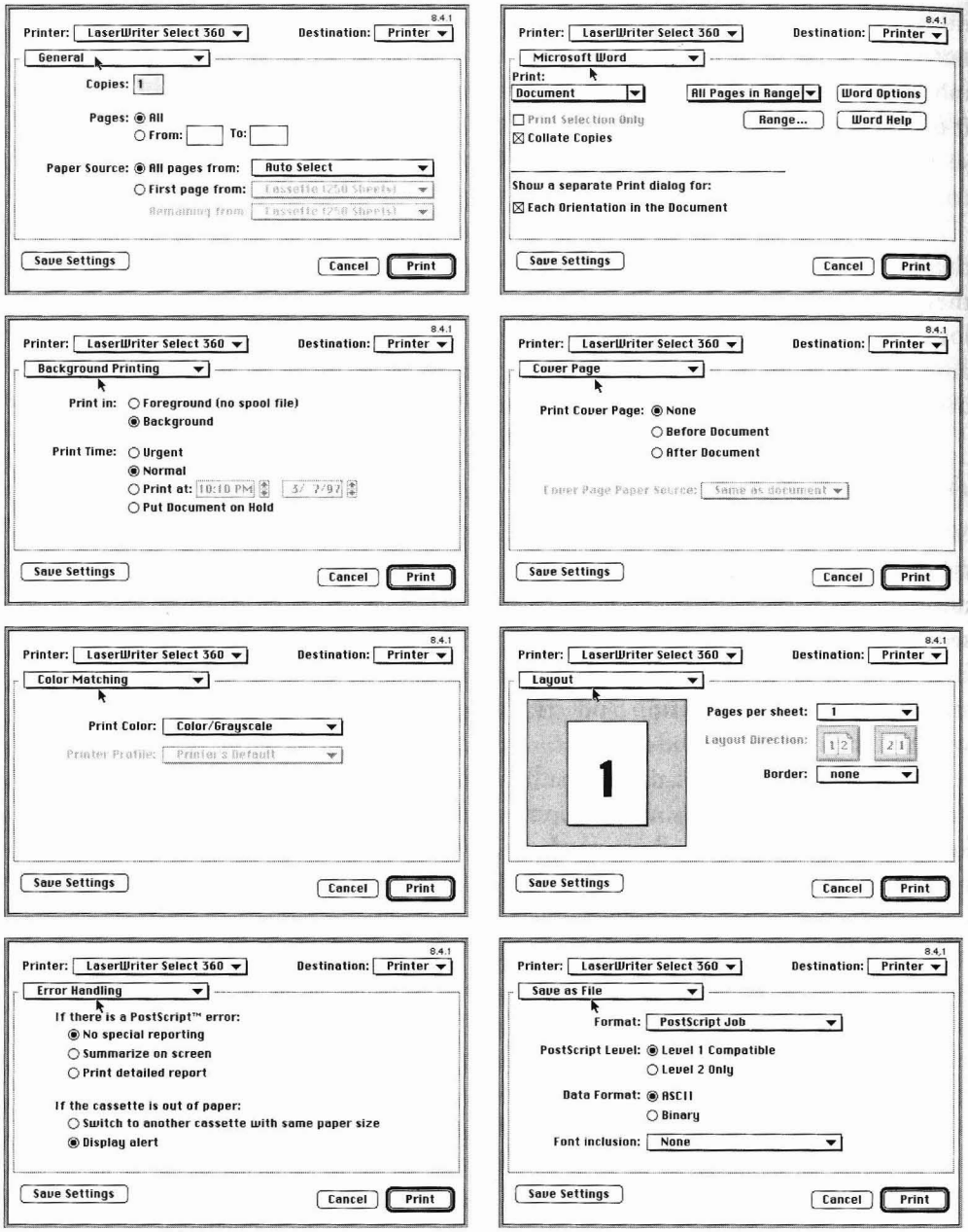



**FIGURE 14.2** Apple's LaserWriter 8.4.1 print dialog—what happens when printing becomes an application. How many different dialogs are really needed to print a file?

street musicians. When structure is well designed, it's flexible enough to support additional uses, unforeseen by the designers.

In the same way, designing a system based solely on storyboards—or use cases—would optimize each sequence of use at the expense of the system as a whole. Tell the story of nine different users, each with different printing needs, and each dialog in Figure 14.2 might make sense on its own. It's only when they are seen together that it's clear the interface is overcomplicated. Walk two separate cases for filtering address books and filtering sent mail, and each interface in Figure 14.1 makes sense. It's only when put together in the context of a system that they become inconsistent.

Good design tends to alternate between sequential and structural thinking. The initial quad design was a structure, designed as a consistent whole and put in place to be used by many people many ways. The actual use by any person is sequential: they came in here, crossed to there, sat on the grass after that, and left over there. Each individual sequence of use hangs together for that person. The next step of design switches back to structure. The groundskeepers looked at all the patterns of use together (as recorded by worn grass) and redesigned the structure to better fit its use. This step of seeing all the parts of a system as they relate to each other is an intrinsic part of systems design. Seeing and balancing the parts of a system against each other goes beyond a pure task-oriented approach by introducing a focus on the structure of the system itself.

*Good design process alternates between sequential cases and structural models*

## THE STRUCTURE OF A SYSTEM

A courtyard's easy enough to see and design. How can the design process make it equally easy to see the structure of a complex system? What *is* the structure of a system that designers need to see and manipulate, and how does it relate to the structure of work?

Consider a user reading mail: First she scans her new messages looking for something important. She doesn't care to see the whole content of every message—that would be a distraction. She just wants to know who it's from and the subject line to decide what to read first. Then she decides on one to read, and suddenly

*The structure of a system determines how well it supports work*

she needs a new context for doing this new activity. She no longer needs to see all her messages, but she does need to see the whole content of this one. Her intent changes—she's not wondering what important messages she might have been sent anymore, she's reading to find out what this one message says. Accordingly, her tools change—instead of seeing and scrolling over message headers, she's reading through message content.

Our user situated herself in a place in the product that suited the needs of the activity she was engaged in and stayed there a while, scanning messages with the tools provided. Then, when her activity and needs changed, she moved to a different part of the product where she could do the different kind of work associated with the new activity. The structure of the system consists of the *places* in the product where she can work, the *functions* that support work in each place, and the *links* that allow her to move from one place to another. The places do not impose any one sequence of use. Like the areas within a quad, they all exist together, offering the possibility of any number of different uses. But the structure they offer may make work convenient and easy to do or make it difficult. Customer-centered design seeks to build a structure into the system that supports the user's natural movement through her work and is flexible enough to enable the invention of new ways of working. Seeing this structure and reworking it to fit the user better is equivalent to the gardener restructuring a quad to better fit its usage. That's what designing the system work model is all about.

*Systems provide places where related tasks are done*

Designing the system work model as a whole runs counter to the engineering principle that every part should be self-contained so that changes to any part are isolated to that part. That's one reason for thinking of the implementation in terms of opaque modules or objects. But some approaches to design that work well for engineering the implementation get in the way when designing the system work model. Suppose Joe invents a way for users to scan and search without entering a query and uses it in the piece he was assigned. There's no way to keep users from expecting this approach throughout the system because the parts aren't isolated in the users' experience. Having encountered the mechanism in one part, users expect to find it in every similar situation. So keeping the design

*Any design change potentially affects the whole system work model*

coherent means that after a part changes, the designers must step back, look across the whole system, and see what impact that change has on the rest of the system. The system work model is a single whole—every part exists in relationship to every other part, and a change to one may ripple throughout the system.

## DESIGNING STRUCTURE PRECEDES UI DESIGN

Designing the system work model to fit the user is a problem of structuring the system well, not of designing the user interface or implementation. User interface and implementation are the next layers of detail in the design process. When the makers of PowerPoint decided to make one view, and only one, that edits the contents and layout of a slide, they made a decision about the structure of their product. By implication, they made an assumption about the structure of users' work—that it is reasonable to concentrate all slide changes in this one view. In the same way, when they decided not to give control over flying bullets for on-screen presentations from this view, they decided that was a function that did not need to be part of this work. These are structural decisions—they decide what the system should do and how it should be organized, but say nothing about how it should look or be implemented (Constantine 1994a).

Structural decisions of this sort precede decisions about the user interface. It doesn't make sense to design screen layouts until you've decided what function the screen should implement. It would be as though an architect started design by choosing rugs and materials for the countertops. They don't; they start with rough sketches that they work up into a floor plan. The floor plan captures the right level of detail for talking about the structure of a house—it shows the parts and their relationships without showing how the house is decorated. The user interface of a system is equivalent to the decoration of a house. It matters, but if the structure is wrong, no user interface can fix the problems.

*Use a language of structure to maintain a focus on structure*

In our initial design teams, we found that team members tended to slide into conversations about the UI before they were ready—before they had agreed on base structure. They were like architects who could only communicate by drawing pictures of the proposed

house. "We want this function in this window," one would say, sketching a row of buttons. "The style guide says those should go on a pull-down menu," another would reply. "Do you really want to use that word?" a third would ask. When the very language suggests that the user interface is the topic of conversation, it's hard not to be distracted by it. But how could we represent the system work model directly, free of any UI implications?

*Living in a house is parallel to working in software*

The pattern of working we found in software—working in a place, moving to a new location, and doing a new kind of work in that place—is not unique to working in a system. In fact, it's very like living in a house (Winograd 1996). To start dinner, a person goes to the kitchen, where the tools for cooking are located (knives, bowls, stove). A drawer sticks, and he decides to take it to the workshop and plane it down while the water boils. He moves with the drawer to another place, which has the different set of tools useful for minor carpentry, and works on the drawer there. Then he goes back to finish dinner. A house consists of places to do things, functions or tools that help do things, and access allowing people to move between places. The parallels between living and working in software and in houses suggest that studying the role of a floor plan might shed light on the appropriate representation of a system work model.

*A floor plan supports the structure of life in the house*

A floor plan occupies a unique role in the design of a house (Figure 14.3). It's less physical than an elevation, which shows a view of the house as though you were looking at it (an elevation is more like a UI picture). It doesn't show wall color or how the house is finished (which would also be more like a UI). Yet it's not at the nuts-and-bolts level of a construction diagram, which might show how to put a wall together but doesn't show anything the homeowner can relate to. The floor plan selects a few of the most salient aspects of a house as it supports living and represents them: the spaces in the house, their sizes and relationships to each other, the things in each space that support the kind of living done there (stoves, refrigerators), and the access between spaces.

As a diagram, a floor plan supports a conversation about how well a design supports a particular style of life, and allows the architect to compare that with the life the house's prospective owners want to have. Architects can walk stories of living through the floor plan to see how well it fits

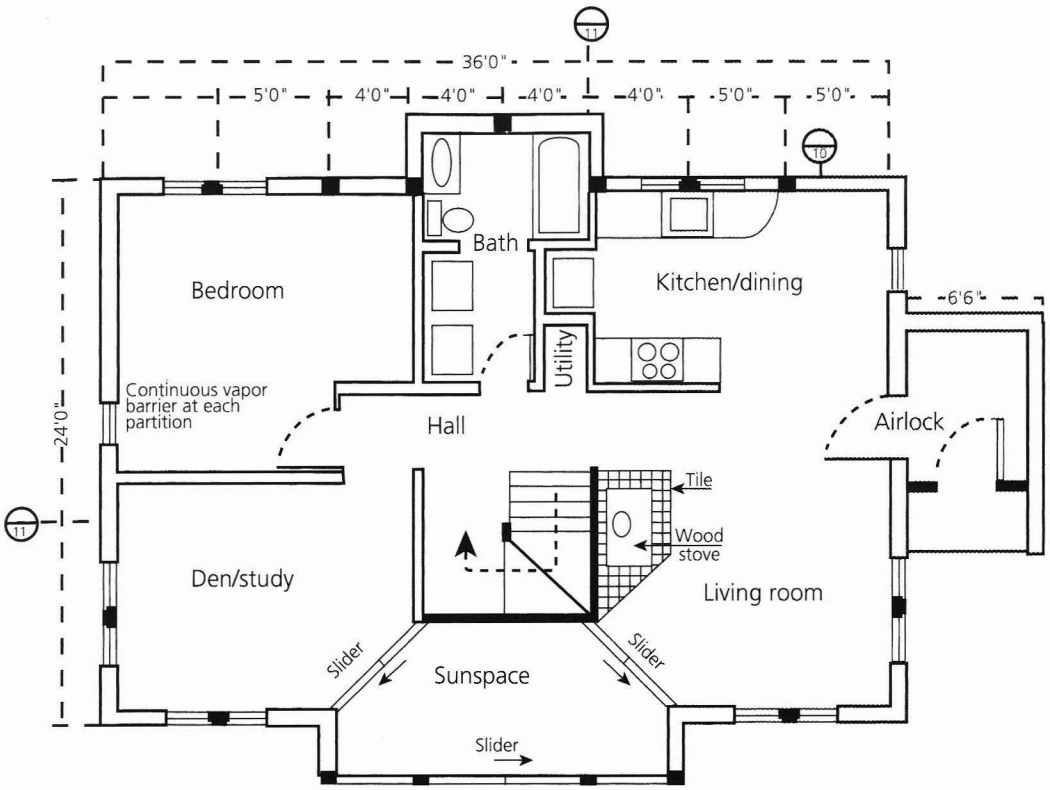



**FIGURE 14.3** A floor plan. Notice how the important distinctions are immediately apparent—the relative size of the spaces and the access between them. Details that are unimportant for understanding the structure of the house—rugs, wall surfaces—are absent or inconspicuous. (Even the tile around the woodstove affects access because people will walk around it.) But the drawing does tie to the users' experience of moving through a house. It also puts construction details in context—the dark squares in the walls indicate supporting posts, and the numbers in circles key this diagram to cross sections showing wall construction. This is what we are missing in software design—a single representation that shows how all the parts of the system relate in the users' experience and how they relate to the implementation.

the homeowners. Is a room too small for the way the owners want to use it? Is it too hard to get from one room to another? Is there a lot of dead space devoted to halls or intermediate areas? The floor plan lays out all the parts of a house, letting the architect walk different cases and scenarios through it. Rules of thumb, such as constraints on minimum clearances, layouts that work well, and the limitations of construction materials,

ensure that the resulting design is usable and implementable. Of course, once the house is built, meals may be eaten in the kitchen, and the dining room may become a music room. A good structure will permit different uses, which the architect never designed explicitly.

## The User Environment Design

Contextual Design represents the system work model in a new modeling technique, the *User Environment Design* (UED). The User Environment Design plays the same role in Contextual Design that the floor plan plays in house design. Just as a floor plan represents the key distinctions for supporting living, the User Environment Design represents the key distinctions for supporting work practice with software systems. Like the floor plan, the representation shows all the parts of a system that the user knows or cares about, what aspects of work each part supports, and how the parts of the system relate to each other (see Figure 14.4 for an example). Like the floor plan, it ignores UI details to reveal the underlying structure, uncluttered by surface appearance or by implementation details. In fact, the User Environment Design has no representation for these details, so it's hard for a conversation focused on a User Environment Design to go into details too soon. (In this way, it fills the need for a blueprint for software design identified by Denning and Dargan [1996].)

*A UED supports the structure of work in the system*

The User Environment formalism highlights the key concepts for designing a system work model. *Focus areas* show the coherent places in the system that support doing an activity in the work. They're the "rooms" of the system. Like rooms in the real world, they should support the activities that happen in them well. They should provide the function that's needed to do that work, and only the function that's needed. They should contain, organize, and present the objects that users need to work on. And like rooms in the real world, they are connected—the arrows between focus areas show how the user can move from place to place as their work requires it. Like paths traversing a quad, a new User Environment Design is built from storyboards, collecting the different stories of use into one structure supporting them all. User Environment Designs can also be built to represent existing systems, revealing their structure and highlighting problems (Figure 14.5).

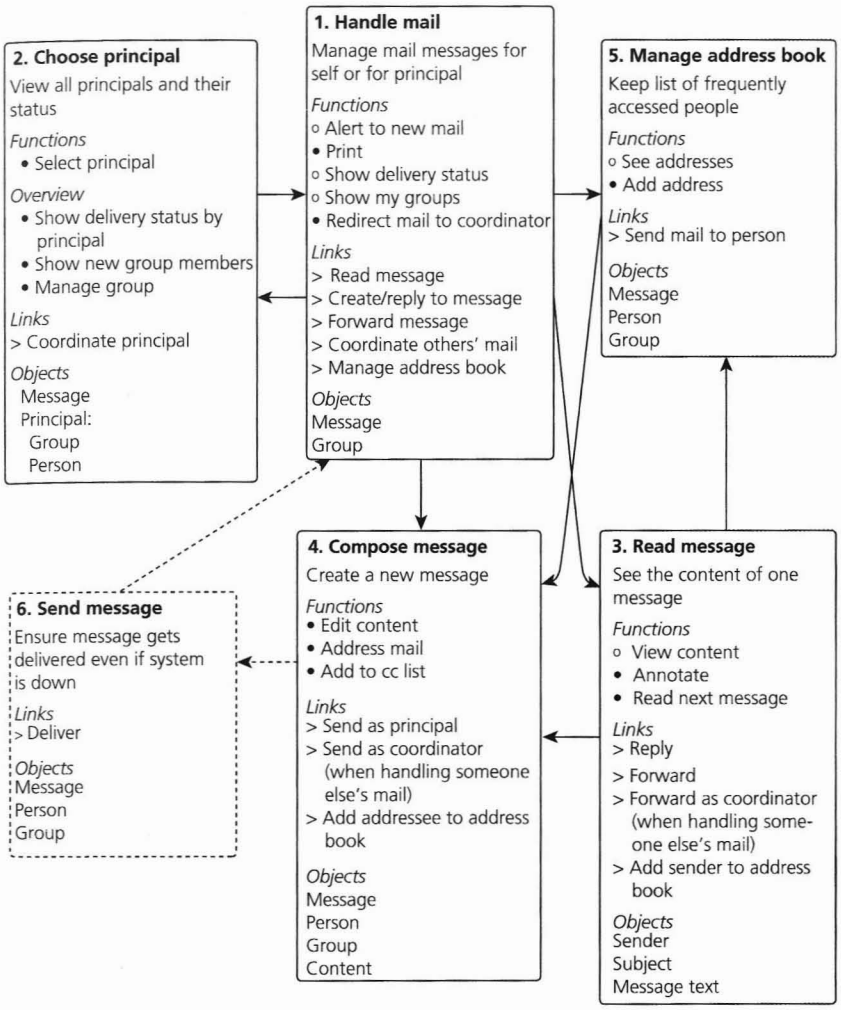



**FIGURE 14.4** A User Environment Design for a part of a mail system. Each box represents a "focus area," a place in the system. These are like rooms in a house, which permit the user to focus on one particular activity. The purpose statement describes the work the focus area supports. Functions, which enable the user to do work, are listed in the focus area, as are the work objects that the user works on there. The arrows between focus areas are links and show how the user can move through the system.

This form of User Environment diagram is on its way to becoming a specification for the system. Each focus area collects and describes the functions provided the user in that focus area. The focus areas act like checklists, allowing designers to review the function in each place and verify that all the function is needed there and that all needed function is available.

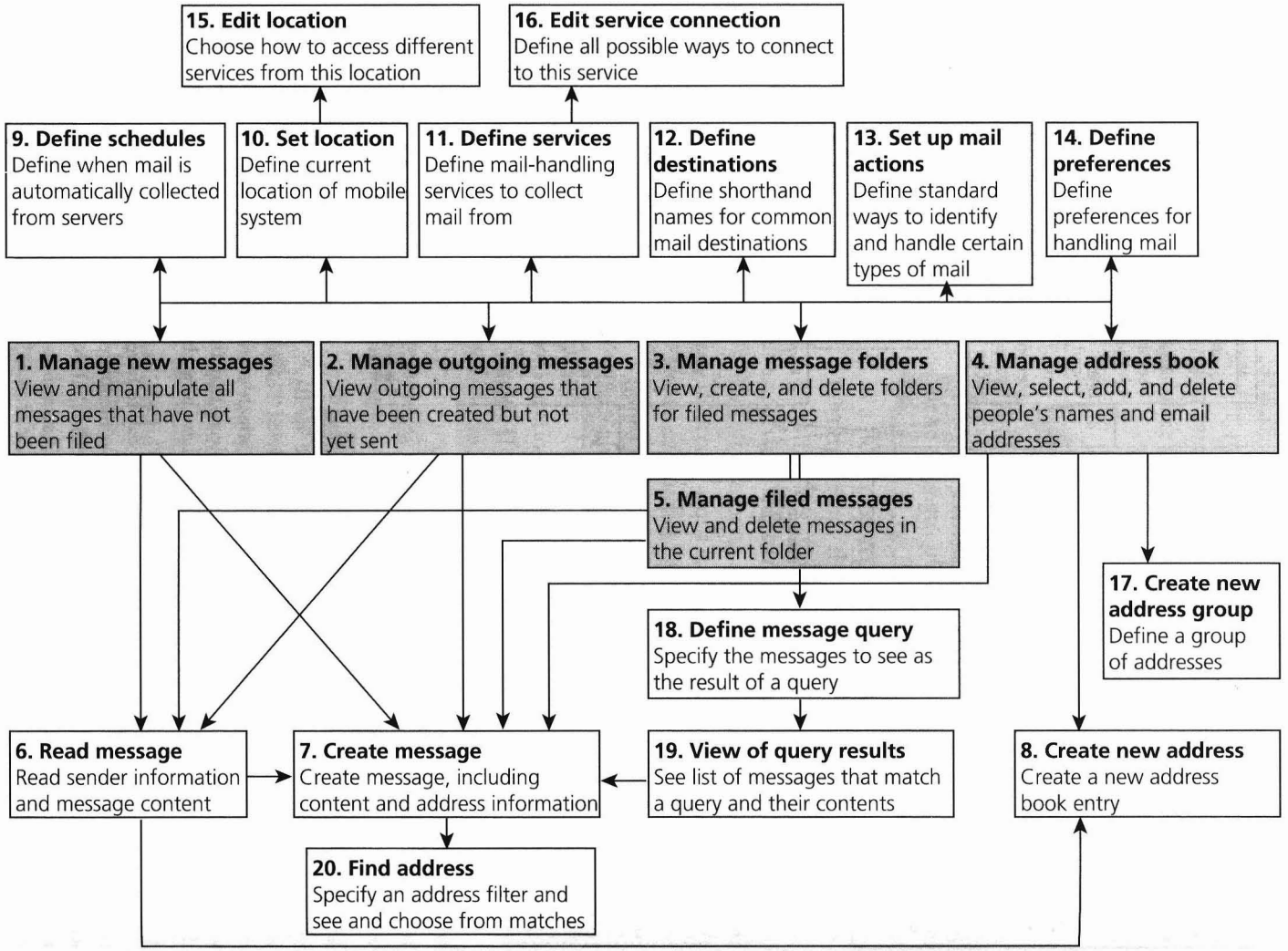

15. Edit location
Choose how to access different services from this location
16. Edit service connection
Define all possible ways to connect to this service
9. Define schedules
Define when mail is automatically collected from servers
10. Set location
Define current location of mobile system
11. Define services
Define mail-handling services to collect mail from
12. Define destinations
Define shorthand names for common mail destinations
13. Set up mail actions
Define standard ways to identify and handle certain types of mail
14. Define preferences
Define preferences for handling mail
1. Manage new messages
View and manipulate all messages that have not been filed
2. Manage outgoing messages
View outgoing messages that have been created but not yet sent
3. Manage message folders
View, create, and delete folders for filed messages
4. Manage address book
View, select, add, and delete people's names and email addresses
5. Manage filed messages
View and delete messages in the current folder
17. Create new address group
Define a group of addresses
18. Define message query
Specify the messages to see as the result of a query
6. Read message
Read sender information and message content
7. Create message
Create message, including content and address information
19. View of query results
See list of messages that match a query and their contents
8. Create new address
Create a new address book entry
20. Find address
Specify an address filter and see and choose from matches

**FIGURE 14.5** A full User Environment model of Claris Emailer UI. This more graphic form of User Environment diagram reveals the overall structure of a system, rather than the exact function in each place. Here, the core focus areas of Emailer are shaded for emphasis; in the product, each is a tab within the main window. The User Environment model immediately reveals that focus area 3, "Manage message folders," has a different user interaction style from the others—it's split into two interacting focus areas, 3 and 5. Beyond this core, the model shows two distinct parts of the product. One set (above) manages setup and preferences. Scanning the model reveals that setting one's location, a common function for those who travel, is treated as part of setup rather than an ordinary part of the core product. This contrasts with CompuServe's Information Manager interface, which more conveniently allows location to be viewed and changed right from the status window.

The second set of focus areas (below the core) support the actual work of creating and sending messages. Here, the model shows in 18 and 19 that users can only query on filed messages; that the results of a query appear in a special interface that shows both results and message content (not 6, the normal window for reading messages); and that though users can create a message from this results window, they cannot add the sender to their address book (8) as they could from the normal "Read message" interface. The model shows that the "Query/View results" pair (18 and 19) for messages provides a different structure for queries than the single "Find address" focus area (20) for searching the address book.

These are all structural issues, not user interface issues. They have to do with what the product does and how it is organized to do it. Whether this is a good organization or not depends on the actual structure of work in reading and handling mail—but the User Environment Design provides physical support for the conversation.

*A focus area is a place where users focus on one kind of work task*

Because focus areas are the most visible concept captured by a User Environment model, the model helps designers organize the system so it fits the work. Do users spend time scanning their mail messages, choosing which ones to look at? Then it makes sense to create a distinct part of the system that helps them do that. After choosing a message, do they then concentrate just on that message and what it's about? Then seeing other messages is a distraction, so it makes sense to provide a separate area to focus on a single message. What's involved in reading a message? If users often want the sender in their address book, adding the name to the address book should be a function available in that place. If users never scanned messages, but simply worked through them one by one, there would be no need for a separate place to see all messages. If they needed to see what messages preceded and followed the message they were reading, the place to read should be integrated with the place to scan messages. (In a bulletin board, where messages are captured in threads of connected conversations, readers do want to see the context of messages, and products often do connect the two places.) The structure of the system must be designed to fit the structure of the work, and the User Environment model makes the system structure explicit.

*The UED reveals structure*

## Representing the System Work Model

Capturing and representing the system work model in a User Environment diagram gives designers a way to see the whole system and keeps design at the right level.[2] A user interface would be too detailed—it invites the team to get caught up in issues of layout and appearance that can be put off until later, after the base structure is in place. Data flow diagrams show movement and transformation of data intrinsic to the work, not the structure of the system that the user experiences. Structure charts show the components of the system at a higher level of detail than code, but are focused on the structure of the implementation, which is not experienced by the user directly. The

[2] For a more detailed discussion of reflection in the design process, see Schon (1983).

same is true for object models: an object hierarchy provides a way to see and structure the implementation, not the user's experience.

Of course, people claim object models can represent anything, and it's true that an object model can represent the parts of a system design. But to be a good design and thinking tool, a model should evoke the thing being designed, making the right issues explicit and concrete. Designers manipulating the model need to feel like they are manipulating the real thing. An object model could capture the data in an architect's floor plan—but no architects using such a model would ever feel like they were manipulating space, as they do when they manipulate a floor plan. If the model is too far from the actual design issues, people using it have to make a translation in their heads. So it's not good enough to be able to make a mapping from a modeling technique to the issues for the system work model—a new model is needed to represent the user's experience. An effective model will influence designers' thinking by making the relevant issues jump out, just as work models influence what interviewers see in the workplace.

*A good model evokes the reality it represents*

Such a model won't supplant object-oriented design, of course. In an object-oriented design, the object model keeps the *implementation* consistent—in the object model, developers bring functions from different use cases together into a single object class. They identify the reusable parts that different system components can use. But that's at the implementation level—the object model represents the different parts of the code, invisible to the user. The structure of the system as the user experiences it needs to be kept coherent as well.

## The User Environment formalism in the design process

The User Environment Design occupies a place in the design process between storyboards on the one hand, and user interface design and object analysis on the other. It makes the discussion of the system work model tangible by providing a physical representation. In this way it helps to separate the conversation about the system work model from the redesigned work process (represented in storyboards), from the system appearance (represented

*Explicit models help keep design conversations separate*

### TASK-ORIENTED OR OBJECT-ORIENTED?

Is designing with storyboards and User Environment Design task-oriented or object-oriented? On the one hand, it's clearly not object-oriented because the User Environment Design does not focus on identifying common objects as its primary feature. Its most salient concepts are coherent activities—the focus areas—and flow between them. Yet it's clearly not task-oriented either. A User Environment Design prescribes no order, as storyboards do. It shows the parts of the system and their relationships independent of time. Many different stories of use can be walked through a User Environment Design to see how well it supports them just as many different stories of individual actions can take place in a house.

In fact, even in pure object-oriented design from use cases, object modeling does not stand on its own. The purpose of a use case is to tell a coherent story of how the users will work and the system will meet their needs (Jacobson et al. 1992). From this, object modelers can extract the objects and their behaviors. But neither use case nor object model provide a good representation of the system work model. The use case is task-oriented, telling one story of use, for one task, but it doesn't provide a coherent view of the system. The object model gives a coherent view of the system, but not the system the user experiences. Instead, it's a view of the system the developers will implement. It's not customer-centered because it's not focused on keeping the user's work coherent—and rightfully so, since it's an implementation tool. It is supposed to keep the implementation coherent—elegant, evolvable, extensible, and maintainable.

This is true even of so-called object-oriented user interfaces. These reveal to the user only a small proportion of the objects and behaviors of a full object model for the system. Object-oriented user interfaces achieve consistency by presenting objects as identifiable screen artifacts with consistent behavior. But what objects should the system present? How should they be organized? And what behaviors should they have? These are the questions answered by the system work model.

The basic question is, How do designers decide what the objects and behaviors should be to support the user? That's the question answered by the User Environment Design. Rather than base object definitions on use cases, the User Environment Design introduces a coherent model of the system that can be designed, structured, and corrected before object definition starts. In the end, it's neither task-oriented nor object-oriented. By focusing on the structure of work in the system, it's work-oriented and that's what makes it powerful. (Rosson and Carroll [1995] suggest another approach to integrating object-oriented and task-oriented system views.) ❑

by the user interface), and from the internal system structure (represented by the object model). When each conversation has its own physical representation, the design discussion is easier to have. Is the team arguing about how to change the user's work? Then they're standing in front of a storyboard, changing it to reflect their thinking. Are they arguing about how to organize the system to support that work? Then they are modifying the User Environment Design. Are they

arguing about appearance and layout? Then they're changing parts of the user interface. Everyone can tell which issues to pay attention to because that's the model the team is updating.

User Environment Designs support the natural alternation between sequential and structural thinking. Storyboards and use cases are sequential; they tell a single series of events in order. The vision, User Environment Design, and object model are structural; they show all parts of the system and how they interrelate, though they focus on new work practice, the system work model, and internal structure, respectively. Each sequential step follows a story of use to work out the details of the preceding structure and uncover problems with it, so storyboards help the team work out the details of a vision, and use cases help work out the details of a User Environment Design. Each structural step pulls together the implications of different stories into a coherent system. The designers step back to see how each sequence affects the structure as a whole. So the User Environment Design integrates one system out of multiple storyboards, and the object model integrates one model out of multiple use cases. When a single structure is created, it can be checked for accuracy and completeness—the functions of a User Environment Design and behavior of an object can be reviewed and anything missing added. This continuous process of working out details, integrating, and checking ensures the integrity of the resulting system. Each transformation acts like a structured walkthrough, forcing the team to review all parts of the system from a different perspective (Figure 14.6).

*Alternating between sequential and structural thinking drives design details*

The User Environment Design is created first to support design. It enables the design team to keep the system coherent. But because it represents the structure of the system as the user experiences it, it supports customer-centered project planning. Grouping focus areas that address specific roles identifies subsets of the whole design that support a coherent part of the work and could be delivered together. Identifying focus areas that are closely associated with each other reveals a subset that is appropriate for assignment to an implementation team. Building up a User Environment Design to include external and third-party products creates a strategic design showing what corporate partnerships to create. Building a reverse User Environment

*The UED aids planning*

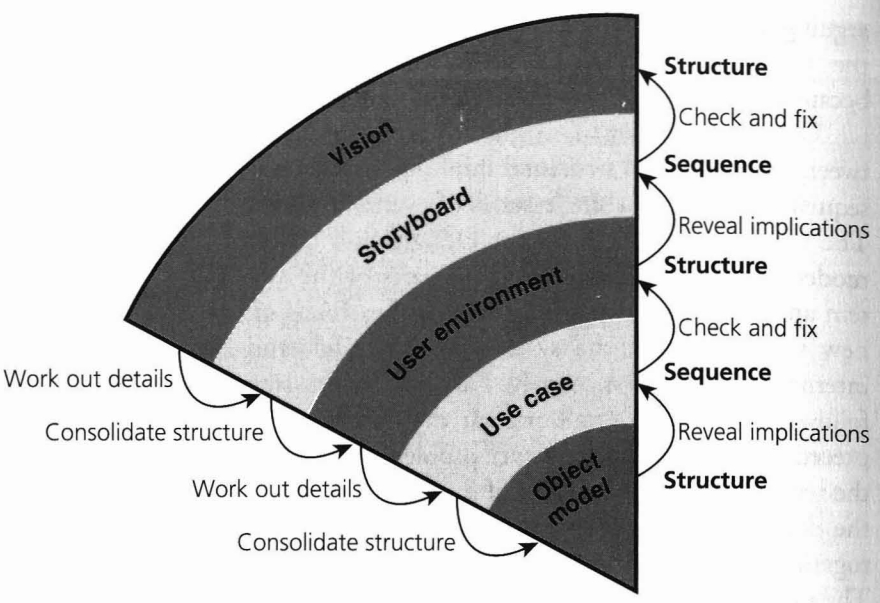



**FIGURE 14.6** The progression from design to development. This diagram shows the process of going from work models through systems design to implementation design. It shows the alternation between sequential, story-based thinking and structural, model-based thinking intrinsic to design. The stories build a structure that can be checked for coherence and completeness; the structure drives lower-level stories specifying more detail. Working out the stories reveals holes in the structure defined previously, and putting together the structure reveals inconsistencies in the stories. The stories show a particular instance of using the system; the structure shows how the system can support multiple stories. Contextual Design alternates between the two, providing physical representations all along the way.

model of existing systems identifies duplicated function and holes in the suite. By representing the parts of the system from the point of view of the user's work, engineers can see how their work relates to each other and to the user. And that keeps the whole development process coherent.

One of the challenges of any design process is to keep the design coherent—to maintain the design team's ability to comprehend and operate at the level of the whole system while working on a part. Contextual Design continually returns to a coherent representation that pulls all the detail together. The consolidated work models show the whole customer work, structured and represented along each of the five dimensions. The User Environment model shows the whole system as

experienced by the user, with all the parts and their relationships, independent of UI or implementation. The object model shows the whole implementation architecture and how it is organized. Each of these representations is focused on the appropriate issues for its place in the design process, but each represents the whole system coherently. The User Environment Design responds to the work models on one side and drives the object model, the user interface, and project planning on the other. It's the pivot between customer work and system implementation, making sure that the work as it happens in the system hangs together.

*The UED keeps the work coherent by keeping the system coherent*

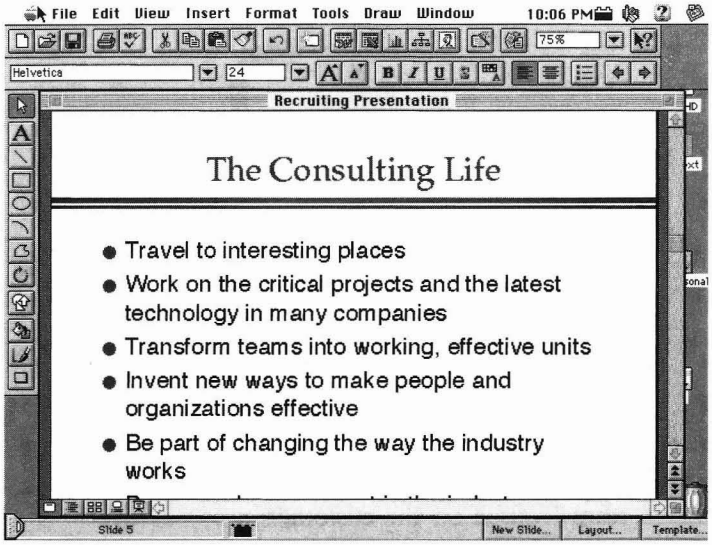
# 15 *The User Environment Design*

The goal of the User Environment Design is to present structural issues, making the key considerations salient for keeping the user's work coherent. IT keeps the design team focused on the customer by giving a physical representation to the structure of work that the proposed system will enable for its users. To be a good tool for accomplishing this task, the User Environment formalism organizes the presentation of the system into a structure that supports a natural flow of work.

We saw in the previous chapter that work consists of coherent activities. Each activity is oriented toward accomplishing some intent, requires a certain set of actions to accomplish, and is naturally connected to other activities that the user might choose to switch to, given what they are trying to achieve. By their structure, systems create places that can support an activity if they have the right organization and make the right functions available. The system work model fits the user when it matches the structure of activities and actions that the user needs to accomplish.

Just as any house has a floor plan, no matter how it was designed, any system has a User Environment model implicit within it. Any system can be analyzed, and its underlying User Environment model revealed. We introduced the User Environment Design informally in the previous chapter and showed some models taken from commercial products. To introduce the parts of the User Environment formalism and their definitions, we'll walk through the analysis of another commercial product, Microsoft PowerPoint. As we go, we will show each part of the User Environment formalism and how they build up to a complete model.

*Every system has a UED*




**Edit slide**
Purpose: Create, view, and change the content of an individual slide

*Functions*
o View slide content
• Edit text
• Add shape to slide
• Add text box to slide
• Save slide show

*Links*
> Edit slide show
> Edit slide notes

*Objects*
Slide contents

**FIGURE 15.1** The main screen of Microsoft PowerPoint and the focus area that represents it. This window creates a place in the system in which to focus on creating, viewing, and changing the content of individual slides. This place is represented on the right, with the work that is done there summarized in the statement of purpose. Functions are available through different mechanisms—toolbar buttons, pull-down menus, the keyboard, and direct manipulation. In the focus area, we show only the functions, with no indication of how they are accessed. As is usual in a model built to analyze an existing product, the functions are high level ("add shape" rather than listing all the different shapes), and the model lists only the primary function in support of the purpose. A model built to design a new product would list every function and every shape designers intend to implement.

Figure 15.1 shows the main window of PowerPoint, a tool for making slide presentations. This window provides a place for creating and editing the content of slides. In the User Environment Design we represent places as *focus areas,* where you focus on doing a certain kind of work. Every focus area has a *purpose,* a succinct statement of the work the focus area supports. If you can't write a single sentence that describes the purpose of the focus area because there are so many different functions doing different things, it's likely that the system is poorly structured. Use the purpose statement to describe everything the focus area does.

This window provides functions that enable doing work in the place—to put rectangles, text boxes, and other slide objects on the

slide, color and rotate them, and manipulate them in other ways. Functions are made available through menus, toolbars, keyboard commands, and by direct manipulation. These are alternative UI mechanisms for performing a function; some functions can be accessed all three ways (e.g., to save the presentation, choose "Save" under the "File" menu, click the disk icon on the toolbar, or type CTRL-S). Which mechanism the designers chose to implement for a function matters—a poor UI or inconvenient access to a function gets in the user's way—but it doesn't change the purpose of the place or the work done there. The UI mechanisms and screen layout are as much a distraction to understanding system structure as rug color would be on a floor plan. So we list the functions once on the right, with no indication of how they are accessed.

*The UED focuses design on coherent work activities and the functions they need*

The "Edit slide" window also makes available the things the user needs to work on to edit a slide—the slide itself and also text boxes, shapes, lines, and clip art. These objects are collected and organized in the place appropriate for the job at hand—in this case, laid out on the screen to reveal the design of the slide. The focus area captures these work objects as an important part of the definition of the focus area. Later, when the object model is developed, they will be harvested as starting points for the objects.

Some of the function in this window leads to other places. Selecting the small icon at the bottom changes to the slide sorter view. This changes the view and the function available—it is no longer possible to create and edit the content of slides here. Instead, the slide sorter supports viewing a whole presentation in order, changing the order of slides, and controlling the transition from slide to slide. Because the work that can be done is different, the slide sorter supports a new activity in a new place, and we represent it with a new focus area. The function that switched from one to the other is a *link,* shown on the User Environment Design as an arrow (Figure 15.2). You'd expect to find links between focus areas whenever the user might need to switch between the activities they support.

*Links between focus areas enable a shift in attention to another activity*

This much of the User Environment formalism will represent 90% of most products. However, there are some additional cases, the

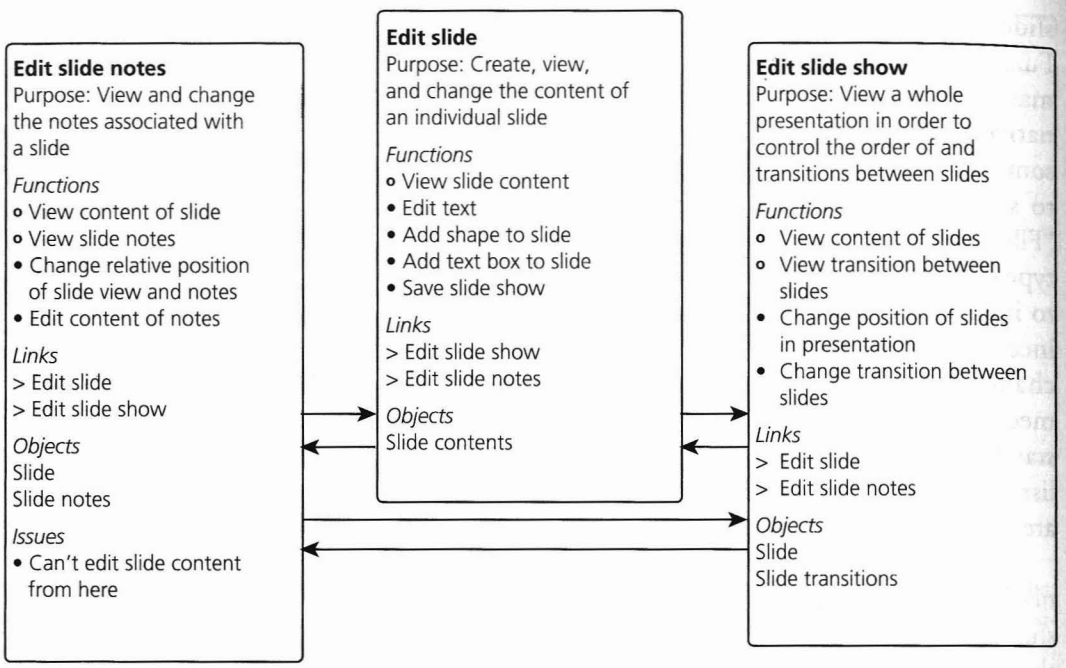



**FIGURE 15.2** Links between focus areas. These three focus areas support distinct but related activities. They declare that when a user is worrying about the detailed content of one slide, she is not concerned with the overall structure of the presentation. Conversely, if she is worrying about the overall presentation, she needs to see and recognize slide content, but doesn't need to change it there; she's willing to switch back to the "Edit slide" focus area and lose the context of the whole presentation. This works reasonably well, but on the other side users do need to change slide notes and slide content together. When developing a slide, we have found that users naturally develop notes and slide content in parallel, moving information from the notes to the slide and back as the idea of what is presented shifts. The division of the work in the current design does not support a fluid movement between notes and slide content. The User Environment model above shows the connections and reveals the issue.

most important of which is the *double link.* When the user needs to do the work of one focus area in the context of another, we show a double link between the two focus areas. The spell checker is clearly a separate focus area from the main slide show—in this focus area, you think about spelling and dictionaries, not about the overall layout and content of your slide. But on the other

*A double link says to keep one focus area in the context of another*

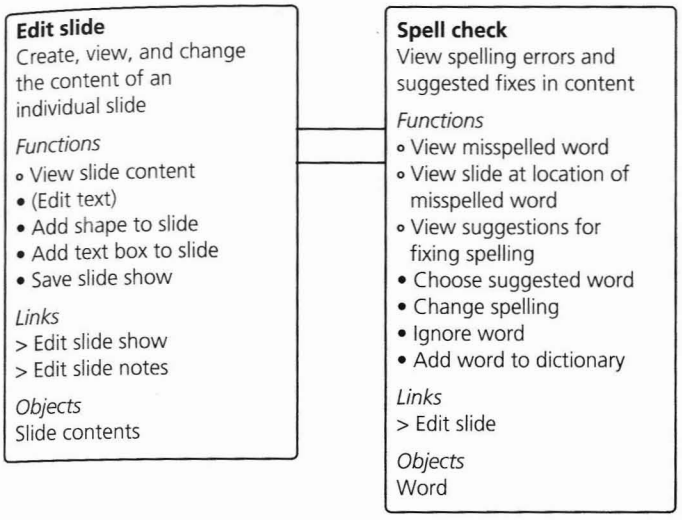



**FIGURE 15.3** A double link between focus areas. The double link indicates that the work done in the second focus area, spell checking, needs the context of the main focus area and that the user will switch back and forth between the two. Designing the user interface for this is a challenge because the user needs to switch between focus areas without losing her context in either.

hand, it's also closely linked to the "Edit slide" focus area: when you move to the next spelling error, the main window switches to display the slide with the error on it and attempts to position the slide so the error is visible. So we represent the spell checker as a double link to the "Edit slide" focus area (Figure 15.3). This indicates that the two cooperate to support the work, that the user needs to know where they are on the slide while checking spelling and needs to switch back and forth rapidly between the two. (When errors are marked as you type, the function has been merged into one focus area.)

This is a partial reverse User Environment model of PowerPoint, showing the primary parts of the formalism and what they represent in a real product. (See "User Environment Formalism" for a complete definition of the formalism.) Each box or focus area represents a coherent place to do work. The links between places show how the system supports the flow of activities but doesn't prescribe any particular order of work. The double-linked focus areas in Figure 15.3 show how the spell checker is related to the slide view; it says nothing about when the spell checker is run.

## USER ENVIRONMENT FORMALISM

**Focus area**

A focus area collects functions and work objects into a coherent place in the system to support a particular type of work. A function should be necessary to do the work, not to manipulate the UI:

—Supports performing a coherent part of the work
—Named with a simple active phrase
—Lists functions that are needed to do the work
—Lists the work objects that the *user* needs to perform the work
—Numbered for unambiguous references to the focus area

*Purpose*

Short description of what the focus area does in supporting the work

*Functions*

Functions are described on the UED with a short phrase. They are written up online with a description of their behavior and justification.

- Functions invoked by the user to do work
- ○ Functions that are automatically invoked by the system as necessary. The user knows these functions exist, but does not invoke them explicitly.

(name) Function clusters that appear in multiple focus areas. This is shorthand for listing all the functions in the cluster. The function cluster name appears between parentheses and is separately defined once to apply to all focus areas.

*Links*

Links and double links to other focus areas:

> Functions that support links between focus areas. An arrow between focus areas represents the link. The function name may not be the same as the destination focus area name, in which case the name or number of the destination focus area should be given in parentheses.

≫ Functions that support *double links* between focus areas. A double line between focus areas represents the double link.

*Work objects*

The things the user sees and manipulates in the focus area

*Constraints*

Implementation constraints on the focus area: speed, reliability, availability, form factor (for hardware), etc.

*Issues*

Open design issues associated with this focus area, UI ideas, implementation concerns, and quality requirements ⇨

**Hidden focus areas**
Conceptual units of work done by the system that the user knows and cares about, but doesn't have to interact with. Often they automate work that used to be done by a person. Represented as boxes formed of dotted lines, connected to other focus areas with dotted lines.

**External focus areas**
Conceptual units of work delivered by other teams. External focus areas show how your system works with others to provide coherent support to the customer. ❑

## The reverse User Environment Design

There are two ways to take advantage of the User Environment Design. One is while designing a new system: seeing the structure ensures that the system stays simple and close to the user's needs and helps a team plan how to deliver. We'll show how to do that below. The other is to do what we did with PowerPoint—build a *reverse User Environment Design* to represent a product that already exists.

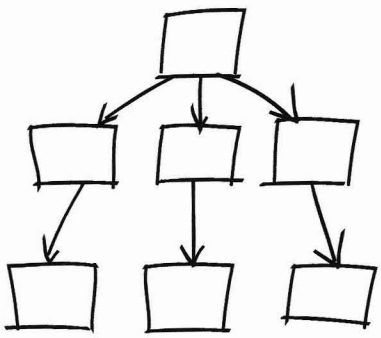
Building a reverse User Environment Design has a number of uses: to analyze a competitive product, to reveal the structure of multiple systems that need to be integrated, or to represent an existing system version so it can be extended in a new version. Building a reverse User Environment model of an existing system reveals its underlying work model. It reveals what users can think about and do together in the system, and assumptions built into the system about how users work. In the PowerPoint example above, PowerPoint supports changes to notes and changes to the slide content in separate focus areas. This implies that creating notes is an unrelated activity to creating slide content. It doesn't really matter whether the system's designers intended that consciously; that's what's built into the system.

*A reverse UED shows your implicit existing system work model*

Building a reverse User Environment model can be the first step in designing the next version of an existing system. It's easy for systems to

get more unstructured over time—what started out as a reasonable and elegant design turns into a rat's nest of features and connections with no clear structure. Before adding new function, build a reverse User Environment model to see structural issues in the existing system. Modify the design to capture decisions about what to fix. Then you can make storyboards to capture new work practice for the next version to support, and you can use the process we describe below to roll them into the User Environment Design. In this way, you can add function without losing the system's overall design coherence.

When one developer was introduced to the User Environment Design, he started laughing hysterically, then grabbed a piece of paper and started sketching boxes and arrows on it. "I just figured out why users hate our system," he said. "This is what it looks like." He showed us the diagram he had drawn:



"See? They have to go all the way back up to this top box and then down again to do anything."

*A reverse UED of your competitor reveals the field of opportunity*

Using the reverse User Environment Design to see the structure of competitive products can make it clear to a team what the grounds for competition are. For example, most presentation packages have essentially the same structure at the User Environment level. The three focus areas of outliner, slide sorter, and slide editor are very common. The grounds for competition in presentation packages is at the level of detailed function and UI; the first product to shift the ground through a fundamental improvement in structure or UI paradigm will gain a substantial competitive advantage. Conversely, QuickMail Pro's market message (from their

marketing literature) is that it offers a base structure different from that of other mail products: "The 'All-in-One' Message window lets you simultaneously view your incoming messages, create and send new messages, and file or sort existing messages." Instead of providing separate focus areas for the in-box, sent messages, and filed messages, it's got one focus area for all three. This will only work if the work practice of users naturally mixes these different activities; otherwise it will be confusing.

The reverse User Environment Design is a good way to step back from a system and get insight into it. Surprising numbers of systems have the hierarchical structure that the engineer recognized in the story above. But the reverse User Environment Design may also reveal the values and assumptions about the work practice built into the existing products. When these are explicit, the team can compare them to the work models representing real customers and decide whether the assumptions built into the current systems work for the market or organization. For example, one team developing a collaborative work tool that allowed anyone to drop in on any conversation realized they were promoting a value of open communication to an extent that might stifle the use of their system.

*Discussion during a reverse UED reveals designers' values and assumptions*

The reverse User Environment Design gives a team a way to see what the users experience as they move through a system. It's a valuable tool in its own right. But the User Environment Design is also central to designing new systems.

## Building the User Environment from storyboards

In the design of a new system, storyboarding drives the design of the User Environment. We discussed in the last chapter how design alternates between sequential thinking and structural thinking. Storyboards are sequential and run a single thread; the User Environment Design is structural and reveals how all the threads fit together coherently. Storyboards give a lot of

*A good structure suggests and supports unforeseen ways of working*

information about a part of the system in the context of a specific use—how that part of the system supports one work task. The User Environment Design lets a team build the single coherent structure that supports multiple different tasks performed by different roles. It's a framework, a structure for doing work that is not constrained to the particular storyboards used to build it. Users will invent new ways to do their work based on the structure in the User Environment Design, if it's designed well.

*Separating storyboards from the UED keeps design conversations separate*

Separating storyboards from the User Environment Design (and from subsequent user interface design) helps a team separate different kinds of design thought. Storyboards support following a single story of use: "I'm a user sorting my mail. How do I approach it? What do I do?" This is one approach to designing a system. It ensures that the system hangs together from the point of moving through a task, but it tends to hide the relationship to any other tasks the user might do. The User Environment Design supports structural thinking: "What's really going on in this place? Is it supporting a single, coherent activity? Does it provide everything the user needs to do that activity?" With two diagrams, each focused on supporting one kind of thinking well, the conversations can be separated for the team, making them clearer and easier to have.

*Storyboards imply the new system structure*

The User Environment Design is built from storyboards one at a time. Each storyboard contains implications for place, functions, and links in the User Environment Design. After the implications of each storyboard have been incorporated, the team steps back and looks over the whole User Environment Design with an eye to maintaining coherence. They identify focus areas that overlap in purpose and merge them, clean out focus areas that have accumulated extraneous functions, and reorganize the structure so that every focus area has a clear purpose and appropriate links to the rest of the system. (Constantine [1995b] describes building systems from a "use context model," a similar process.)

A team pulls structural implications out of a storyboard by walking through it cell by cell. Each cell may suggest a new focus area, function, or link in the emerging User Environment Design. Storyboards are pictorial and help a team recall the context and the implications of

each cell for the design better than a scenario or other textual description. The team discusses these implications and revises or extends the User Environment Design to capture their decisions.

The sketches that are part of storyboards give designers a way to think in the language most natural to them, while still staying out of the low-level details for as long as possible. The User Environment formalism is a direct representation of the issues for structuring the user's experience of the system. But we've found that teams coming up to speed on the User Environment formalism don't find this new representation a natural form for thinking. They do better thinking and designing in UI sketches, capturing them in the storyboard, and then pulling out the implications for the User Environment. The more they go back and forth between User Environment Design and user interface, the more they start to see the design implications from the User Environment diagram directly, and the more it will work for them not only as a seeing and checking tool, but as a design tool.

*Moving between storyboards and the UED helps designers see structure in the UI*

Here's how the process works in practice. The storyboard in Figure 15.4 shows the first steps of a user getting help in a new work redesign. The vision implies a mix of hardware and software to implement: the phones are altered to have a "help" button, and the phone system is tied into the computer system so that the call is associated with the office and user where the phone is located. When the call is routed, the first-line helper's phone rings, and at the same time this information is displayed on her screen.

Because the system is a mix of hardware and software, some focus areas in this User Environment represent physical hardware places as well as software screens. In this way the User Environment diagram can be extended to represent the total system delivered to the user: hardware, software, documentation, and other systems. (It won't, of course, represent other aspects of the corporate response such as marketing or services.) The phone acts as a place to do work in an office: the work it supports is communicating with others. The help button adds a function to the place: get quick help on system problems. So the implication of the first cell of the storyboard is a new function on an existing hardware focus area in the user's office (Figure 15.5).

*The UED can represent hardware and software that the user interacts with*

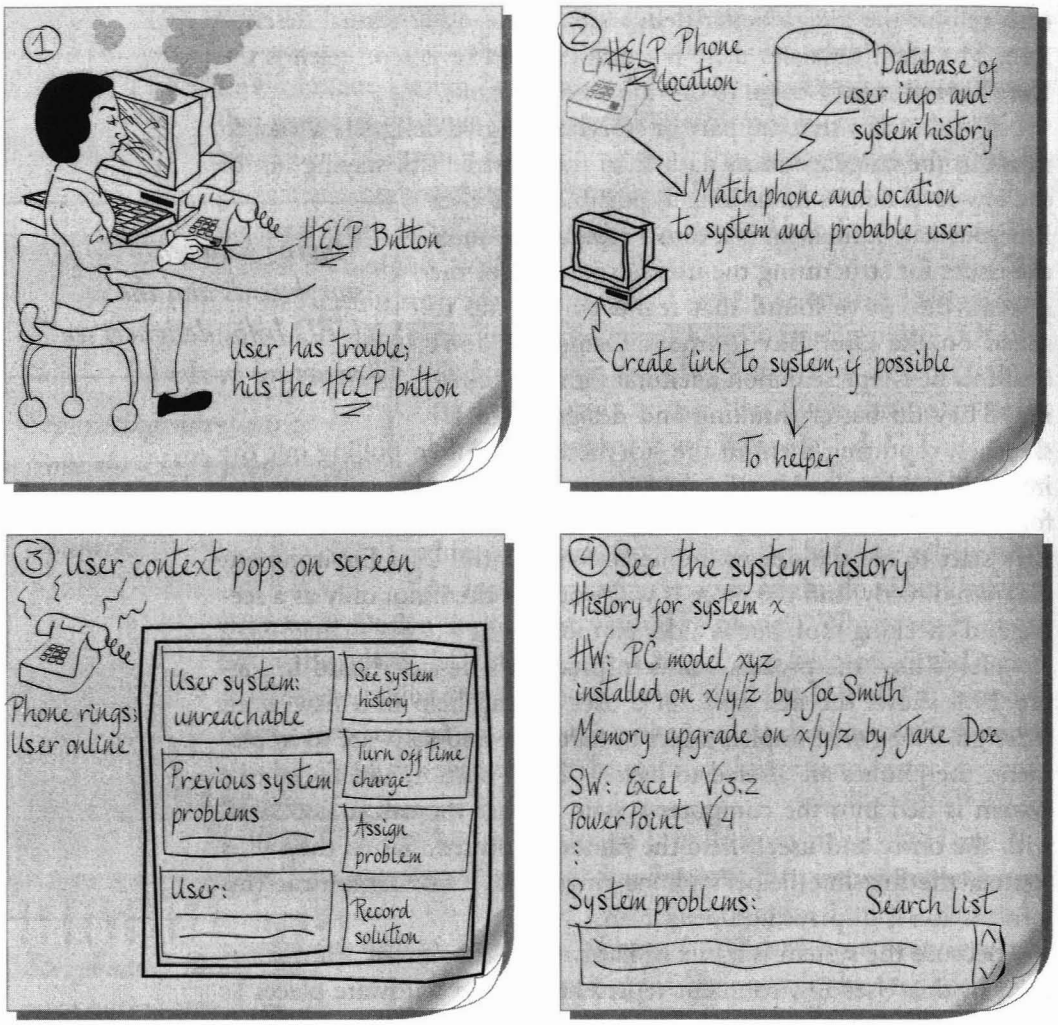



**FIGURE 15.4** Storyboard for getting help from system management.

The next cell shows how the system acts when the help button is pushed. It's necessary for working out what the system will do, but it isn't part of either the user's experience or the first-line helper's experience; it's entirely behind the scenes. In the next cell, the result of these behind-the-scenes actions is to display information on the first-line helper's screen and ring the telephone. So

*The UED shows only what users care about or interact with*

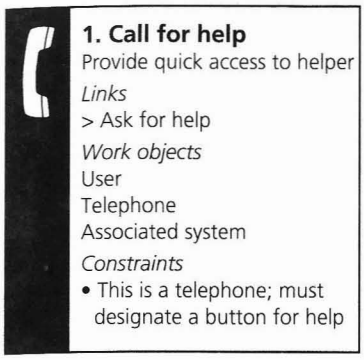



**FIGURE 15.5** A focus area representing new functions on the user's phone.

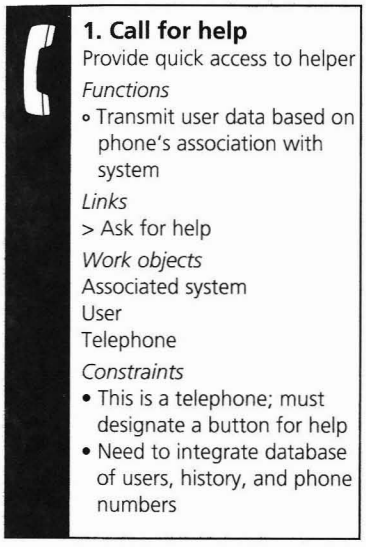



**FIGURE 15.6** Function added to an existing focus area.

these actions flesh out the definition of the "help" function; they don't lead to a new focus area (Figure 15.6).

When the screen comes up on the helper's workstation, it creates a new focus area showing the information necessary to work on the user's problem. The information about the user, his system, and any history is displayed immediately, without any explicit request on the part of the helper. This is represented as an automatic function. We choose a name for the focus area that is terse and describes the primary

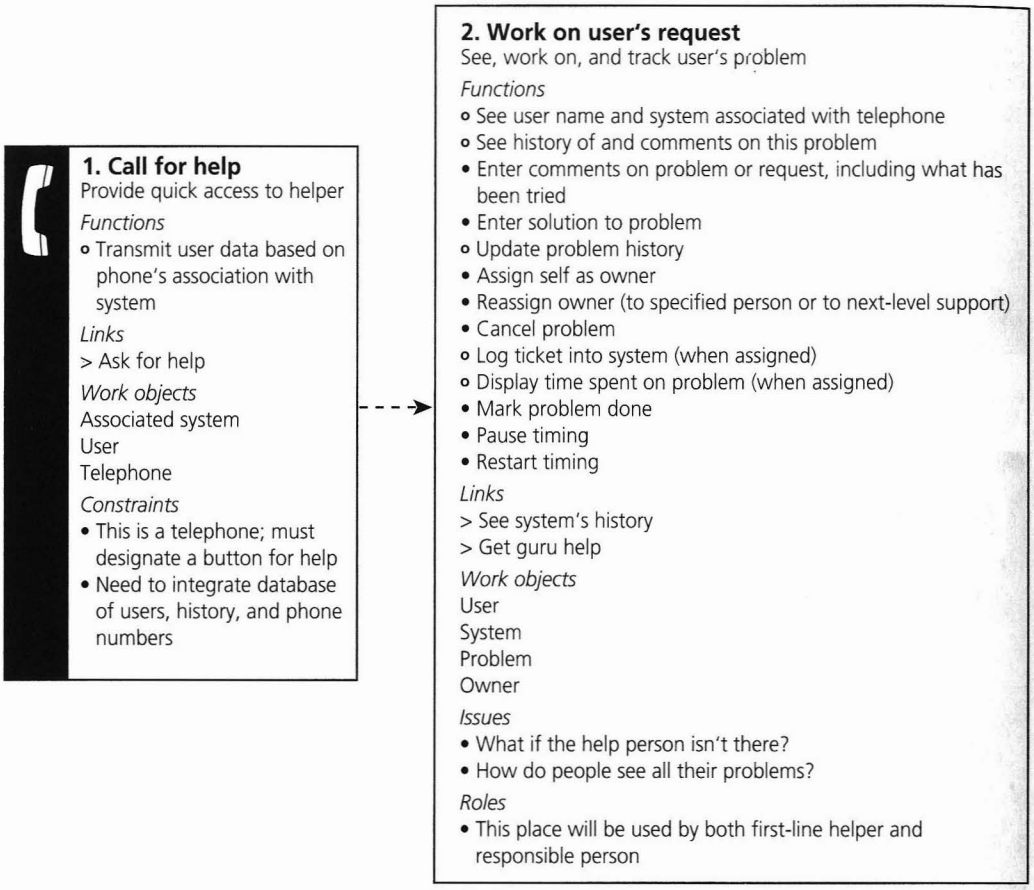



**FIGURE 15.7** Two focus areas connected by a hidden link. Each focus area collects the functions out of all storyboards needed to support the work of that place. They begin to act as a system specification, organized into clusters that support a coherent work activity.

work it supports: in this case, "Work on user's request" captures the essence of what the place is for (Figure 15.7).

The link between the phone and the new focus area is not an explicit link; neither the user nor the helper move between the phone and the "Work on user's request" focus area. The communication between the two is in the behind-the-scenes work. We show this on the User Environment as a dashed line, showing how the system supports communication between the focus areas.

The storyboard defines additional functions needed by the helper in this place: the ability to turn on and off time charges, assign ownership of problems, and record solutions. We write these functions right into the place. And we add the objects the user works on in this place to the focus area also—the problem report and the user information. Later, when it comes time to build the object model, the functions will define the behavior these objects must support by specifying what to write into the use cases that drive object modeling.

*Objects in a focus area reveal the things the user works on*

The next storyboard step has the helper looking at detailed system history. At this point, she's not thinking about the overall problem and system anymore; she's thinking about what has happened on the system that might tie into current behavior, either to support or suggest hypotheses. This is a different kind of work from the initial, direct discussion with the user about their problem. The system support for it is quite different—this part of the system is organized around browsing, free-form searching, and locating pieces of history by association. All this implies a new focus area, "See system's history," which is linked to "Work on user's request." Links are like other functions in that the user has to take an explicit action to follow the link; they're different in that the effect they have is to move the user to a new focus area. We find it useful to collect the links together in the User Environment Design so people can see the connections all together.

This decision about when to create a new focus area is critical to the User Environment Design. Focus areas support one part of the work and are organized to support it well. Whenever the user is doing a new kind of work, worrying about a different set of concerns, or engaged in a different style of thought, it implies a new focus area. This generally means that the user should work in any focus area for some amount of time, just as people expect to spend time in a room. It's hard for people to shift their attention from one kind of work to another frequently—the system should not force such a shift unless the work demands it.

*A good focus area doesn't complicate or fragment a coherent activity*

When rolling storyboards into the User Environment Design, it's the work the storyboard represents that defines the focus areas in the User Environment. The designers of the storyboard were thinking in the UI and may have created subwindows or dialog boxes, but if they

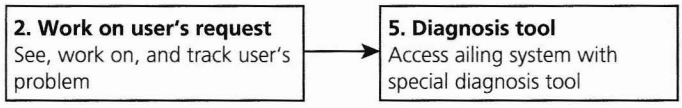
don't support a different kind of work, the system doesn't need new focus areas. Conversely, if the storyboard mixes unrelated work in one interface, it implies several focus areas in the User Environment Design. This is the time to clean that up.

*Inquiry into multiple storyboards drives the UED*

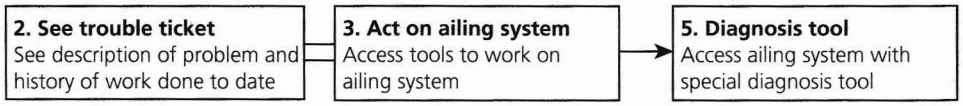
The process of generating a User Environment Design from a storyboard continues in this manner, using the discussion of each cell in the storyboard to identify and capture new focus areas and extend existing focus areas (Figure 15.8). But the User Environment is the structure that supports all stories. After doing the first storyboard, roll in additional storyboards in the same way.

The first cell of the storyboard in Figure 15.9 identifies a place in the system we haven't seen before—a place for seeing all the work assigned to the user. We add it to the User Environment Design. Then the next step defines a place for seeing an existing problem. But when we look at the User Environment Design we already have, we see that "Work on user's request" already allows us to see and work on a problem. Should this cell reuse that focus area or create a new one? This is a question about the appropriate system structure for the work.

The new storyboard suggests a new way of thinking about the system structure. The first storyboard created one place from which to manage all the work of dealing with a problem. That place acted like a control panel or command center, providing access to all the different tools that might help resolve the problem:




The new storyboard suggests a different approach. Instead of a single command center, the new storyboard breaks out the passive work of seeing the description of the problem and any work done on it to date and documenting any new actions. By breaking the act of working on the problem into a separate cell with a separate UI sketch, the storyboard suggests that access to tools be part of a second focus area:



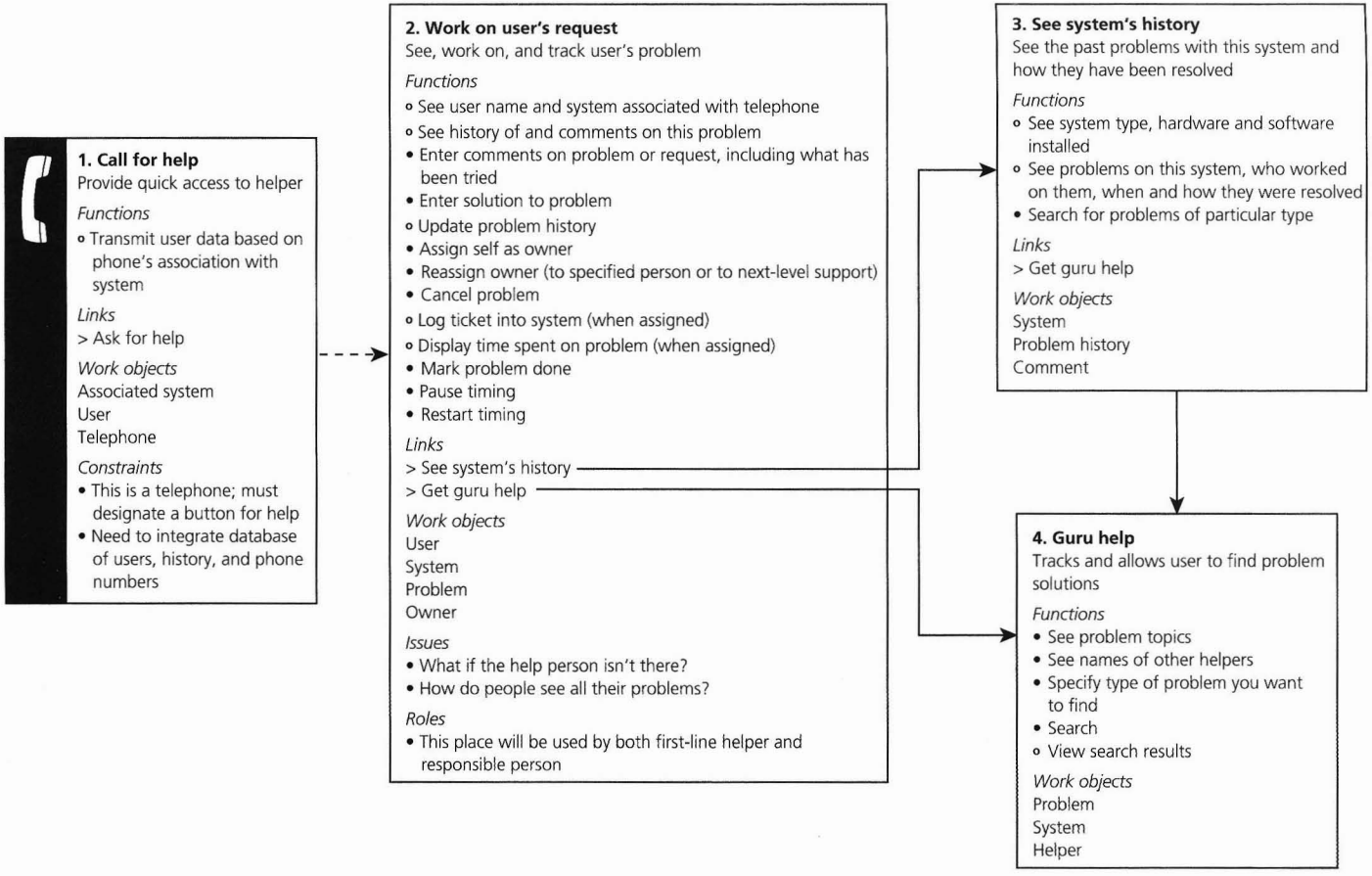



**FIGURE 15.8** The complete User Environment Design generated from the first storyboard.

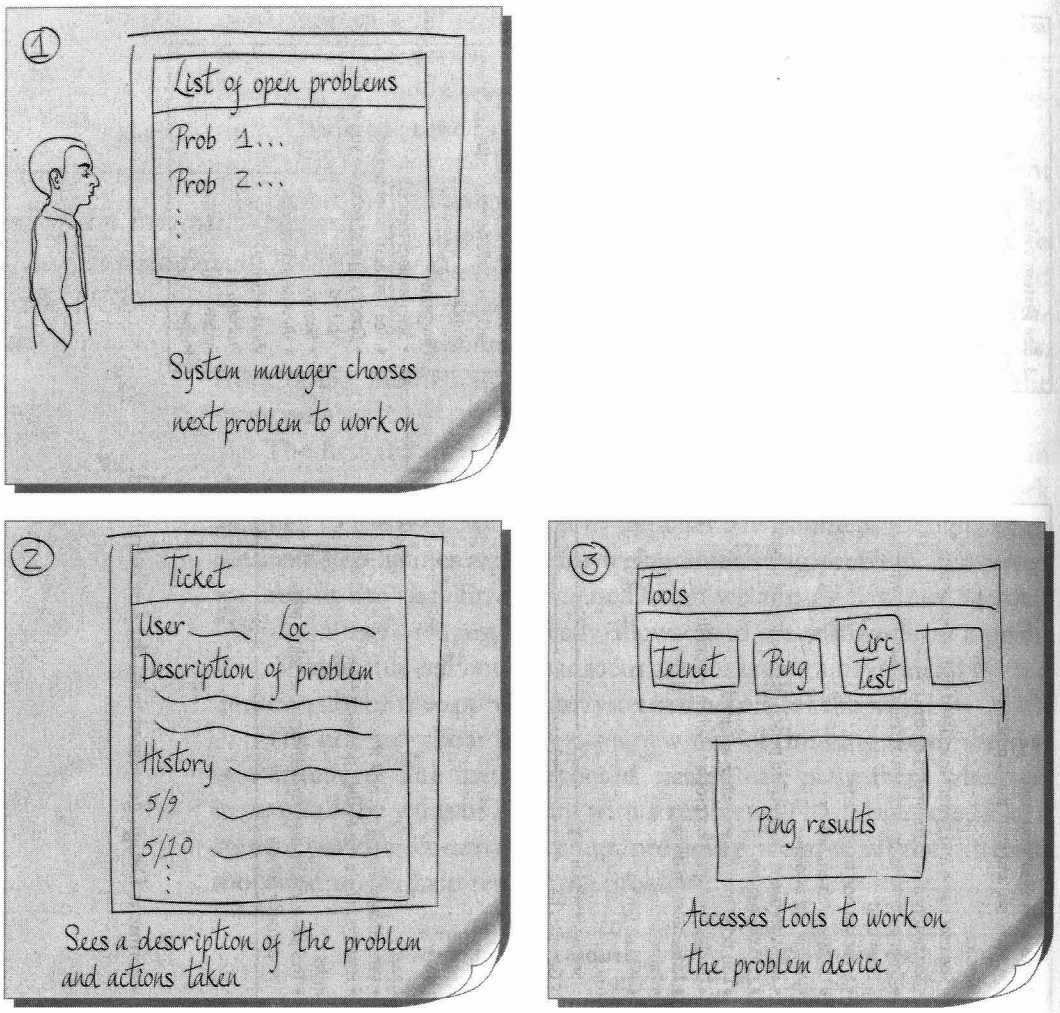


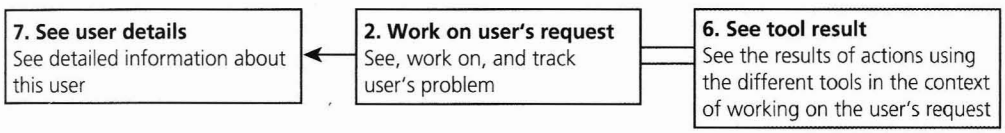

**FIGURE 15.9** A second storyboard, in which the system manager starts from a list of assigned tasks instead of starting by answering the phone.

These are different options for structuring the system. Up to this point, neither option has been given careful thought. The designers did what made sense for each storyboard without careful consideration of the implications for the system. Now that the two storyboards are coming together in the User Environment Design, the team can have the conversation about which structure would be best for the work as they have observed it. Should a trouble ticket be like a form,

capturing the whole history of the work that has been done on this problem? That would be a close duplication of a paper ticket, an essentially passive holder of information. Or should a trouble ticket be an active working place bringing together the knowledge and context of the problem with the tools needed to work on it? The User Environment Design helps the team have this conversation with the aid of sketches like Figure 15.9. By removing UI details from the conversation, the User Environment diagram keeps the conversation focused on structure.

In the actual case, the design team decided on the first of the two options and prototyped it with the helpers. The helpers liked having a single command center for dealing with problems but went further: they wanted the interaction with their tools to happen in the same place. And they didn't need to see all the details of the user in that place. The User Environment Design implied by these changes keeps a place to work on the user's request, but it integrates the tool results into that place through a linked focus area. And the detailed information about the user is moved into a separate focus area, accessible, but out of the way:

*Different storyboards suggest alternate structures to reconcile in the UED*




The concepts provided by the User Environment diagram make this discussion easier to have. They focus on the critical question for this level of design: what are the places the system will create, and what work will they support? The different diagrams above support a discussion about what structure fits the user's work best, disregarding UI considerations.

Similarly, these discussions precede any object modeling for the system implementation. If objects were derived directly from the storyboards, there would be no opportunity for this level of structural thinking. Each of the different options above suggests different technical challenges, a different set of use cases, and a different object model. In particular, the third option suggests a use case describing how invoking a diagnostic tool causes that tool to run on the appropriate system and show its results right in the "Work on user's request" place.

*Different UEDs imply different object models*

Neither of the other two options suggest that use case. By designing the structure of the system work model first, the User Environment Design helps stabilize the design before object modeling starts and limits the amount of rework needed afterwards.

## DEVELOPING SPECIFICATIONS

When you have to work within a software process that expects a software specification, the User Environment Design can take you much of the way. The User Environment Design defines how the new system will behave and organizes its function in a way that makes sense for the user. Based on this, you can drive the different parts of the specification. A typical specification might have the following parts:

**Overview:** The first part would give an overview of the whole system, its goals, and its basic structure. This is illustrated with a high-level User Environment model—titles, purposes, and links only, as in Figure 14.5. This section introduces the reader to the system and orients them to the parts of the system, showing how the different parts support users' roles and tasks.

**Supporting data:** This section summarizes the customer data on which the system design is based. It shows key sections of the affinity and consolidated models, reviews the roles that the system primarily supports, names the primary influences that drove the design direction, and summarizes the structure of consolidated sequences for key tasks. Particularly when customer-centered design is new to an organization, it's important to emphasize how a design is built on and responds to concrete customer data.

**Functional requirements:** This is the basic definition of what the system does. It's organized by focus area. Each section introduces the focus area and describes the work done there. For each function, it names the function and provides a full description of the function's behavior. In this way, it avoids presenting long lists of functions with no organized intent—instead, it's clear how the functions together support particular activities. Objects manipulated in the focus area are named, and constraints and issues are listed. Where the specification includes user interface designs, they are described with the focus area definition.

**Nonfunctional requirements:** Additional requirements on the system—performance, reliability, maintainability, evolvability, platforms supported, and so on—are listed in their own section. These are collected from the affinity and extended while building the User Environment Design but aren't associated with any particular focus area. They are kept on the side for inclusion in the specification later.

**Objects:** The objects manipulated in the different focus areas are listed with the focus areas but described once, here. The meaning and usage of the objects across all focus areas are described. This will act as a starting point for later object modeling. Use cases will describe the detailed behavior of the system, and out of that, the behavior of each object can be defined, and additional implementation objects identified. ⇨

**External interfaces:** External interfaces to the system are described. We'll show in the next chapter how links between focus areas can define interfaces between one system and another. In this part of the specification, these interfaces are collected and described.

It's easy to integrate detailed requirements tracing into this structure when your organization requires it. In each function definition, list the storyboard cells that used that function. Document each storyboard and record the consolidated sequence that you used to define it. List any additional data you used—sections of the affinity, role definitions, or other pieces of consolidated models. Document each consolidated model online, and link it to the individual models from which it was built. Do this, and you'll be able to take any function and walk the steps backwards to the actual customer data that suggested the design of that function. ❑

## DEFINING A SYSTEM WITH THE USER ENVIRONMENT DESIGN

The User Environment Design keeps the user's work coherent by holding the whole definition of a focus area in one place. If you have no physical representation, it's too hard to look across a whole system and decide if the parts of it are coherent and where a new function should go. But when the system is concrete in a diagram, it's not hard to scan the purpose and existing function to find the right place for a new extension. When a focus area gets too complex, it's straightforward to review it and related focus areas. What roles does the focus area support? What tasks? For each role, is the focus area reasonable? What's really needed? Using questions like these, designers can rebalance the focus areas and clean up the design.

Within each focus area, the list of functions, links, and constraints summarizes what can happen in that place. As a list, it supports checking the completeness of the focus area—it's easy to scan and check against the issues raised by models and storyboards. Keeping the UI sketches from the different storyboard cells that contributed to a focus area gives additional context: they show what the designers were thinking about when they developed the place. Because they are sketches, they are more concrete, helping designers envision what a system based on this User Environment Design might look like. And

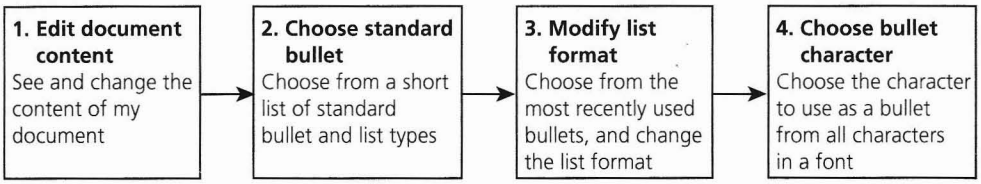



**FIGURE 15.10** When a focus area leads to one other focus area, which leads to one other after that, you have a "leggy" User Environment structure. The user will have to go through multiple layers of windows to accomplish a function. This is the structure for defining the bullets in a bulleted list in Microsoft Word. How many different ways are there to choose a bullet? And how many different focus areas do you have to go through before you can choose one? In this case, the focus areas are created by dialog boxes; each dialog box creates its own concern by offering a different interface and different function that the user has to parse and understand. Not every dialog box would be represented as a focus area. Microsoft Word's "Zoom" dialog box is simpler and would be considered part of its parent focus area.

they give UI designers a starting point for designing the presentation of the focus area.

Thinking in terms of focus areas and links tends to keep the basic work of the focus area *in* the focus area, rather than spreading it over several. Thinking in terms of today's user interfaces allows—or encourages—spreading the function across windows, panes, dialog boxes, tabbed dialogs, and other gewgaws. Look at the way MS Word uses three layers of dialog box to specify bullets (Figure 15.10). Thinking in the UI raises worries about constraints of screen real estate and problems of specifying every detail of a function; it's easy to punt and decide to put the function in a dialog box. Thinking in the User Environment Design takes away that excuse—if the function is part of the work of a focus area, it goes into the focus area.

*The UED works against proliferation of dialog boxes*

Later, when it's time for the UI designer to create a user interface, the User Environment Design will have collected all the different functions from all storyboards and organized them into coherent areas, each focused on one kind of work. It's up to the UI designer to figure out creative ways of making the function available in one coherent place in the interface. This gives the UI designer the most flexibility to be creative—deciding to split a focus area because it will be too hard to design the UI prejudges what the UI design will be able to do.

The sketches from the storyboards offer suggestions for the UI design and show the concepts the storyboard designers intended to reveal. But the UI designer has to decide, for all the storyboards collected into this place, and for all the roles and tasks the place might support, what is the UI appearance that will support the work best. In the above example, the "Work on user's request" focus area has to let the first-line helper see what work has been done on a problem and also support the system manager doing the work on the user's system. The User Environment Design specified that they could both take advantage of the one focus area; now the UI design has to support both roles. The first-line helper has an irate user on the phone; he needs a clear and direct interface. But he does want to see the whole history of the problem. The system manager wants powerful access to all the tools, but if that access is provided, she can benefit from the clear and direct interface the first-line helper needs. The UI designer has to consider both roles when designing the presentation and access mechanisms.

*The UI designer makes function accessible for all users and tasks*

This is the ongoing process of extending a design: create a storyboard to work out the implications of a new component to the user's work practice, then roll it into the User Environment Design to see how the system structure can support the work practice you've designed. Storyboards keep the work coherent; the User Environment Design keeps the system coherent. Additional storyboards build up the User Environment Design into a structure that responds to all the multiple tasks and roles the system must support. The resulting User Environment Design shows all the parts together, how they relate, and how they overlap.

### Developing the Object Model

The next task facing the team after developing the User Environment Design (and checking it with users, which we discuss in the next part) is to start the design of the implementation. This is what use cases and object modeling are all about. We will not treat object modeling in depth, but the design work done in storyboards and User Environment Design gives the team the basis they need to design the implementation quickly. ➪

The usual method is to start with use cases and define classes and class hierarchies from them. Going from use case to object model is another example of switching between sequential and structural thinking; the use cases are a story of use, the object model is structural, and (if used) object interaction diagrams are a story again. But, as practiced in the industry, use cases may be very high level, showing a whole task in the work, or low level, showing the accomplishing of a smaller function. And it's always an issue to decide what ought to be specified by the use case anyway.

When building on a Contextual Design project, we incorporate use cases at two points. First, storyboards act like high-level use cases. They show how real users interact with the system to get tasks done. At this level, the storyboard is well grounded in a consolidated sequence and the vision, so there are clear criteria for what should be included. But the storyboard format is more appropriate to this high level of design. Their pictorial nature makes it easy to scan and see the emerging design. And, while use cases include preconditions, postconditions, and exceptions, we've not found it necessary to specify these at this high level.

The User Environment Design provides a high-level structural thinking step that responds to the storyboards. Change the structure or function at this level, and the object model for the implementation will change; merge two focus areas and expand the function of a work object in the User Environment Design, and the corresponding implementation object will take on new responsibilities.

Later, object modeling captures these implications by switching back to sequence-based thinking in low-level use cases. At this level, each use case tells the story of how one function or closely related group of functions operates. The use case is based on storyboards and User Environment Design: the storyboard defines what the user will do, while the User Environment Design defines the function. The use case works out precisely what happens when the user operates these functions, how the system responds, and how the system internals make the designed response possible. They reveal flaws in what went before and drive the next step. Use cases bridge the gap from design of the system work model to design of the implementation.

Similarly, the design team derives events and triggers driving the implementation from the User Environment Design. Whether initiated by the user invoking a function or initiated by the system as indicated by automatic functions, the User Environment Design defines the events that the implementation needs to handle.

Building the User Environment Design as an intermediate step between storyboards and use cases helps ensure that the structure built into the use cases holds together for the user. Until the User Environment structure is stable, there isn't a design to build use cases on—changes at the User Environment level will change what happens in the use cases. Without an explicit representation such as this, the only way to work out structural issues is in the implementation and the UI. The more we can reveal, identify, address, and test these issues with users before starting implementation design, the faster implementation design and coding will go. ❑

## User Environment Design walkthroughs

The walkthrough is the final step of building a User Environment Design, and it should never be skipped. It's always done before going on to design a UI or test the design with users. The User Environment Design walkthrough uses principles of good system structure to check the design. Even a careful team will, as they roll more and more storyboards into the design, start to destructure it. A focus area that started out clean will accumulate function until the original purpose gets blurred. Perhaps any individual function could be justified, but taken together they suggest a different work focus that should be separated out. Two focus areas that started clearly distinct will, as function is added to each, start to overlap to the point that the distinction is no longer clear. The team needs the walkthrough as a chance to withdraw from the design, take stock of it, and reorganize what has started to get messy.

You'll see another level of structure when you walk through your design. The design itself suggests new possibilities when you pause to inquire into it. A set of focus areas taken together may imply support for a whole task or role; three focus areas might be consolidated into one addressing the fundamental task more directly; or functions in several focus areas suggest an activity that could be supported directly in its own focus area. It's this step of rationalizing the design against the work that will lead to a solid, flexible base structure that supports many different uses.

*The walkthrough lets you be the groundskeeper redesigning the quad*

Walking the User Environment Design also gets the team into position for the next phase of design. It ensures that the whole team is clear on what they intend by the design and how they think it will work. It identifies *test cases*—conditions or design elements that become a focus to test with users in prototypes. In this way it becomes the first step toward iterating the design with users.

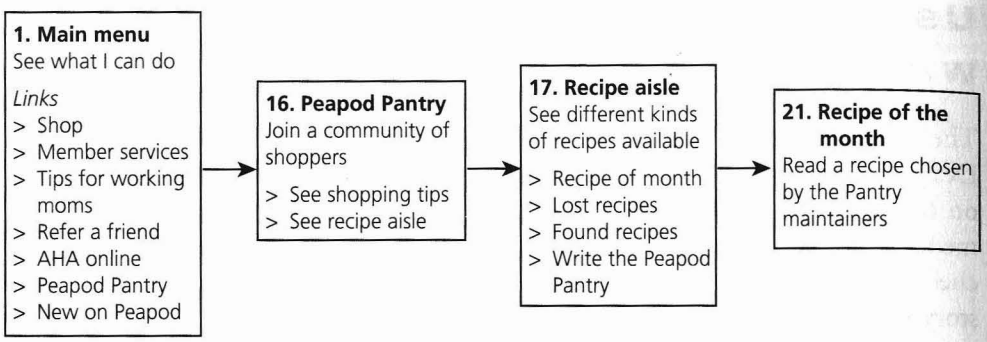



**FIGURE 15.11** When a focus area contains no function, only links to other places, you've got a hallway. Here in Peapod are three hallways in succession (1, 16, and 17) before the user can get to doing anything real. System designers frequently create places that have no purpose except to organize access to other places. They are like hallways in a house, where no actual living is done but doors open onto other rooms. Hallways are necessary in houses because of the physical constraints in laying out a house, but in a software system every place can support real work. This kind of structure is often an indication that the designer is carrying over old ways of thinking from non-GUI systems.

## PROBING USER ENVIRONMENT DESIGN STRUCTURE

The questions to ask when checking a User Environment Design are similar to those that drove building it:

**Are focus areas coherent?** Does each focus area support one activity within the overall task? Is that represented by the title and purpose statement? Be suspicious of any focus area that has no purpose. It's often because the team isn't clear on what the purpose is.

**Do focus areas support real work?** Look for focus areas that are really glorified dialog boxes—they've turned a simple command into a whole subtask (see Figure 15.10). Look for focus areas that group related functions, but that don't support something you might work on. Look for focus areas that don't support a coherent work task, but instead only reveal the data associated with an object in the system.

**Are functions correct?** Look for functions that are not in direct support of the focus area's purpose. Do they imply a separate activity that should be separated into another focus area?

**Are focus areas distinct?** Collect the focus areas that support the same part of the work—the same activity, task, or role—and compare

them. Are they clearly distinct? Do they, taken together, provide coherent support for this part of the work? Can they be recombined to give a cleaner set of distinctions for doing the work?

**Do links make sense?** Do they support the work task as you know it from the consolidated and redesigned models? Certain patterns of links and focus areas always indicate trouble (see Figures 15.10 through 15.12). Do any of these patterns appear, and do they indicate problems in the design? (Incidentally, notice the simplified form of focus area used in Figure 15.10, with only title and purpose. This is a useful way to highlight structural issues.)

**Is the work supported?** Finally, use the consolidated models to refresh your memory, and look at the User Environment Design from the point of view of the different roles and tasks. Does the design work for each different kind of user? Does it account for the issues they care about? Run actual sequences through the model, asking how this user would have done this task given the new system. See if you can make it break down.

Using a walkthrough this way pulls the User Environment Design back together into a structure that makes the user's work coherent. Like a groundskeeper rethinking path layouts, the walkthrough gives you a chance to step back from your design. Check it for fit against the user, for missing parts, and for internal balance. Clean up the structure, and then you can either test it with users or extend it with more storyboards. Or, better yet, do both in parallel—the sooner you get feedback from real users on their design, the better off you are.

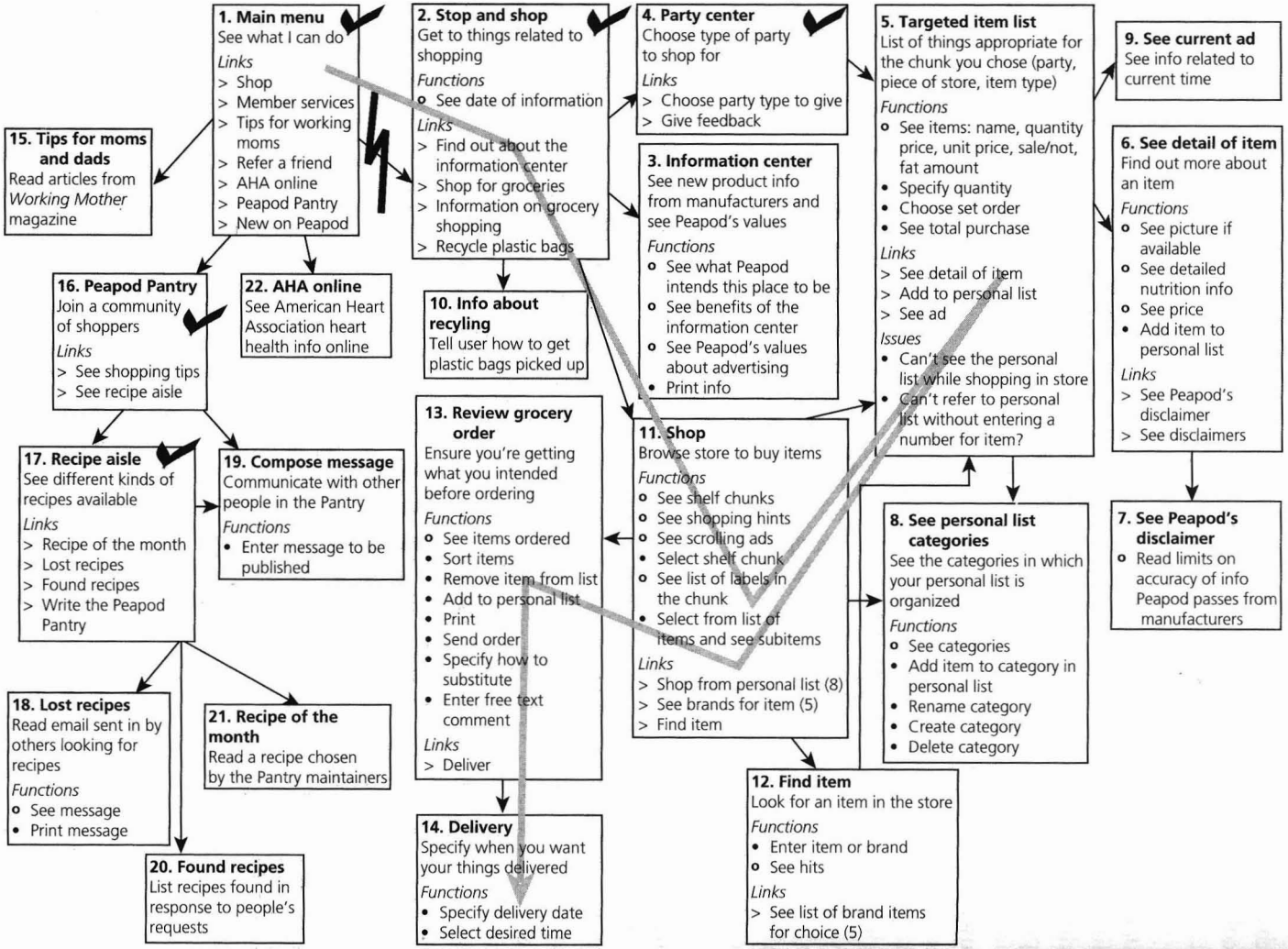

1. Main menu
See what I can do
Links
> Shop
> Member services
> Tips for working moms
> Refer a friend
> AHA online
> Peapod Pantry
> New on Peapod
2. Stop and shop
Get to things related to shopping
Functions
o See date of information
Links
> Find out about the information center
> Shop for groceries
> Information on grocery shopping
> Recycle plastic bags
4. Party center
Choose type of party to shop for
Links
> Choose party type to give
> Give feedback
5. Targeted item list
List of things appropriate for the chunk you chose (party, piece of store, item type)
Functions
o See items: name, quantity price, unit price, sale/not, fat amount
• Specify quantity
• Choose set order
• See total purchase
Links
> See detail of item
> Add to personal list
> See ad
Issues
• Can't see the personal list while shopping in store
• Can't refer to personal list without entering a number for item?
9. See current ad
See info related to current time
6. See detail of item
Find out more about an item
Functions
o See picture if available
o See detailed nutrition info
o See price
• Add item to personal list
Links
> See Peapod's disclaimer
> See disclaimers
7. See Peapod's disclaimer
o Read limits on accuracy of info Peapod passes from manufacturers
15. Tips for moms and dads
Read articles from Working Mother magazine
16. Peapod Pantry
Join a community of shoppers
Links
> See shopping tips
> See recipe aisle
22. AHA online
See American Heart Association heart health info online
10. Info about recyling
Tell user how to get plastic bags picked up
3. Information center
See new product info from manufacturers and see Peapod's values
Functions
o See what Peapod intends this place to be
o See benefits of the information center
o See Peapod's values about advertising
• Print info
17. Recipe aisle
See different kinds of recipes available
Links
> Recipe of the month
> Lost recipes
> Found recipes
> Write the Peapod Pantry
19. Compose message
Communicate with other people in the Pantry
Functions
• Enter message to be published
13. Review grocery order
Ensure you're getting what you intended before ordering
Functions
o See items ordered
• Sort items
• Remove item from list
• Add to personal list
• Print
• Send order
• Specify how to substitute
• Enter free text comment
Links
> Deliver
11. Shop
Browse store to buy items
Functions
o See shelf chunks
o See shopping hints
o See scrolling ads
• Select shelf chunk
o See list of labels in the chunk
• Select from list of items and see subitems
Links
> Shop from personal list (8)
> See brands for item (5)
> Find item
8. See personal list categories
See the categories in which your personal list is organized
Functions
o See categories
• Add item to category in personal list
• Rename category
• Create category
• Delete category
18. Lost recipes
Read email sent in by others looking for recipes
Functions
o See message
• Print message
21. Recipe of the month
Read a recipe chosen by the Pantry maintainers
20. Found recipes
List recipes found in response to people's requests
14. Delivery
Specify when you want your things delivered
Functions
• Specify delivery date
• Select desired time
12. Find item
Look for an item in the store
Functions
• Enter item or brand
o See hits
Links
> See list of brand items for choice (5)

**FIGURE 15.12** The Peapod User Environment. A reverse User Environment Design for a shipping commercial product supporting home shopping. This User Environment Design reveals a number of structural issues in the product. The checked focus areas are all hallways, supporting no real work (they have no functions, only links). The gray arrow shows what's required to order one item, suggesting the design is too leggy. And the lightning bolt shows how the recipe/information part of the product is almost totally divorced from the shopping part of the product (only one link connecting the two parts—and that connects two hallways).

This example shows some strategies for analyzing complex real products. In larger products, major subcomponents—such as the "AHA Online" focus area above—often are represented by only the first focus area in the component. This hides the complexity of the component while still revealing the relationships with the larger system. Each focus area only lists four or five primary functions of that focus area, rather than listing them all. Of course, when the User Environment Design is defining the product, such a diagram has to be backed up with a complete definition of the function of each focus area.

# 16 *Project Planning and Strategy*

Because the User Environment diagram shows all parts of the system in relationship to each other, it's a basis for planning as well as a basis for design. Most systems are large enough that they need a team of people to build them and have to be delivered over a series of releases. Most systems don't stand alone; they work together with other systems to support a whole job or business process. It's this collection of systems that taken together must support a coherent work practice. And software development organizations don't care only about individual applications or products—they're often looking for ways to tie their different systems together into a unified strategy for supporting their target market or business. Such a strategy makes the corporate response we discussed in Part 4 possible.

The challenge for project management is to define releases that keep the user's work coherent *and* can be implemented by the people available in reasonable time. Planning coherent releases can take advantage of the User Environment diagram as a representation of the systems, their parts, and their relationships. The User Environment diagram guides planning by breaking the design into natural components, relevant to the customer, that can be considered independently. Whether these components represent a small part of a single product or a complete application in their own right, the User Environment diagram shows what's going on in that component and how it relates to the rest of the system. Based on this, a team can organize and plan their development strategy.

*Management challenge: define releases that keep user work coherent*

## PLANNING A SERIES OF RELEASES

The usual situation with a systems development effort is to envision a larger and more complete system than can easily be delivered in a single engineering cycle. Whether it's a product for sale or an internal system, customers typically don't want to wait years to see the first version. By then, they'll have taken their business to other vendors, or their business will have changed so much that the system will no longer be useful to them anyway. It's not even good engineering to spend years producing the maximal solution—any system will miss the mark to some degree. The sooner there's a version out there, the sooner the team can correct their mistakes and build on the new work practice that customers will invent around the new system.

It's important to envision the bigger picture. It gives you a goal to strive for, a direction to your development. But use the larger vision to define a series of releases, each leading you closer to the vision and each deliverable in a reasonable time frame. Many organizations aim to have the first release out in under a year, even for significant projects. This release sets the customers' first impression of the system and organization that delivered it. The system should make a splash in the market or make a significant contribution to the customers' business. But it also needs to hang together as a coherent way of working. Every function interacts with other functions in the design—it's a waste to do large amounts of work to ship a function and none to ship the other functions that make it useful. The last-minute sessions to decide exactly what will make it into a particular version are the most painful. What's the criteria for choosing what to cut? The last feedback from a user group? The most recent customer to call an account representative on the carpet? Whoever shouts loudest on the engineering team? Teams need a process for deciding what functions are most important for the work of the customer and how to deliver them in chunks that keep the work coherent.

*The UED showing the whole system lets you see how to carve it up*

When delivering to an internal client, basing development plans on a long-range vision creates the possibility of integrating the development schedule with the organization's business plans. Each piece of the User Environment Design suggests new roles and new ways of working—as each piece is implemented, the organization can put the

process changes in place needed to take advantage of that piece. Keeping the business reorganization and system delivery synchronized keeps the process under control.

When delivering a product to a market, creating a larger vision and delivering to it over a series of releases means you have a coherent market message. Reveal the vision as your strategic direction, and then each release is not only useful in its own right but is also another down payment toward your commitment. Instead of selling individual features, you can sell product directions that address problems people experience in their work. Everyone—marketing, sales, services, and development—can push their work in this common direction.

Focus areas and the clusters of focus areas that together address a common intent are one way of looking at how the functions of a system group into subsets that can usefully be shipped together. In Figure 16.1, the "Select base configuration" and "Find configurations" focus areas together let the user view, search for, and select a configuration to use as the starting point for any changes. These two focus areas work as a unit. It wouldn't make sense to ship one without the other. However, the system could ship without both—developers would then have to type the name of the base configuration directly. This might be reasonable for a first cut at the system.

*Focus areas work together to support tasks or roles—ship them together*

Another way to prioritize the system is to deliver coherent support for a role, responsibility, or task. The first responsibility of a configuration management system is to support developing code, so the marketing team might decide that the first release has to support developers. "Modify product" is the core focus area for developers, so it had better be included. A minimal release would include just enough other focus areas and functions to support making a change: choosing a base configuration to modify, choosing the specific "parts" to edit, making changes and testing them, and finally packaging up all modifications into a single "change," which is put back in the system.

Once they make the decision what to include in the system, the team makes a *shipping User Environment Design* showing just those focus areas and functions that are intended to be part of this release (Figure 16.2). This shipping design, when all focus areas and functions are fully specified, forms the core of the software specification for this release. By extracting the subset they intend to ship and representing it

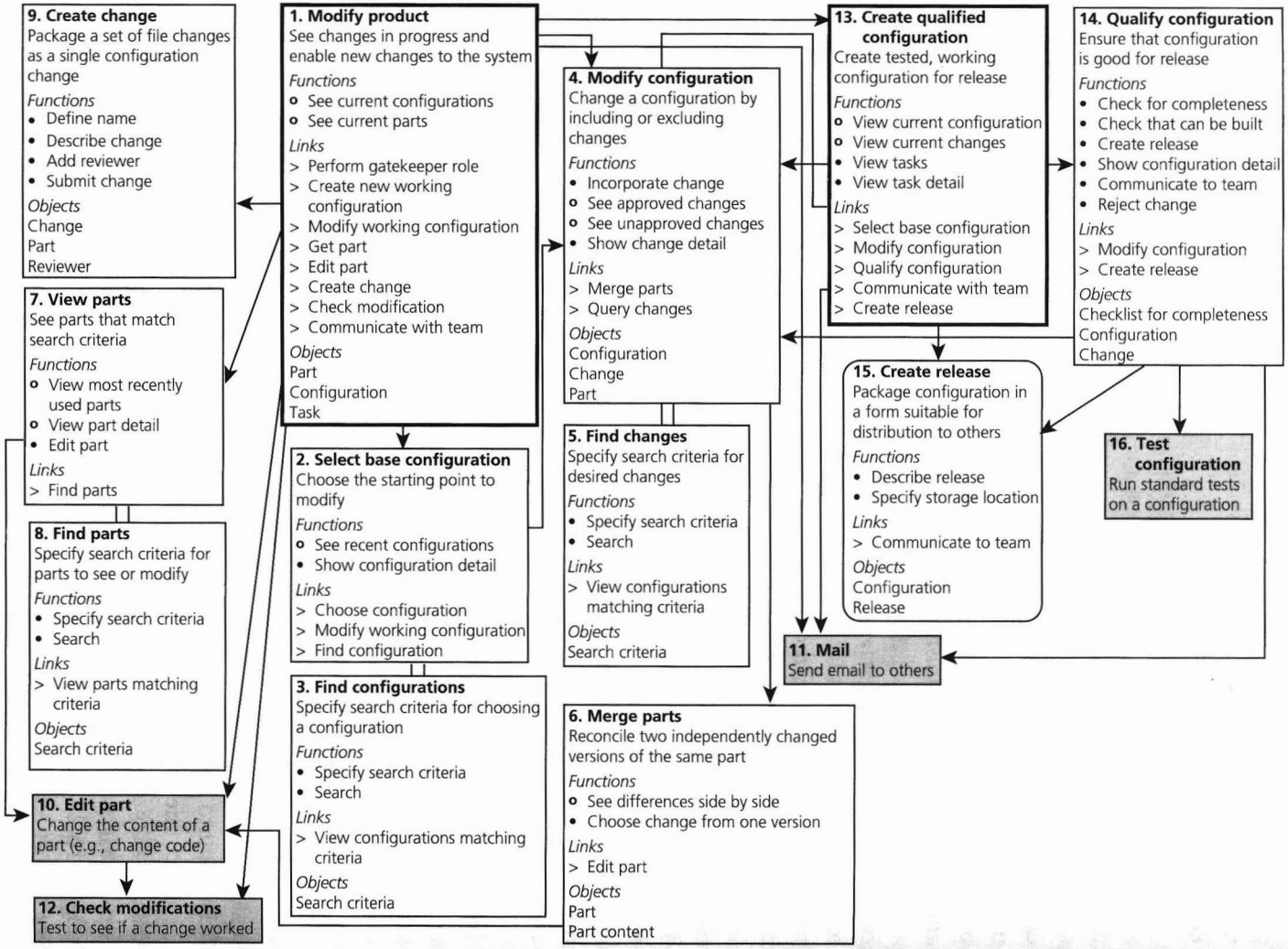

9. Create change
Package a set of file changes as a single configuration change
Functions
• Define name
• Describe change
• Add reviewer
• Submit change
Objects
Change
Part
Reviewer
1. Modify product
See changes in progress and enable new changes to the system
Functions
o See current configurations
o See current parts
Links
> Perform gatekeeper role
> Create new working configuration
> Modify working configuration
> Get part
> Edit part
> Create change
> Check modification
> Communicate with team
Objects
Part
Configuration
Task
4. Modify configuration
Change a configuration by including or excluding changes
Functions
• Incorporate change
o See approved changes
o See unapproved changes
• Show change detail
Links
> Merge parts
> Query changes
Objects
Configuration
Change
Part
13. Create qualified configuration
Create tested, working configuration for release
Functions
o View current configuration
o View current changes
• View tasks
• View task detail
Links
> Select base configuration
> Modify configuration
> Qualify configuration
> Communicate with team
> Create release
14. Qualify configuration
Ensure that configuration is good for release
Functions
• Check for completeness
• Check that can be built
• Create release
• Show configuration detail
• Communicate to team
• Reject change
Links
> Modify configuration
> Create release
Objects
Checklist for completeness
Configuration
Change
7. View parts
See parts that match search criteria
Functions
o View most recently used parts
o View part detail
• Edit part
Links
> Find parts
2. Select base configuration
Choose the starting point to modify
Functions
o See recent configurations
• Show configuration detail
Links
> Choose configuration
> Modify working configuration
> Find configuration
5. Find changes
Specify search criteria for desired changes
Functions
• Specify search criteria
• Search
Links
> View configurations matching criteria
Objects
Search criteria
15. Create release
Package configuration in a form suitable for distribution to others
Functions
• Describe release
• Specify storage location
Links
> Communicate to team
Objects
Configuration
Release
16. Test configuration
Run standard tests on a configuration
8. Find parts
Specify search criteria for parts to see or modify
Functions
• Specify search criteria
• Search
Links
> View parts matching criteria
Objects
Search criteria
11. Mail
Send email to others
3. Find configurations
Specify search criteria for choosing a configuration
Functions
• Specify search criteria
• Search
Links
> View configurations matching criteria
Objects
Search criteria
6. Merge parts
Reconcile two independently changed versions of the same part
Functions
o See differences side by side
• Choose change from one version
Links
> Edit part
Objects
Part
Part content
10. Edit part
Change the content of a part (e.g., change code)
12. Check modifications
Test to see if a change worked

**FIGURE 16.1** A User Environment Design supporting configuration management. This design supports two primary roles: Developers, who code changes and extensions to the application, and Gatekeepers, who ensure that any new code is good enough to include in "official" versions. Each specific application version is represented as a "configuration"—the set of specific file versions for that variant of the system. Developers work by selecting a "base configuration" to modify and making their changes as changes to that configuration. When they are done, they package all files they've changed along with the original change order and any assembly instructions and submit that package as a "change." The Gatekeeper can then review and test each change as a coherent unit.

as a User Environment diagram, the team can see whether that subset stands on its own as a self-contained system supporting coherent work. They can validate it, run scenarios through it, prototype it, and test it with users. They can find out both whether it works as a coherent system and whether it works as an interesting release—whether it provides enough to make customers interested in adopting it.

Reviewing the User Environment Design in Figure 16.2 suggests that a cut supporting only development provides only minimal support for one role—hardly a competitive product, and with no extra features for product differentiation. So marketing might decide that supporting the Gatekeeper is a requirement for a viable product. It's a fundamental responsibility of the Gatekeeper's role to review a configuration and decide whether it's good. So there's no point in shipping any of the focus areas supporting the Gatekeeper if the system doesn't include "Qualify configuration." It won't be used by Gatekeepers if it doesn't support that part of the role.

*Ship the functions needed by the role or task you intend to support*

It may not take all the functions of a focus area to support a role. Only some of the functions of "Modify configuration" are needed by developers, so if they are the targeted users for a first cut of the system, the other functions could be left out. When a role is the target for a release, looking across focus areas for the core function to support that role reveals what's most important to include.

Finally, because the decision of what to cut is an engineering tradeoff that has to account for implementation difficulty, the team can consider alternative presentations of a function or focus area in the UI. The UI can make the function easy or complicated to implement. For those functions core to a focus area or to a role, it may be worth designing a sophisticated UI that makes the operation of the function smooth and easy. But for less central functions that are nonetheless needed to support the work, a bare-bones implementation may be sufficient.

*Identify your core contribution to your customers and ship that*

All these ways of looking at how to prioritize a release depend on understanding what the core innovation of the new system is. What is the one key change in people's work practice that the system introduces? Don't look for a feature; look for the key way in which the system makes work different. Look for the key differentiator your product offers over the competition or the core way your system

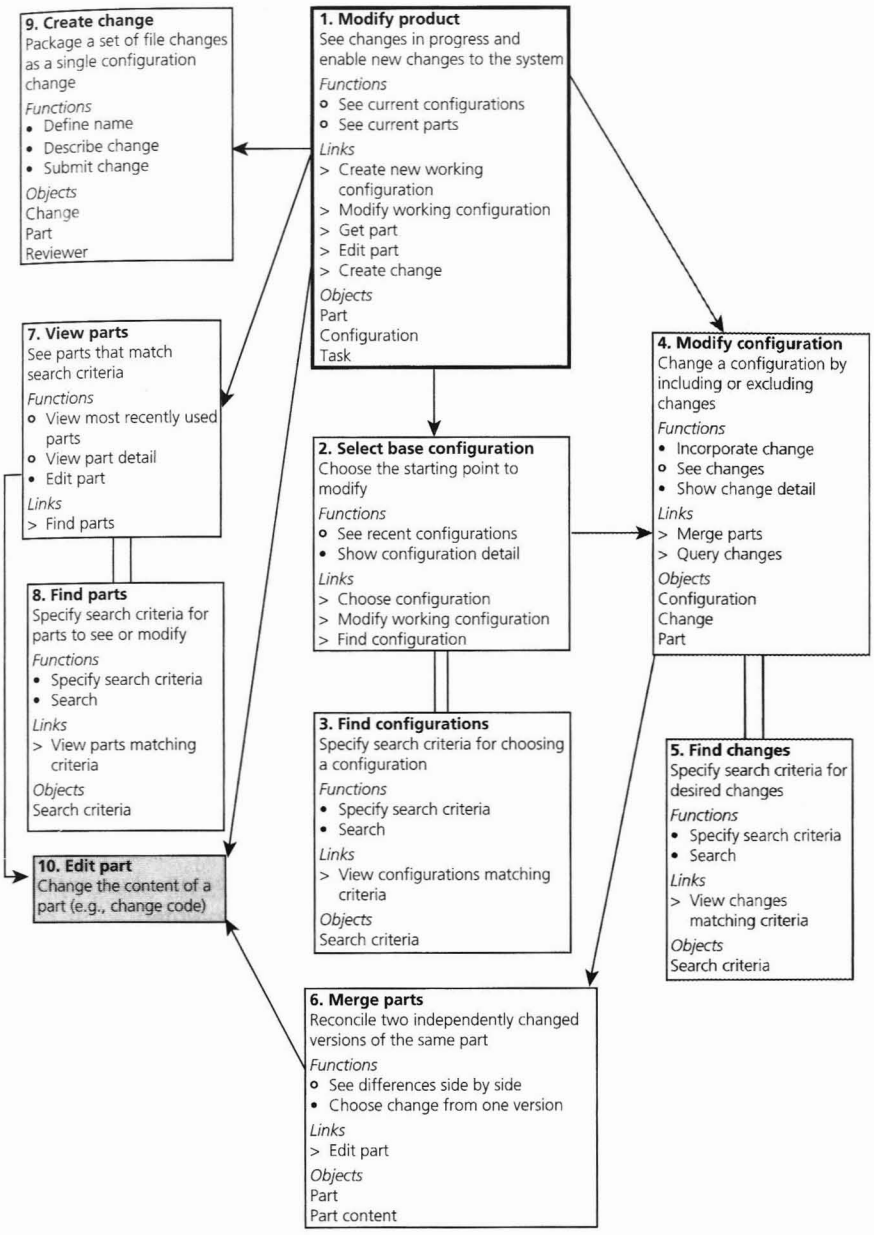



**FIGURE 16.2** A shipping User Environment Design: a subset of the configuration management User Environment Design supporting developers.

helps your customers advance their business goals. Once you've identified the key differentiator, ask, What's the minimum subset of the system necessary to introduce that change? The User Environment Design helps to maximize the impact of a new system by showing what part of the system will implement the core innovation coherently. Then you can build on that to support more of the work, more completely.

## PARTITIONING A SYSTEM FOR IMPLEMENTATION

Real systems are built by more than one person—by teams working together. If dividing up the system is to be useful, every developer must be free to focus on his or her own part. But any requirements document has holes—developers in front of their machines at 2:00 in the morning will have to make decisions that affect the user. With the User Environment Design, those decisions can be made with the knowledge of how it affects the overall design and other design teams. The User Environment Design organizes requirements to show how the system is structured for the customer. But the User Environment Design also helps manage a project by showing how it can be split up for implementation by teams or individuals working in parallel.

*The UED maintains work coherence during implementation*

The concepts of the User Environment diagram can help a team keep the coherence of user work during implementation. Assign work purely based on technology or implementation considerations, and each developer may not have a coherent piece of the work to code. That will lead developers to lose the focus on the customer. They can't see how the work is supposed to hang together, so they have no way of knowing if a decision they make disrupts the flow of work or supports it. Each focus area represents a coherent concern. It makes sense to assign whole focus areas together, or sets of focus areas addressing a role or task, so that developers can see one complete piece of the whole. If the User Environment Design can be broken into components, as above, whole components can be assigned together. People can think about and design these coherent units as a piece because they hang together in the system and in the work (Figure 16.3).

An implementation subteam needs a coherent part of the system to design, and focus areas provide such an organizing theme. But a large system will have multiple subteams, and some may be organized around components of the implementation such as common objects or technology components (e.g., the interface to a database). Organizing a project for delivery is a balance between keeping the user's work in the system coherent and keeping the implementation coherent.

For any system to work, the teams focused on implementation components need to understand how they relate to user-visible behavior. A team implementing a reusable component to embed video clips in mail messages, for example, would have to understand how the activities of reading and sending mail are structured so that the reusable component can fit into the work smoothly. It would be important to them to see as many different situations of use as possible, to understand the requirements on the component and how to make it a seamless part of the host system. A team implementing a reusable component to support an underlying database link would need to know what kinds of demands the systems might make on that link.

*The UED reveals reusable components*

The User Environment Design reveals how reusable components relate to the system work model and shows who needs to work with whom. A team building an object class to implement a work object from the User Environment Design needs to work with the teams building the focus areas where that work object is used. The teams working on focus areas need to manipulate the objects and have a stake in the design of the class. In the UI, the object is a visible screen artifact and should appear consistently in all parts of the system. The User Environment Design flags all the players who need to be concerned about these elements so they can agree. In this way the whole team—even those working on internals—stays grounded in the user's work. And the User Environment Design provides a map of which development teams need to work together.

Links between focus areas assigned to different subteams show points of integration. A link shows that one part of the product needs to provide access to another part and that the work flows from one part to the other. The parts need to connect technically, so some kind of call or invocation mechanism needs to be provided. This might be through the underlying platform (moving a mouse from one window

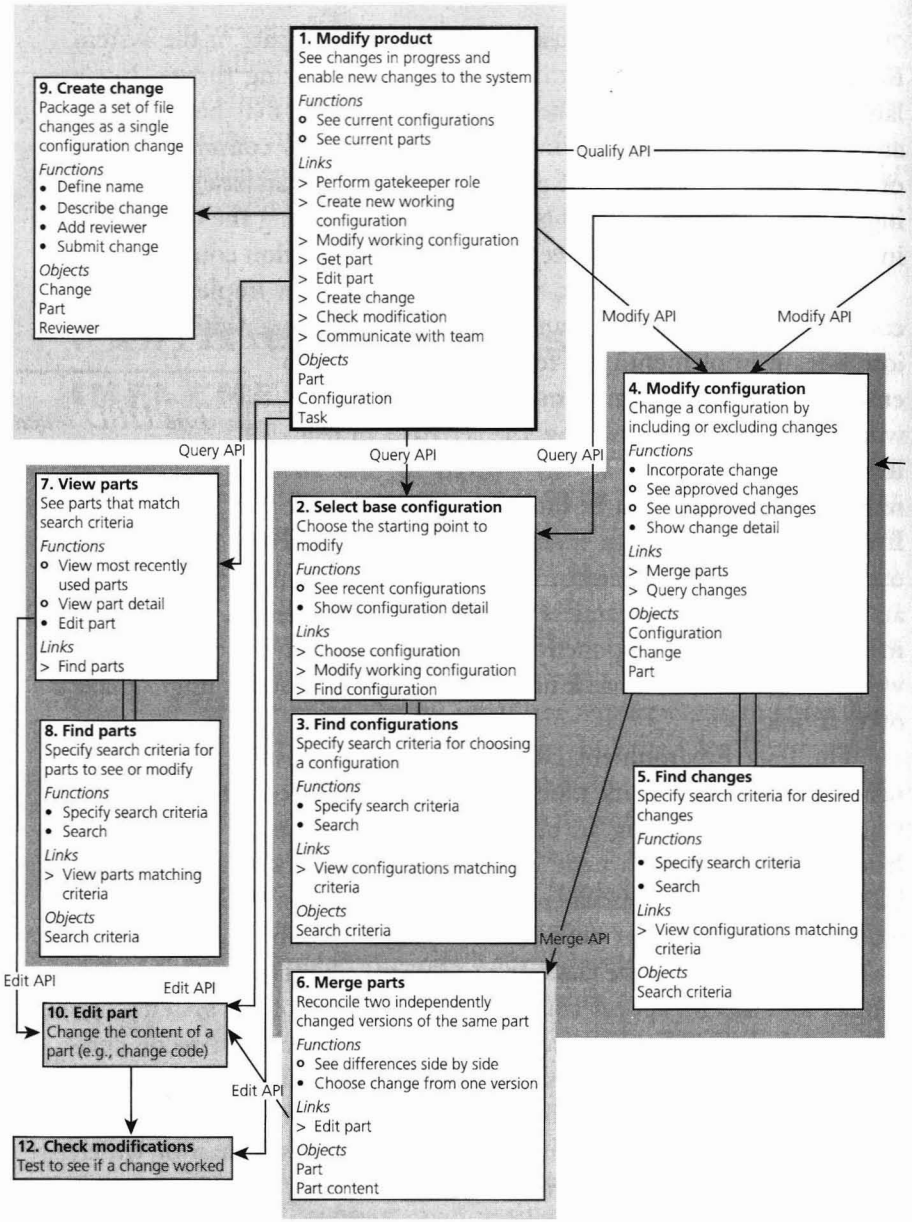
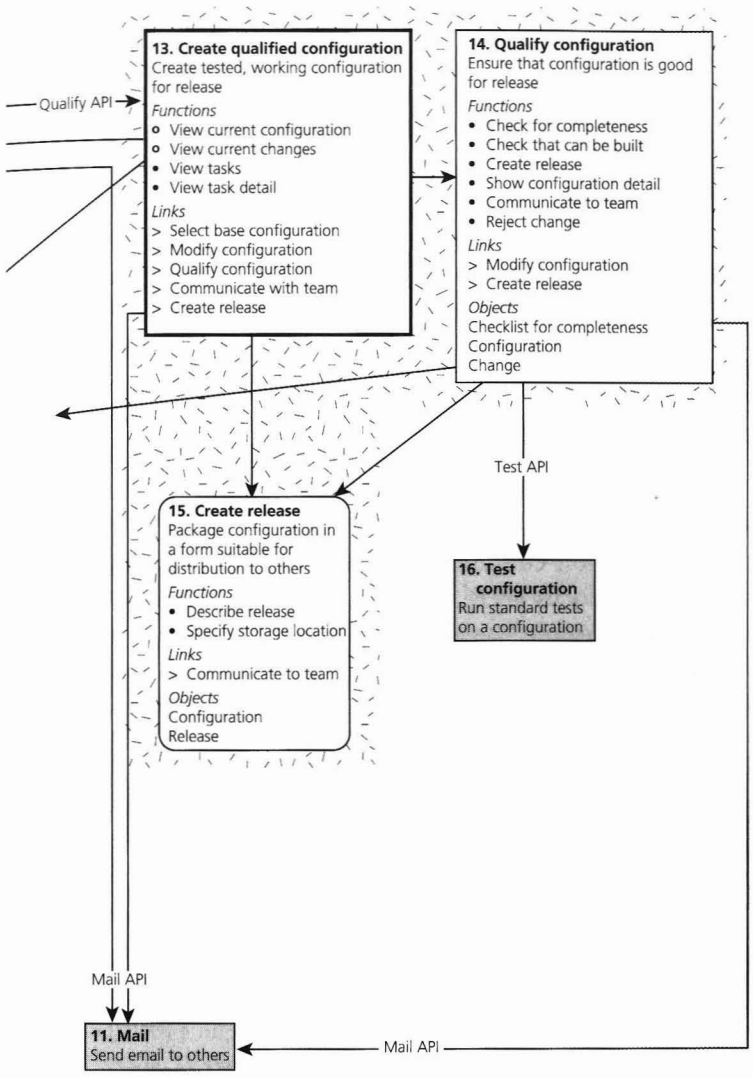



**FIGURE 16.3** The configuration management User Environment Design annotated to show implementation details. The different patterns and shading show how the focus areas have been assigned to implementation teams: one team has the Developer's parts, another has the Gatekeeper's, and a third has all the "Find/View result" linked focus areas. Because these focus areas all present the same interaction style, it made sense for one team to implement them all.




The User Environment Design has also been annotated to show internal application programming interfaces (APIs) that the different implementation teams have to agree on. There will be a standard Query API, for example, that will be consistent across the "View/Find" focus area pairs. This Query API will be used whenever a part of the system wants to present an interface that allows the user to search for and choose an element to work on. Similarly, the User Environment Design shows external APIs that the team has to conform to—the Mail and Edit APIs.

to another), by using standard application integration mechanisms (OLE or CORBA), or through special APIs. But not only do the software components need to access each other, but the user needs to feel like it's one consistent system. It's important that the system have a consistent appearance and behavior across the focus areas that are linked and used by the same people.

*Use the UED to help coordinate implementation teams*

The User Environment Design helps keep teams from becoming myopic and overfocusing on one situation. The "Modify configuration" focus area is used both by Developers and Gatekeepers. If it were given to the subteam implementing the Developer's part of the system, they could easily overemphasize the Developer as their user. The User Environment Design reminds them they support two roles, two kinds of tasks, achieving two separate intents.

*Use the UED to help developers see their part in the whole*

UI implementation considerations may also guide the assignment to teams. If several focus areas need a particular technology—such as natural language query mechanisms in "Find configurations" and "Find parts"—it's natural to assign all these focus areas to the same subteam so they can work out the solution once and apply it everywhere needed.

By showing the structure of the system, the User Environment Design provides a map to the implementation. Just as electricians can use a floor plan to talk to carpenters about how to locate the holes inside the walls so that the users can get their outlets where they want them, so the User Environment supports a conversation between the parts of an implementation team about how to deliver the system. It splits up the implementation into coherent units, shows how they relate to each other, and shows how teams focused on internals need to coordinate with the rest of the project. (Hsia et al. [1996] suggests another approach to sectioning a system for delivery.)

## Coordinating a Product Strategy

More and more, both internal organizations and software product companies are shipping sets of applications, each supporting a different aspect of the user's work. More and more, these organizations

are looking to tie these point solutions together, so they provide seamless support to the work while still being packaged as separate products. Or they're looking to support a new market or process that they've never addressed before and will need to address with a suite of cooperating applications. This is hard to do, especially when starting from multiple existing applications. It's difficult to put the essentials of each application out next to the others to see how they could relate.

A large User Environment model can show how a set of existing applications combine to support the user's work. Extract such a model as a reverse User Environment Design going application by application. It's usually not necessary to do a full model—representing focus areas, purposes, and flows (as in Figure 14.5) is enough to see the structure without getting overwhelmed. Use a validation walkthrough of the resulting model to look for all the ways your current product set fails to deliver a coherent system work model: all the missing links between components, duplicated functions, missing functions, and inconsistencies. Then collect data on the systems in use to see how the work hangs together in practice. Identify changes to the User Environment model that address the problems. Use the structural principles for a User Environment Design to guide these changes, and use the work models to see where the current design falls down and how to fix it. When you have a new User Environment Design, showing how your existing suite should be modified to provide a coherent solution, you're ready to decide how to change the applications.

*A reverse UED ties existing unintegrated systems together*

The links between each part of this new design show integration points, where the applications need to share data or support the user's transition from one kind of work to another. The work objects that appear in different parts of the User Environment Design show key points for data integration across the different systems—these are the objects that will need common definitions, common storage, and common UI. Each project can define a plan for moving to the design specified by the User Environment over one or more releases. Build a shipping User Environment Design showing the first release for all projects, and you'll be able to keep them synchronized.

*The UED reveals points of integration*

Building a systems strategy when there are no existing applications is actually easier. You can design the overall system directly, from storyboards, like a single system with a wide scope. Then use the User Environment Design to identify good places to partition the system into applications. Each application should support a coherent part of the work or role and have clean interfaces with the rest of the system. Once you've partitioned the system, the links across partitions and common work objects identify integration points.

*Drive acquisition from the specification in the UED*

IT shops can use the User Environment Design to identify not only the parts they will build, but also the parts they want to buy from vendors. The User Environment defines requirements for the acquisition, showing what it must do, how it must be structured, and how it must fit with the other IT systems. IT development teams have done this—in one case, they designed their desired solution directly from a vision and storyboards, representing it in a User Environment Design. Then they brought vendors in, showed them the User Environment Design, and invited them to bid on delivering it using their products. The vendors had to prove they could customize their system to support the structure and functions specified in the User Environment Design. They chose the vendor who was most successful at showing that, with reasonable modifications, they could support most of the design the team had specified.

*The UED helps projects cooperate*

The User Environment is a model that enables the project teams to talk to each other about where system boundaries should lie, how to create the bridges between systems necessary to support the work, and how to assign and reuse implementations of common system functions. Teams use their User Environment Designs as an artifact to talk over in coming to an agreement on the relationship between groups. In one case, two teams laid out their respective User Environment Designs to support their discussions and ended by canceling one of the projects—the diagrams had made the overlap so obvious that they couldn't justify the existence of both.

## DRIVING CONCURRENT IMPLEMENTATION

The User Environment Design defines the structure of a part of the overall vision—the part that is instantiated in software (and possibly some hardware). It provides the next level of detail about the vision by working out the system work model for the vision. Just as the vision guides different groups in creating a single corporate response, the User Environment Design guides different groups in delivering parts of the system and associated processes in parallel, yet in coordination:

**Planning process:** The User Environment Design represents the system work model and can be used to support planning business processes. Once you've worked out the coherent units of work in focus areas, you've also laid out coherent chunks for a business process. Defining the process and defining the User Environment Design go hand in hand. You can walk through the new process to see how it's supported in the system. Out of this, identify problems with the system or process, what training needs to be developed, and how to introduce the new way of working to minimize disruption.

**Implementation:** The User Environment Design specifies behavior without specifying the user interaction mechanisms. An implementation based on the User Environment Design will be free of bias toward one UI over another. When the UI is designed, it can be hooked to the underlying implementation so that there's a clean separation between the UI-specific code and the code that implements behavior—a cleanly layered implementation. The basic function of the system is defined in the functions and objects of the focus areas. As an additional guide to the implementation, annotate the User Environment Design with implementation constraints—for example, the required speed of following a link or the constraints on size or access time of a focus area.

**Documentation:** The User Environment Design specifies the function of the system so documentation can start to describe what it does. Furthermore, the specification of coherent focus areas, each with a defined purpose in supporting the work, gives documentation writers a clear structure and motivation to communicate. The User Environment Design reveals opportunities for additional user services,

such as help desk support, training seminars, and follow-on consulting, and provides the information needed to plan them.

**Test plans:** The combination of storyboards and User Environment Design provides the information necessary to start test plans. The storyboards show how the system should work; the User Environment Design provides the formal definition of the functions. It's straightforward to build a test plan that checks these statements of the plan against the actual system.

*The UED ensures cross-functional teams deliver a coherent work practice*

Because the User Environment Design is focused on the system work model—the system as experienced by the user—it gives a way to structure and think about the system that keeps the system work model coherent. The chunks of the User Environment Design map to chunks that designers need to think about and design as a unit. As such, it's a natural structure for presenting system requirements. Implementation has its own coherence, which will come later and which may be represented in an object model. But the structure of an implementation is less useful for planning a customer-centered project than the structure of the system work model. Thinking about the system work model—the User Environment Design—ensures that the parts of the system and the components that are delivered are coherent from the user's point of view. That's the key value of the User Environment diagram in planning: it ensures you don't lose coherence for the user in the turmoil of getting to the implementation.

### MARDELL'S STORY

In the early days of WordPerfect, the founders of the company worked in the same building as their users. It wasn't a problem to stay close to their customers. But as the company grew, developers lost that immediate connection. So the company decided to put together a strategic effort to decide on the direction of the WordPerfect product and recover that immediate sense of the customer.

We put together a team of four or five developers, a marketing person, a documentation writer, a UI designer, and a usability specialist. This was simply not done in the company at the time—design was driven by engineering, with marketing getting involved later. ➪

We used the complete Contextual Design process in this team to build a picture of our market and design new product directions to address the market better.

The new designs we came up with were well received in the company, but suggested changes beyond WordPerfect itself. So we split into two teams: one focused on broad strategic issues across the product set, and one focused on improvements to WordPerfect. This required both teams to refocus and redefine their mission.

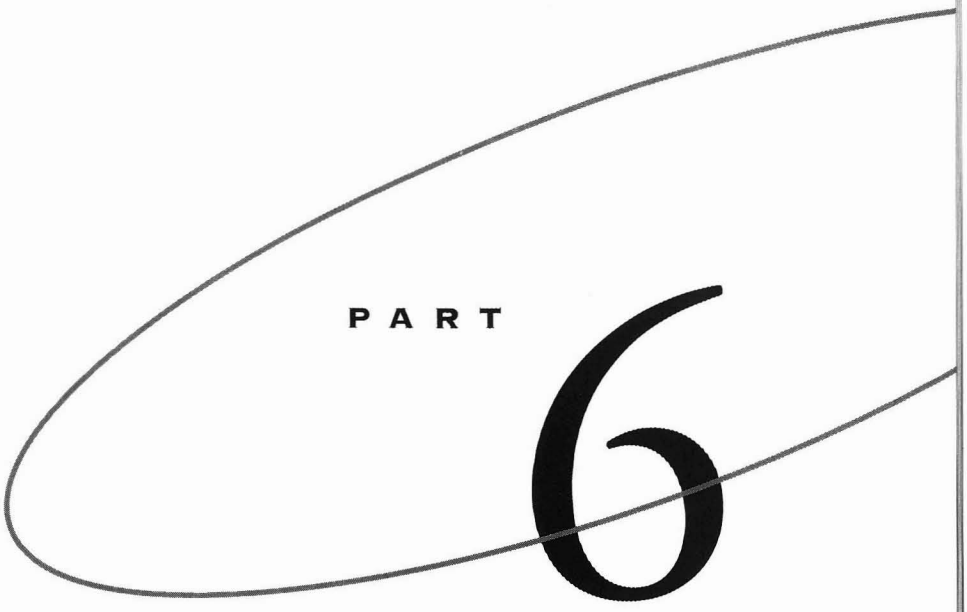
I led the team focused on WordPerfect. We found that the most important parts of the process for us were interviewing, sequence model analysis, visioning, and paper prototyping. The strategic team continued to use the User Environment Design to show the company's different products and how they related to each other. But we had the strategic direction from the first round: we were focused on developing one product and concentrated on one focus area, WordPerfect's editing window. We didn't need the User Environment Design for that.

As we worked with the process, we recognized that it was a backbone for understanding the customer that could incorporate different activities. We started to use it less rigidly than we had. We decided we weren't creative enough and started to incorporate other techniques to expand the possibilities in our visioning. We found that our customer insight helped us make better use of enhancement requests in our customer feedback database because we understood more of the context of a request. We did a teardown of the current WordPerfect product and competitors to find places for improvement. And we analyzed other kinds of products for ideas as well.

Early on, we had a choice whether to focus on short-term improvements or long-term new directions. Since we did have critics in the early days, we decided we had to show concrete results. We developed specific ideas, prototyped them in paper and in code, and got them into WordPerfect 6.1. When these features were the ones reviewers and customers picked out as being the important innovations of the product, our credibility went way up.

Though we conceived of our ideas as integrated product directions addressing whole themes in the customer's work, we found it easiest to communicate them to developers as features. We would select features to push out together, to address some aspect of the customer's work. We did a lot of one-on-one work with developers, showing them prototypes of a new idea and taking them along on customer visits. (We've tried to make sure every developer goes on at least one customer visit.) We transfer ownership of the idea to the developer—they get credit for refining it and making it real.

WordPerfect's been sold now—several times—but that was an amazing team, and the ideas we developed are still important to the product. ❑




# Prototyping

# 17 *Prototyping as a Design Tool*

We call this a customer-centered process, but it's been quite a few chapters since talking to customers was the main activity (not counting any customers on the design team). The activities covered in the last few parts have focused on the customer—understanding how they work as individuals and the common structure of their work, visioning new ways for them to work, and designing those ways into a software system. These activities keep the customer's work practice coherent and use customer data as the final arbiter. And the consolidated models and vision suggest holes in the team's knowledge of the customer, which they fill through additional interviews. But it's now time to get direct customer feedback again.

One of the difficulties with explaining any process is that each part of the process must be described in turn, and the explanation itself takes up time. The description unrolls the process and lays it out, making it possible to see and examine each part, but also making the process appear more sequential than it is. In fact, the period from the beginning of consolidation to the first prototype interview should be no more than a month, even for complex systems, and the team gathers additional data to fill gaps in their understanding and inform the vision during this time. For smaller projects, this period may be as short as a week or two. The point we have now reached in the process is the norm for a design team: with a base understanding of their customers and a target vision for their design, they extend and iterate their design with customer feedback. Iterating with prototypes is a design tool ensuring that the team builds the right system, that the structure fits the user's work, that the

*The goal: continuous iteration and extension*

detailed structure internal to a focus area works, and that the user interface is usable and reveals the structure clearly. Teams that get bogged down in design are usually those that have lost touch with their customers—that aren't going back out to interview or test prototypes on a regular basis.

The most basic attribute of a customer-centered process is that the customers are the final arbiters of what works and what doesn't. When you create a design, captured in a User Environment diagram, that design is really a claim about what will work for the user. The claim is that this particular system simplifies the user's work, overcomes pain, or otherwise improves their work practice. So, how do we test this claim? How do we find out where the design falls short and how to improve it? How do we communicate the design to users in a form that they can respond to—in a form that helps people see the consequences of different design decisions and react to them clearly?

*The customer is the final arbiter of the design*

## THE DIFFICULTY OF COMMUNICATING A DESIGN

Most of the approaches commonly used to communicate a new design downplay the difficulty of communicating a design. Think about it—it's a conceptual nightmare. Consider presenting a demo of the proposed new system to potential customers in a conference room: they must view the product's user interface, understand from the interface and the verbal description what the product does and how it is structured, apply that implicit structure to their own work practice (which is also unarticulated, as we established in Chapter 2), envision how their work practice will be restructured in the presence of the new system, imagine themselves living in this new way, and decide whether they like it. Then, if they don't, they have to imagine some better way to work, transform it into implications for the design, and express those implications clearly to the designers. The task is overwhelming. It's no wonder most people complain about an icon that confuses them, comment on the color, and ask about one or two key features they care about.

*Demos and specifications can't evoke work practice*

Requirements specifications fare no better. Most are text; most break the system down into categories that relate to the system, not the user (all reliability requirements together, for example). Even when the first level of organization is by UI component, their textual and list-based nature tends to present features in isolation. It's hard even for designers to see how a feature relates to other parts of the design; internal users reading the requirements for sign-off find it even harder. Requirements specifications are less approachable than a demo and make it no easier to imagine the impact that the proposed system will have on users' work. They may have their place in specifying exactly what's in the system, but they aren't a good way to communicate a design.

*The challenge: communicate the experience that a new system will offer*

Talking to the customers with models has a similar set of drawbacks. Process models or object models introduce a new language, which must be learned and understood by the users if they are to participate in the discussion at all. The models represent either facts about their work or the new system. But their work is unarticulated, and the models represent it in a strange and unfamiliar language that offers no touchstones to their experience. In Contextual Design, we don't even try to talk to customers with our work models, unless we're building systems for an internal business partner, and they have an interest in representing their own work practice explicitly. Then the work models become a tool for the whole department to think about how they work and maintain an ongoing conversation about how they might improve it. When customers think models are a tool for them to manage their business, they can learn to use them in the way that designers use them; otherwise it's too hard for them to see how they map to reality.

*Models introduce a new language for customers to translate*

Other forms of communication such as use cases and scenarios attempt to communicate more of the context of use. These methods tell stories of how people will work in the new system, so they communicate better than a model or specification. However, each scenario can only tell one story out of the many the system must support. And they all suffer from the same basic drawback: most customers have only an unarticulated knowledge of their own work and cannot check a proposed design against their own experience unaided. They can react to such a story at the level of "I hate that" or "I love that," so